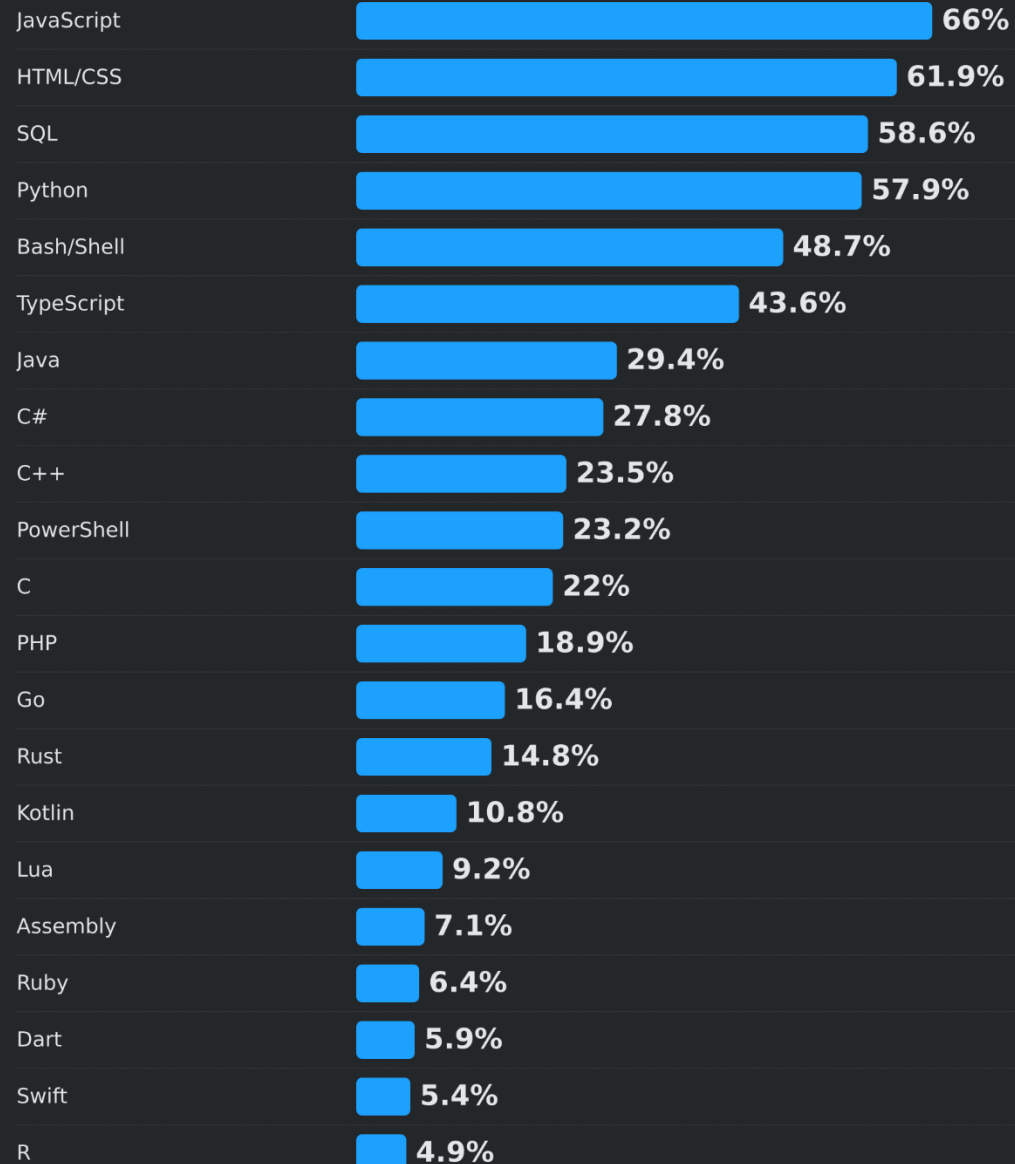


JavaScript เบื้องต้น



Most popular technologies / All Respondents

Programming, scripting, and markup languages



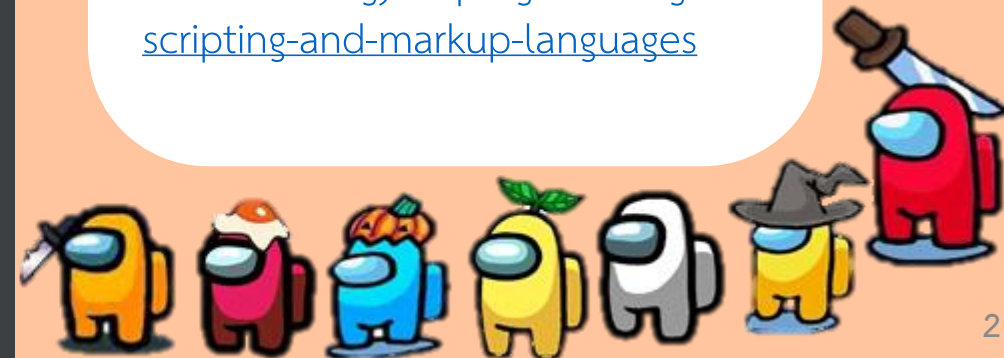
JavaScript is the most popular programming language



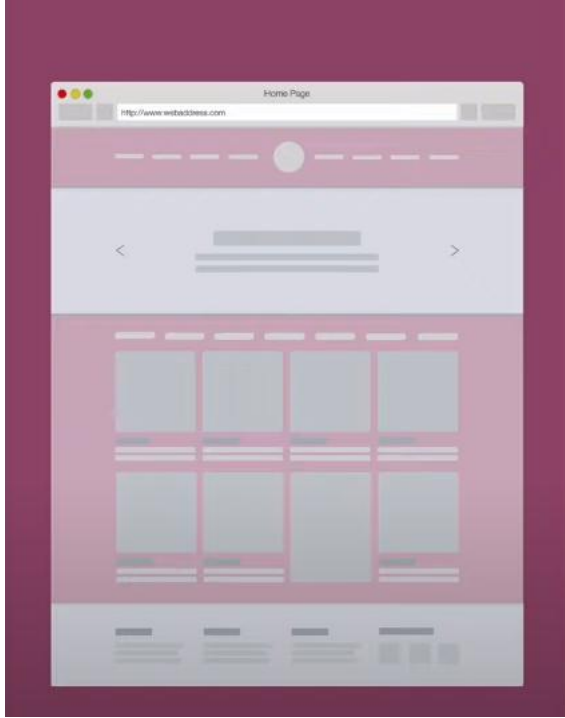
จากการสำรวจของเว็บ stack overflow แสดงให้เห็นว่า JavaScript เป็นภาษาที่ได้รับความนิยมที่สุดอันดับ 1 สำหรับนักพัฒนา 13 ปีซ้อน (ข้อมูลวันที่ 19 ส.ค. 2568)

ที่มา:

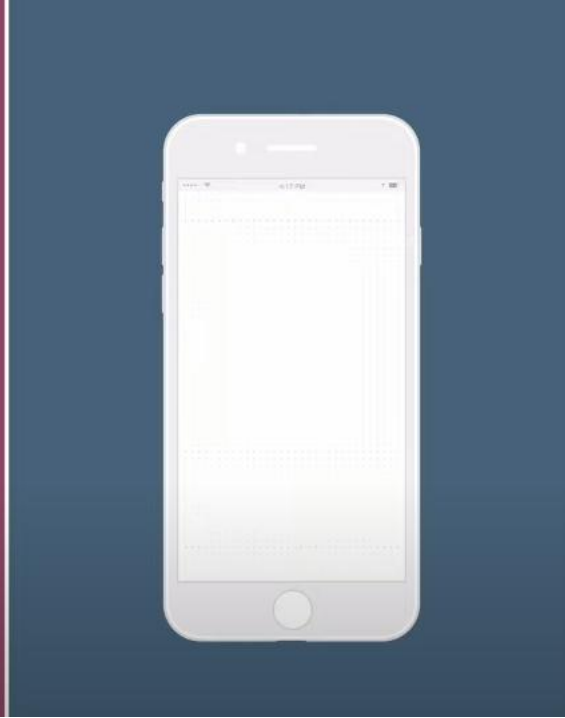
<https://survey.stackoverflow.co/2025/technology#1-programming-scripting-and-markup-languages>



เมื่อรู้ JavaScript สามารถใช้ความรู้ต่อยอดไปสร้างอะไรได้บ้าง



ใช้สร้างเว็บไซต์



ใช้สร้างแอปพลิเคชันในมือถือ



ใช้สร้างโปรแกรมหรือแอปพลิเคชัน
ในคอมพิวเตอร์ ตัวอย่างโปรแกรม
ที่ใช้ JavaScript สร้างขึ้นมา เช่น
Vs code, Atom

ตัวอย่างที่ใช้ JavaScript ในการสร้าง คือ
Facebook, Instagram, Netflix, twitter



<https://www.youtube.com/watch?v=Tz5wk1yPIBE>

พื้นฐาน JavaScript

- JavaScript คือ?
 - การประกาศตัวแปร
 - ประเภทข้อมูล (Data Types)
 - ตัวดำเนินการ (Arithmetic Operators)
 - เงื่อนไข (Conditions)
 - ฟังก์ชัน (Functions)
 - การวน loop
 - continue และ break
 - DOM
- เพื่อให้ผู้เรียนสามารถนำไปต่อยอดเรียนรู้ไลบรารี เฟรมเวิร์ค เช่น React, Vue, Angular, และ Node.js เป็นต้น



ภาษา JavaScript คืออะไร



- **JavaScript** หรือย่อ **JS** เป็นภาษาคอมพิวเตอร์ high-level programming ที่ใช้สร้างและพัฒนาเว็บไซต์ที่กำลังได้รับความนิยม ถือว่าเป็นเทคโนโลยีหลักของการพัฒนาเว็บไซต์ (World Wide Web) โดยจะทำให้หน้าเว็บไม่จำเป็นต้องรีเฟรชหน้าใหม่ในขณะที่ยังโต้ตอบกับผู้ใช้งาน (Dynamic website) การทำงานของ JavaScript เกิดขึ้นบนเบราว์เซอร์ (เรียกว่าเป็น client-side script)
- JavaScript เป็นภาษาเขียนที่ถูกพัฒนาและปฏิบัติตามข้อกำหนดของ ECMAScript (ES) ที่กำหนดโดยองค์กร European Association for Standardizing Information and Communication Systems หรือ Ecma International เพื่อให้ภาษา Script นั้นมีความเป็น Standard ปัจจุบันถูกพัฒนามาจนถึง ECMAScript 2020
- เว็บไซต์จำนวนมากใช้ **JavaScript** สำหรับทำงานที่ฝั่ง **Client-side** จึงทำให้เว็บเบราว์เซอร์ต่าง ๆ มี **JavaScript engine** ที่เป็นโปรแกรมคอมพิวเตอร์ที่ใช้สำหรับประมวลผลโค้ดของภาษา JavaScript ที่รันบนเว็บเบราว์เซอร์
- JavaScript engine ในปัจจุบันได้มีการพัฒนามาให้อยู่ในรูปแบบของคอมไพเลอร์ที่มีการคอมไพล์ในตอนที่โปรแกรมรัน (Just-in-time compilation: JIT) เพื่อเพิ่มประสิทธิภาพการทำงานของโปรแกรม
- **การทำงาน JavaScript เกิดขึ้นบนเบราว์เซอร์** ดังนั้นไม่ว่าจะใช้เซิร์ฟเวอร์อะไร หรือที่ไหน ก็สามารถใช้ JavaScript ในเว็บเพจได้ ต่างกับภาษาสคริปต์อื่น เช่น PHP ซึ่งต้องแปลความและทำงานที่ตัวเครื่องเซิร์ฟเวอร์ แต่ JavaScript ไม่สามารถรับและส่งข้อมูลต่างๆ กับเซิร์ฟเวอร์โดยตรง เช่น การอ่านไฟล์จากเซิร์ฟเวอร์ เพื่อนำมาแสดงบนเว็บเพจ หรือรับข้อมูลจากผู้ชม เพื่อนำไปเก็บบนเซิร์ฟเวอร์ เป็นต้น ดังนั้นจึงยังคงต้องอาศัยภาษา server-side script อยู่ (ความจริง JavaScript ที่ทำงานบนเซิร์ฟเวอร์เวอร์ก็มี ซึ่งต้องอาศัยเซิร์ฟเวอร์ที่สนับสนุนโดยเฉพาะเช่นกัน)



Extension VSCode



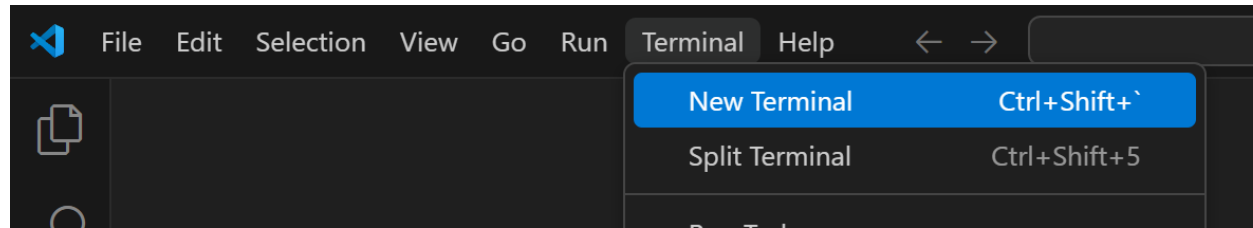
- **JavaScript (ES6) code snippets** ช่วยเขียนคำสั่ง JavaScript (ES6) จะแนะนำคำสั่งให้เมื่อพิมพ์โค้ด
- **Bracket Pair Colorizer** ใส่สีให้วงเล็บและปีกกา
- **Live Server** จำลอง Server เพื่อแสดงผลหน้าเว็บที่กำลังเขียนโค้ดอยู่ เมื่อเราแก้ไขโค้ดแล้วบันทึกหน้าเว็บจะอัปเดตให้อัตโนมัติ
- **Prettier - Code formatter** จัดโค้ดให้สวยงาม กด Alt + Shift + F
- **Indent-rainbow** ใส่สีให้การเยื้องของโค้ด โดยสลับสีที่แตกต่างกัน
- **Auto-Save on Window Change** จะทำให้ผู้ใช้งานไม่ต้องกด Ctrl+s หรือ Save เพราะส่วนเสริมนี้นั้นจะ Save ให้อัตโนมัติเวลาที่เปลี่ยนหน้าต่าง



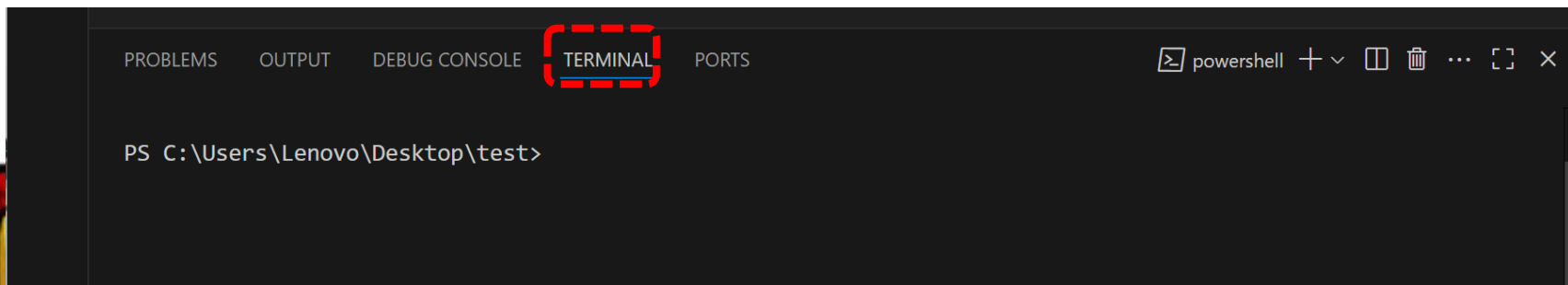
การเปิดใช้งาน Terminal บน VS Code



- การรันไฟล์ .js ด้วย Node.js ต้องเขียนคำสั่งผ่าน Command Line หรือ Terminal
- ใน VS Code สามารถเปิด Terminal ได้จาก Terminal --> New Terminal



- เมื่อเปิด Terminal แล้วจะต้องเลือกแท็บ Terminal เพื่อเขียนคำสั่ง โดยใน VS Code สามารถเลือก 2 แบบ Command Prompt (cmd) และ PowerShell เวลาใช้ Node.js สามารถใช้ได้ทั้ง cmd และ PowerShell
 - ถ้าเป็น ผู้เริ่มต้น ใช้ cmd ใช้งานกับคำสั่งที่ไม่ซับซ้อน
 - ถ้าต้องเขียน สคริปต์ซับซ้อน ใช้ PowerShell



คำสั่ง Terminal พื้นฐาน



Drive (ไดรฟ์) คือ พื้นที่เก็บข้อมูล ที่ระบบปฏิบัติการมองเห็นใน Windows แต่ละ drive จะมี ตัวอักษร (drive letter) เช่น C: = ไดรฟ์หลักที่ติดตั้ง Windows (มักเป็น HDD/SSD)

Root (รูท) หมายถึง ตำแหน่งสูงสุด (จุดเริ่มต้น) ของโครงสร้างไฟล์ใน drive ทุก drive จะมี root directory ของตัวเองใน Windows แทนด้วย \ เช่น C:\ = root ของ drive C

Directory (ไดเรกทอรี) แปลว่า โฟลเดอร์ บนคอมพิวเตอร์ใช้สำหรับเก็บไฟล์และโฟลเดอร์ย่อย (subdirectory)

Path (เส้นทาง)คือ ที่อยู่ (Address) ของไฟล์หรือโฟลเดอร์ แบ่งเป็น 2 แบบ

- Absolute path (เส้นทางเต็ม) → ระบุจาก root ของ drive เช่น C:\Users\maprang\Documents\project\file.js
- Relative path (เส้นทางสัมพัทธ์) → อ้างอิงจากตำแหน่งที่อยู่ปัจจุบัน เช่น .\file.js (ไฟล์ในโฟลเดอร์ปัจจุบัน) , ../project2\ (ย้อนขึ้น 1 ระดับ แล้วเข้า project2)

คำสั่ง	ใช้ทำอะไร	ตัวอย่าง	คำอธิบายตัวอย่าง
cd	เปลี่ยนโฟลเดอร์ (Change Directory)	cd D:\project	เปลี่ยน directory ปัจจุบันไปที่ โฟลเดอร์ project ใน Drive D:
cd ..	ย้อนกลับไปโฟลเดอร์ก่อนหน้า	D:\project\test>cd ..	ย้อนขึ้นไปจากโฟลเดอร์ project ไปยัง โฟลเดอร์ test
cd \	ไปยัง directory เริ่มต้นที่แสดง prompt คำสั่งในขณะนั้น	D:\project>cd \	ย้อนกลับไปยัง root ของ Drive D:
dir (cmd) / ls (PowerShell)	แสดงไฟล์/โฟลเดอร์ใน directory	dir หรือ ls	แสดงรายการไฟล์และโฟลเดอร์ทั้งหมดใน directory ปัจจุบัน
pwd (PowerShell)	แสดง path ปัจจุบัน	pwd	แสดงตำแหน่งของไดเรกทอรีปัจจุบัน

คำสั่ง Terminal พื้นฐาน



คำสั่ง	ใช้ทำอะไร	ตัวอย่าง	คำอธิบายตัวอย่าง
mkdir	สร้างโฟลเดอร์ใหม่	mkdir test	สร้างโฟลเดอร์ test
rmdir	ลบโฟลเดอร์ (ว่างเท่านั้น)	rmdir oldFolder	ลบโฟลเดอร์ oldFolder ที่เป็นโฟลเดอร์ว่าง
del (cmd) / rm (PowerShell)	ลบไฟล์	del file.txt หรือ rm file.txt	ลบไฟล์ file.txt
copy	คัดลอกไฟล์	copy a.txt b.txt	คัดลอกข้อมูลจากไฟล์ a.txt ไป b.txt
move	ย้ายหรือเปลี่ยนชื่อไฟล์	move a.txt C:\backup\	ย้าย a.txt ไป C:\backup\
cls	ล้างหน้าจอ Terminal	cls	ล้างข้อความทั้งหมดบนหน้าจอ Terminal ออก
exit	ปิด Terminal	exit	ปิดหน้าต่าง Terminal
node file.js	รันไฟล์ JavaScript ด้วย Node.js	node app.js	รันไฟล์ app.js



การใช้ console.log



```
test
JS test.js
JS test.js
1 console.log(12345)
2 console.log("สวัสดี", 567)

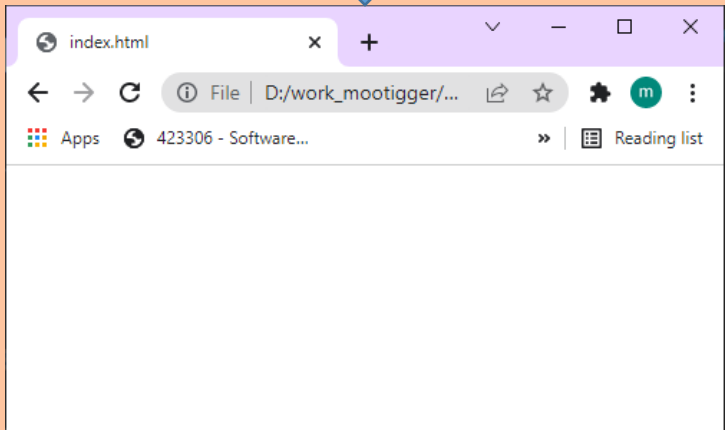
PROBLEMS OUTPUT TERMINAL ...
cmd + v
C:\Users\Lenovo\Desktop\test>node test.js
12345
สวัสดี 567
```

- console.log เป็น ฟังก์ชันใน JavaScriptทำงานโดยพิมพ์ข้อความหรือค่าที่ส่งเข้าไปออกมาใน Console
- ใช้สำหรับ debug (ตรวจสอบค่าระหว่างการทำงานของโปรแกรม) และ แสดงผลลัพธ์

การใช้ console.log ใช้ debug ค่าต่างๆ จะไม่แสดงผลในหน้าเว็บ



```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title> Learn JavaScript</title>
</head>
<body>
  <script>
    console.log("HELLO WORLD")
  </script>
</body>
</html>
```



- จากตัวอย่างคำสั่ง `console.log("HELLO WORLD")` ใช้แสดงข้อความ HELLO WORLD ในรูปแบบ log โดยจะแสดงข้อความออกมาบน console ใน browser
- จะเห็นว่าท้ายคำสั่ง semicolon ไม่จำเป็นอีกต่อไป เพื่ออ่านง่าย
- ในที่นี่ไม่ใช้ว่า JavaScript ES6 (ES2015) ไม่ต้องใส่ semicolon เพียงแต่ เวอร์ชันนี้ JavaScript ฉลาดพอที่จะเติมให้โดยอัตโนมัติ หรือเรียกว่า Automatic Semicolon แต่หากจะใส่เองก็สามารถทำได้

วิธีการเริ่มเขียน JavaScript



- 1. แบบ Internal คือการแทรกคำสั่ง JavaScript ลงในไฟล์ .html ไว้ในส่วนของ <head></head> หรือ <body></body> โดยคำสั่งของ JavaScript จะอยู่ภายใน <script type="text/javascript">...</script>

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script type="text/javascript">
    //ใส่คำสั่ง JavaScript
  </script>
</body>
</html>
```



วิธีการเริ่มเขียน JavaScript



- 2. แบบ External คือ แยกไฟล์คำสั่ง JavaScript บันทึกนามสกุล .js แล้วเรียกใช้โดยใส่ `<script src="ชื่อไฟล์.js"></script>`

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-
Compatible" content="IE=edge">
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <title>Learn JavaScript</title>
  <script src="app.js"></script>
</head>
<body>
</body>
</html>
```

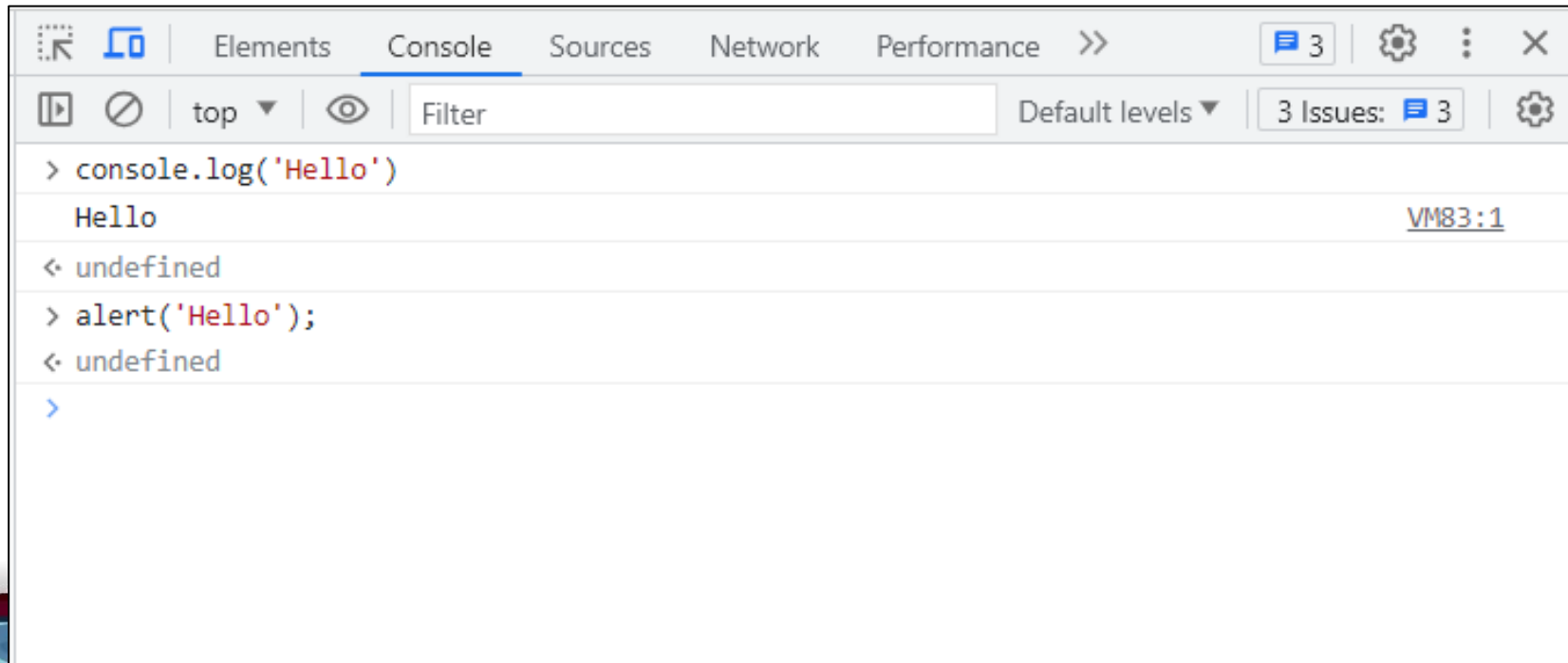
app.js

```
//คำสั่ง JavaScript
console.log("HELLO WORLD")
alert("HELLO WORLD")
```



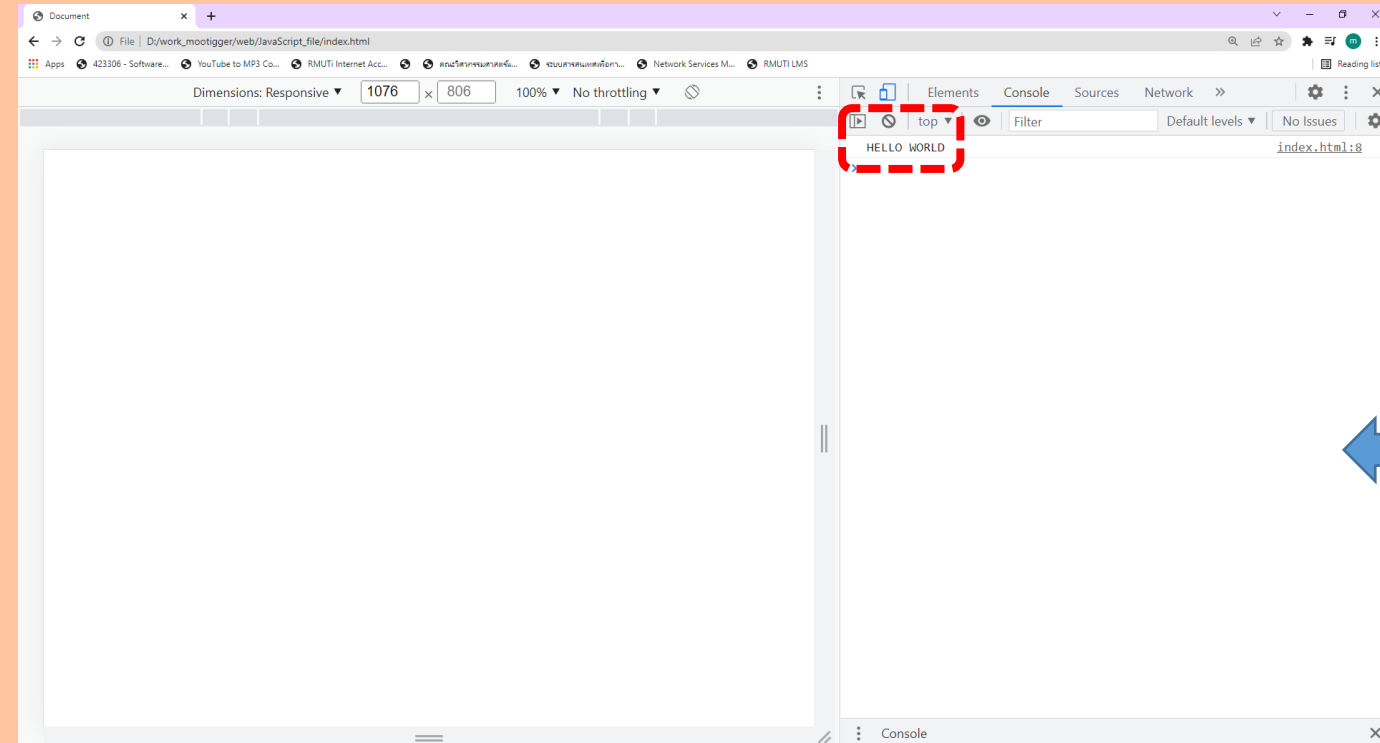
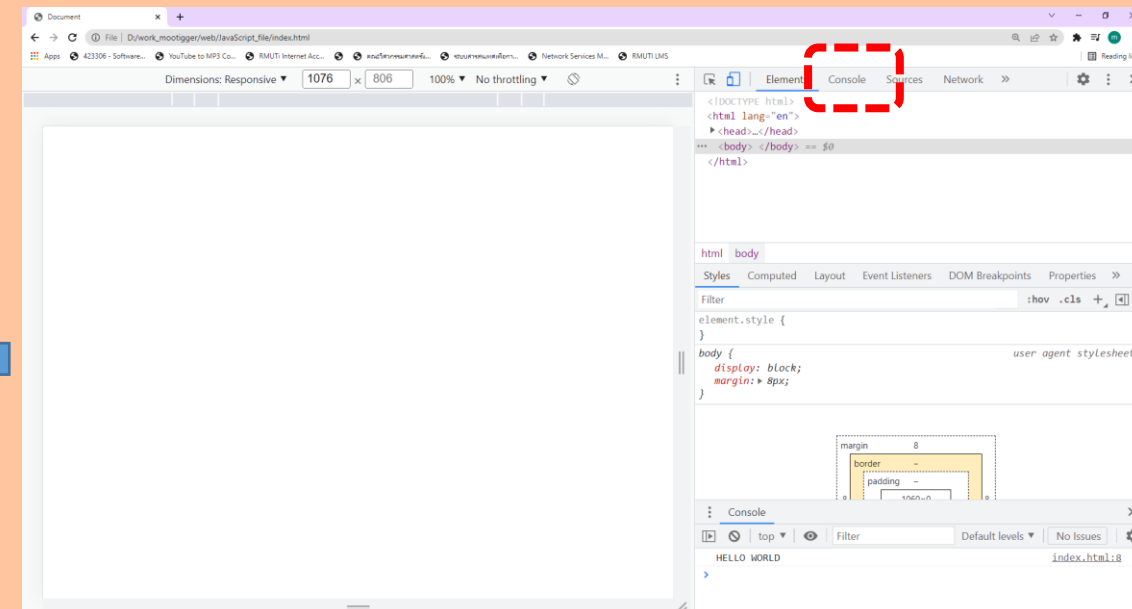
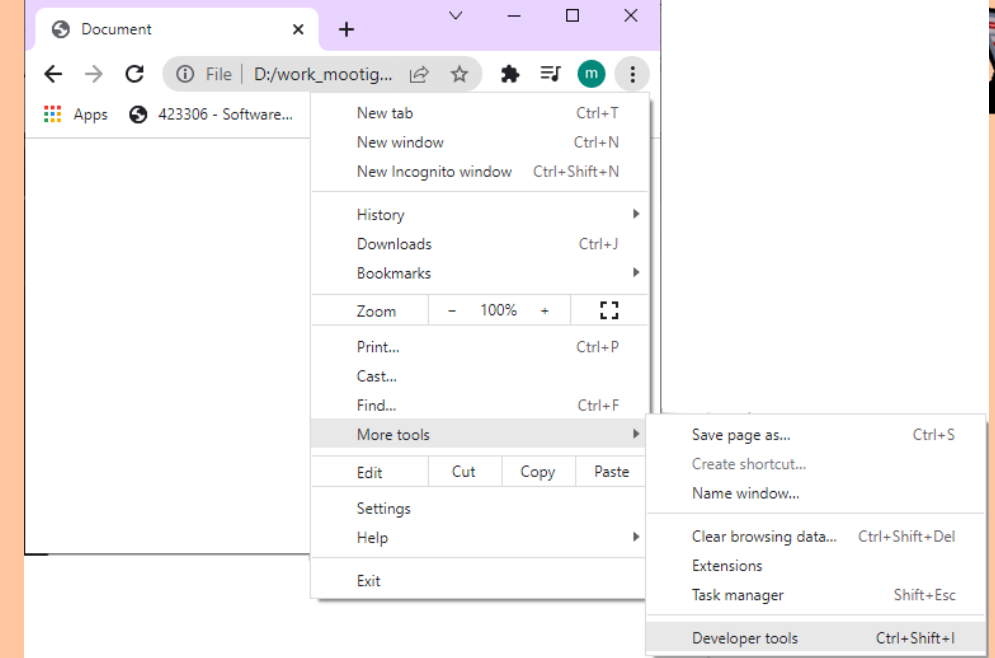
JavaScript เป็นภาษาที่สามารถใช้งานบนเบราว์เซอร์ได้

- เปิด Google Chrome และกด F12 หรือ คลิกขวา เลือกเมนู Inspect
- เลือกแท็บ console



การใช้ console.log ใน JavaScript

```
console.log("HELLO WORLD");
```



การใช้ console.log ใน JavaScript



- console.log() ใช้แสดงข้อความ (label อ่านว่า เล้-เบิ้ล) ในรูปแบบ log โดยจะแสดงข้อความออกมาบน console ใน browser
- เป็นคำสั่งที่ใช้บ่อยสำหรับนักพัฒนาที่ใช้ Javascript เพื่อทำการ Debug สิ่งต่างๆ
- console.log ไม่เพียงแต่สามารถแสดงข้อมูลข้อความได้เท่านั้น แต่มันยังสามารถใช้แสดงออบเจ็กต์ อาร์เรย์ ฟังก์ชัน หรือข้อมูลทุกประเภทในภาษา JavaScript

- แสดงข้อความเดียว

```
console.log("HELLO WORLD");
```



```
HELLO WORLD
```

- แสดงข้อความเป็นส่วนๆ ให้ง่าย

```
console.log("HELLO WORLD","JavaScript",123,'A');
```



```
HELLO WORLD JavaScript 123 A
```

- แสดงค่าจากตัวแปร

```
var name = "HELLO WORLD"  
console.log(name)  
console.log("%s",name)
```



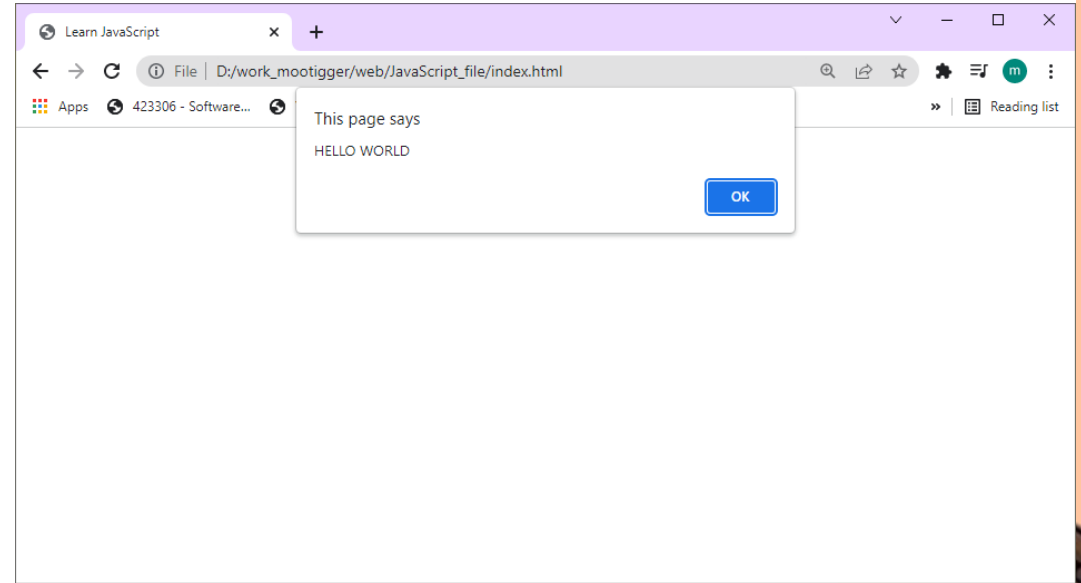
```
HELLO WORLD  
HELLO WORLD
```

การใช้ alert() สำหรับแจ้งเตือนผู้ใช้ในหน้าเว็บ



- alert() เป็นคำสั่งที่ใช้แสดงกล่องข้อความแจ้งเตือนใช้โต้ตอบกับผู้ใช้งาน โดยโปรแกรมจะหยุดทำงานชั่วคราวในขณะที่มีกล่องโต้ตอบขึ้นมาจนกว่าผู้ใช้จะกดปุ่ม OK โปรแกรมจึงจะทำงานต่อไป
 - มีรูปแบบการใช้คำสั่ง alert() ดังนี้
 - alert("ข้อความ"); หรือ alert(ตัวแปร);

```
alert("HELLO WORLD")
```



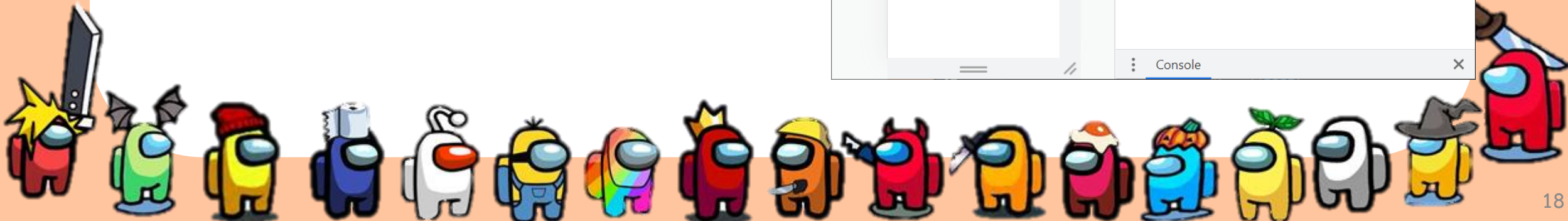
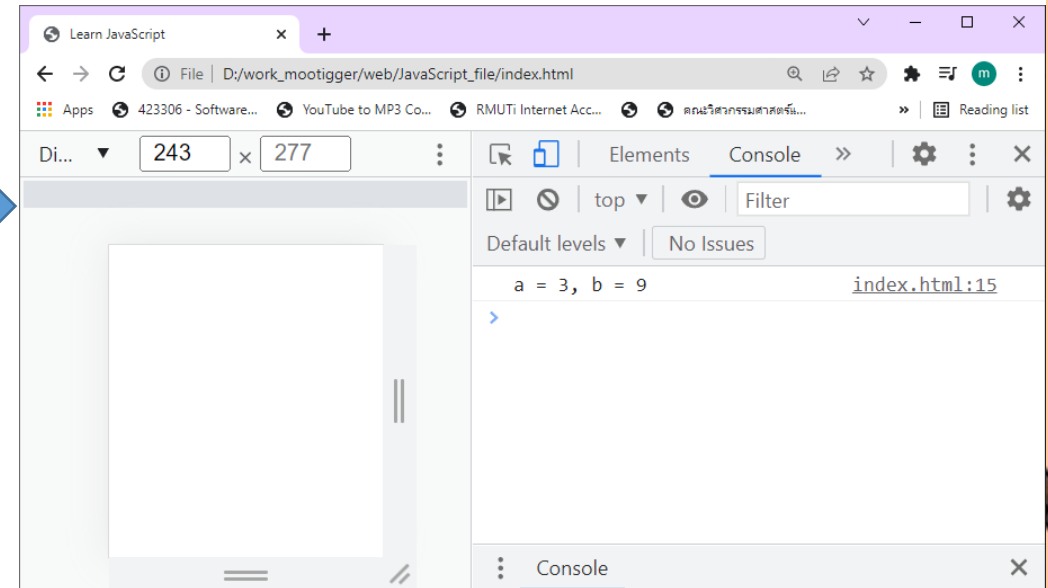
Comments



- Comment (คอมเมนต์) เป็นข้อความที่โปรแกรมเมอร์ใช้สำหรับอธิบายโปรแกรมที่เขียนขึ้น
 - แบบหนึ่งบรรทัด ใช้ `//` วางไว้ด้านหน้าข้อความคอมเมนต์
 - แบบหลายบรรทัดข้อความคอมเมนต์จะต้องถูกครอบด้วย `/*` และ `*/`

```
//ประกาศตัวแปร
let a = 3;
let b = 9; // Set b to 9

/*แสดงค่าตัวแปร
โดย a มีค่า 3 และ b มีค่า 9 */
console.log("a = " + a + ", b = " + b);
```



รูปแบบของตัวแปร



- ตัวแปรแบบ Dynamic Typing ชนิดตัวแปรจะเป็นอะไรก็ได้ตามค่าที่ตัว มันเก็บโดยไม่ต้องประกาศชนิดข้อมูล
- ตัวแปรแบบ Static Typing ต้องประกาศชนิดข้อมูลในตอนเริ่มต้น เช่น int, double, char เพื่อบอกว่าตัวแปรนี้จะเก็บข้อมูลชนิดไหน

** ตัวแปรใน JavaScript เป็นรูปแบบ Dynamic Typing



การตั้งชื่อตัวแปร



- อักษรตัวแรก a-z หรือ A-Z หรือ _ (underscore) หรือ \$ (ดอลลาร์) ห้ามเป็นตัวเลข
 - ถ้าเป็นโปรเจกต์ทั่วไป ไม่ควรใช้ \$ ขึ้นต้นทุกตัวแปร เพราะอ่านยากควรใช้ \$ ก็ต่อเมื่อมี เหตุผลเฉพาะ เช่น ใช้กับ jQuery, Vue, Angular หรือบอกว่าตัวแปรนี้เป็นตัวแปรพิเศษ
- ตัวอักษรตัวถัดไปสามารถเป็น a-z หรือ A-Z หรือ _ หรือ หรือ \$ หรือ ตัวเลข ห้ามมีช่องว่างหรือสัญลักษณ์อื่น
- ชื่อเดียวกันแต่การเขียนต่างกัน ถือว่าไม่เหมือนกัน เช่น One ≠ one ≠ ONE

- ต้องไม่ใช่คำสงวน (Reserved Words)

Break	case	catch	class	const	continue	debugger	default
delete	do	else	export	extends	finally	for	function
if	import	in	instanceof	let	new	return	super
switch	this	throw	try	typeof	var	void	while
with	yield						

- ให้ขึ้นต้นด้วยตัวเล็ก ถ้าหลายคำต่อกันให้ตัวอักษรตัวแรกของคำเป็นตัวพิมพ์ใหญ่ เช่น numDay



การประกาศตัวแปรใน JavaScript



- ในการประกาศตัวแปรใช้คีย์เวิร์ด **var** ในการประกาศตัวแปร แต่จากการมาของ ES6 (ECMAScript 2015) นั้นได้เปิดตัวคำสั่งใหม่อย่าง **let** และ **const** ที่ทำงานกับขอบเขตของตัวแปรในระดับบล็อกของคำสั่งควบคุม ซึ่งเป็นวิธีที่ภาษาเขียนโปรแกรมสมัยใหม่ควรจะเป็น
 - **const** มีขอบเขตการทำงานในระดับ block หรือ ส่วนที่อยู่ใน เครื่องหมาย {} ไม่สามารถเปลี่ยนแปลงค่าได้ ยกเว้นเป็น **object** , **array** สามารถแก้ไขค่าได้ แต่ไม่สามารถ initial ใหม่ได้
 - **let** มีขอบเขตการทำงานในระดับ block หรือ ส่วนที่อยู่ใน เครื่องหมาย {} แก้ไขค่าได้ ไม่สามารถอ้างอิงก่อนการประกาศได้
 - **var** มีขอบเขตการทำงานในระดับ function และ global แก้ไขค่าได้ และสามารถอ้างอิงหรือใช้งานก่อนการประกาศได้ !! ตัวแปรที่ประกาศมักจะทะลุ Block หรือ ส่วนที่อยู่ในเครื่องหมาย {} เนื่องจาก var จะมี Scope อยู่ระดับ Function
- คีย์เวิร์ด **var** ใช้ในโค้ด JavaScript ตั้งแต่ปี 1995 ถึง 2015 และ คีย์เวิร์ด **let** และ **const** ใน JavaScript เริ่มใช้ในปี 2015
- *คำสั่ง **var** ในปัจจุบันยังคงใช้งานได้ในปัจจุบัน แต่ใน ES6 (2015) เป็นต้นมา แนะนำให้ใช้ **let** และ **const** แทน
- **หากต้องการให้โค้ดของทำงานในเบราว์เซอร์รุ่นเก่า ต้องใช้ **var**

Block Scope (let/const) การประกาศตัวแปรใน JavaScript



- การกำหนดค่าคงที่โดยใช้ const นิยมใช้กับตัวแปรที่มีตัวพิมพ์ใหญ่ทั้งหมดถ้ามีหลายคำให้ใช้ _ เชื่อม
- รูปแบบของการประกาศตัวแปร **var/let/const ชื่อตัวแปร = ค่า**
- เมื่อก่อนประกาศตัวแปรโดยขึ้นต้นด้วย var แต่ปัจจุบันใช้ let เนื่องจาก var จะมองตัวแปรแบบ global variable
- สามารถประกาศหลายตัวแปรได้ในคำสั่งเดียวโดยคั่นแต่ละตัวแปรด้วยเครื่องหมายคอมมา (,) และประกาศตัวแปรหลายตัวพร้อมกันที่มีค่าเดียวกันได้

```
var x = 5;  
var y = 6;  
var z = x + y;
```

```
let x = 5, y = 6;  
let z = x + y;
```

```
const PRICE1 = 5;  
const PRICE2 = 6;  
let total = PRICE1 + PRICE2;
```

- การตั้งชื่อของตัวแปรสามารถประกอบไปด้วย ตัวอักษร ตัวเลข สัญลักษณ์ Underscore (_) และดอลลาร์ (\$) ถือว่าเครื่องหมายดอลลาร์เป็นตัวอักษร และชื่อของนอกจากนี้ ตัวแปรเป็น case sensitive หมายความว่า ตัวแปรที่สะกดเหมือนกัน แต่ใช้ตัวพิมพ์ใหญ่พิมพ์เล็กต่างกัน จะถือว่าเป็นตัวแปรคนละตัว

กฎทั่วไป: ค่าของตัวแปรไม่สามารถเปลี่ยนแปลงได้ ให้ใช้การประกาศตัวแปรด้วย const
 แต่ถ้าค่าของตัวแปรสามารถเปลี่ยนแปลงได้ ให้ใช้ let



Block Scope



```
{  
  let x = 5;  
  
    {  
      let y = 6;  
    }  
}  
  
{  
  let z = 7;  
}
```

- ตัวแปร y อยู่ในขอบเขตเดียวกับ x
- ขอบเขตปีกกาสีเขียวสามารถเรียกใช้ x ได้
- ขอบเขตปีกกาสีน้ำเงิน ไม่สามารถเรียกใช้ y ได้ เพราะ y ไม่อยู่ในขอบเขต
- ขอบเขตปีกกาแดงใช้ได้แค่ตัวแปร z



การประกาศและใช้งานตัวแปร



** let ไม่สามารถประกาศตัวแปรซ้ำ แต่ var ทำได้

```
let x = "John Doe";  
let x = 0; // SyntaxError
```

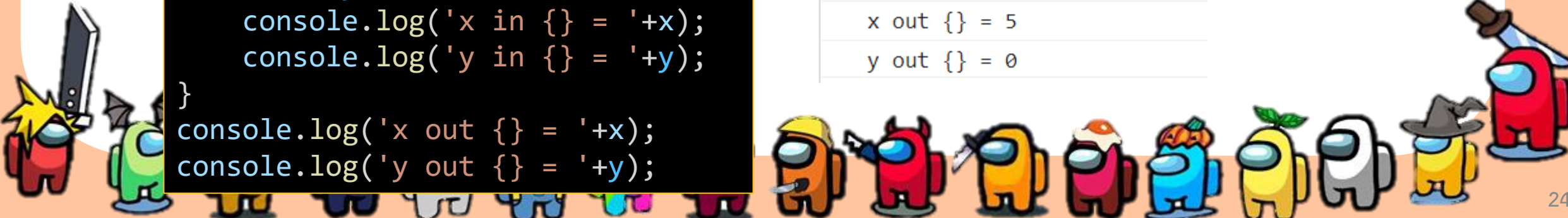
```
var x = "John Doe";  
var x = 0;
```

let และ const มีขอบเขตการทำงานในระดับ block คือ ตัวแปรที่ประกาศภายในบล็อก { } ไม่สามารถเข้าถึงได้จากภายนอกบล็อก
ตัวแปรมีขอบเขตชัดเจน ต่อให้ชื่อซ้ำกัน แต่จะมองต่างกันเพราะอยู่คนละขอบเขต { }

```
let x=5;  
const y=0;  
{  
  let x = 2;  
  const y = 6;  
  console.log('x in {} = '+x);  
  console.log('y in {} = '+y);  
}  
console.log('x out {} = '+x);  
console.log('y out {} = '+y);
```



	Elements	Console
	top ▼	Filter
x in {} = 2		
y in {} = 6		
x out {} = 5		
y out {} = 0		

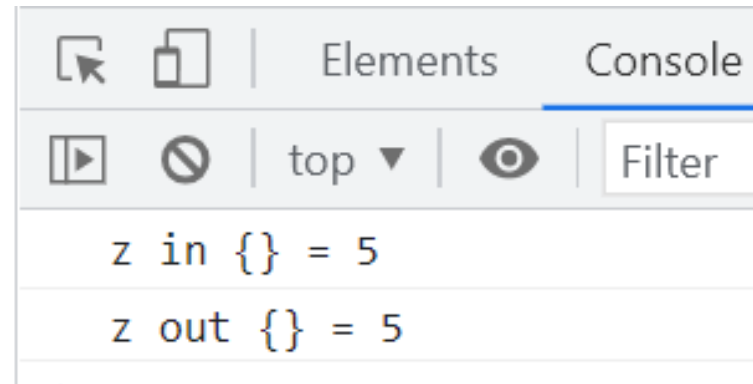


การประกาศและใช้งานตัวแปร



ตัวแปรที่ประกาศด้วยคีย์เวิร์ด `var` ไม่สามารถมีขอบเขตการทำงานในระดับบล็อกได้ ตัวแปรที่ประกาศภายในบล็อก `{ }` สามารถเข้าถึงได้จากภายนอกบล็อก ทำให้ค่าเปลี่ยนแปลงได้ง่าย หากตั้งชื่อซ้ำใน และนอก `{ }` ค่าจะถูกเปลี่ยนแปลงการทำงานโปรแกรมจะไม่ถูกต้อง

```
var z=0;
{
  var z=5;
  console.log('z in {} = '+z);
}
console.log('z out {} = '+z);
```

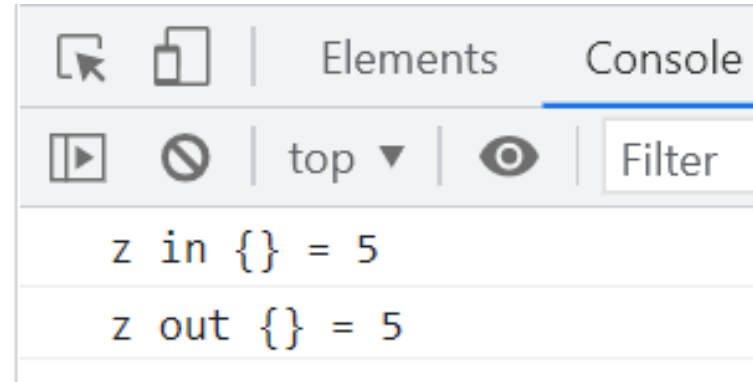


การประกาศและใช้งานตัวแปร



ตัวแปรที่ประกาศด้วยคีย์เวิร์ด `var` ไม่สามารถมีขอบเขตการทำงานในระดับบล็อกได้ ตัวแปรที่ประกาศภายในบล็อก `{ }` จึงใช้ `let` หรือ `const` ประกาศตัวแปรที่ใช้ใน `{ }` เพื่อบล็อกให้ตัวแปรเหล่านั้นใช้แค่ใน `{ }`

```
var z=0;
{
  var z=5;
  console.log('z in {} = '+z);
}
console.log('z out {} = '+z);
```



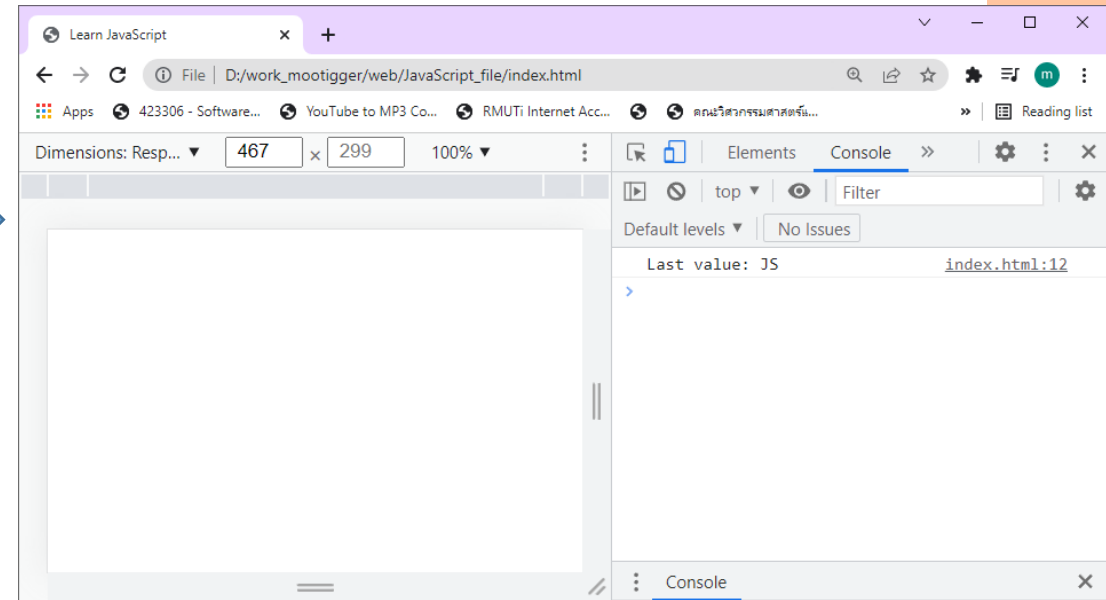
การประกาศและใช้งานตัวแปร



- เนื่องจากภาษา JavaScript นั้นเป็นภาษาแบบ Dynamic typing นั้นหมายความว่าตัวแปรที่ประกาศด้วยข้อมูลประเภทหนึ่ง สามารถเปลี่ยนค่าเป็นข้อมูลประเภทอื่นได้ตลอดการทำงานของโปรแกรม แต่ในทางปฏิบัติ มักใช้หนึ่งตัวแปรสำหรับเก็บข้อมูลประเภทเดียวกันเท่านั้น ยกเว้น null ที่อาจจะมีการใช้งานร่วมกับข้อมูลประเภทอื่น

- เช่น

```
let x = 16
x = true
x = "JS"
console.log("Last value: " + x)
```



ประเภทข้อมูล ในภาษา JavaScript



- ข้อมูลพื้นฐาน (Primitive Types) เมื่อสร้างตัวแปรใหม่โดยอ้างอิงจากตัวแปรเก่า ค่าที่ได้จะเป็นการคัดลอก

- String
- Number
- Boolean
- BigInt
- Undefined
- Null
- Symbol

- ข้อมูลแบบอ้างอิง (Reference Types) คือเป็นการสร้างตัวแปรแบบอ้างอิง โดยที่เมื่อตัวแปรไหนเปลี่ยนตัวแปรอื่นก็จะเปลี่ยน

- Object
- Array
- Function



ข้อมูลชนิดตัวเลข



- ชนิดข้อมูลตัวเลขพื้นฐาน

- ใน JavaScript ตัวเลขทั้งหมด (ไม่ว่าจะเป็นจำนวนเต็ม หรือทศนิยม) อยู่ใน ชนิดเดียวกัน คือ Number

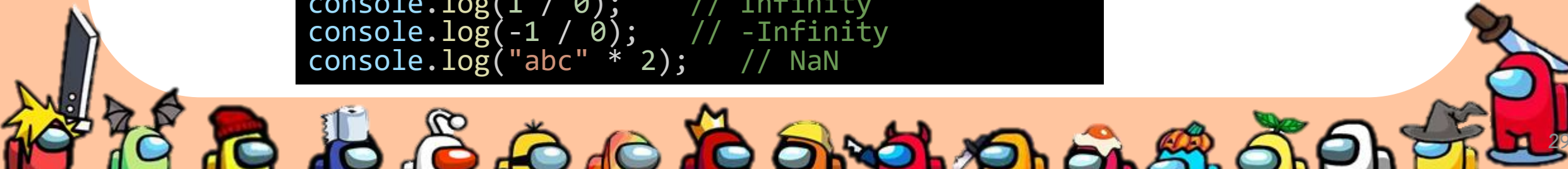
- เช่น

```
let a = 42;           // จำนวนเต็ม (integer)
let b = 3.14;         // จำนวนทศนิยม (float)
let c = -100;         // จำนวนเต็มลบ
```

- ชนิดพิเศษที่เกี่ยวข้องกับตัวเลข

- Infinity และ -Infinity ใช้เมื่อเลขเกินขอบเขต หรือหารด้วยศูนย์
- NaN (Not a Number) ได้จากการคำนวณที่ไม่ใช่ตัวเลข

```
console.log(1 / 0);    // Infinity
console.log(-1 / 0);   // -Infinity
console.log("abc" * 2); // NaN
```



ข้อมูลชนิดตัวเลข



- BigInt

- ใช้สำหรับเก็บตัวเลขจำนวนเต็มที่ ใหญ่เกิน $2^{53} - 1$ (ขอบเขต Number)
 - $2^{53} - 1 = 9,007,199,254,740,991$
- ประกาศโดยใส่ **n** ต่อท้าย

```
let b = 1000000000000000000n;  
let big = 123456789012345678901234567890n;  
console.log(big);
```



ข้อมูลชนิดสตริง



- การประกาศ string ขึ้นมาใช้ ต้องกำหนดเนื้อหาหรือค่าอยู่ในเครื่องหมาย '...' (single quotes), "..." (double quotes) หรือ `...` (backticks / template literal) ครอบข้อความ
- ` การพิมพ์จะต้องเปลี่ยนภาษาคีย์บอร์ดเป็น EN แล้วกด ปุ่ม window+ ปุ่ม backticks

```
let str1 = "Hello";      // double quotes
let str2 = 'World';      // single quotes
let str3 = `Hello World`; // backticks (template literal)
```

- การกำหนดข้อความในสัญลักษณ์ '...' และ "..." ต่อบรรทัดเดียวกัน ยกเว้น `...` ที่ข้อความสามารถอยู่คนละบรรทัดได้ สามารถใช้เครื่องหมาย \ หรือใช้การต่อสตริงด้วยเครื่องหมาย +

```
var text1 = "Lorem ipsum dolor sit amet consectetur adipisicing elit. Corrupti, saepe repudiandae."
var text2 = "Lorem ipsum dolor sit amet consectetur adipisicing elit. \
Corrupti, saepe repudiandae."
var text3 = "Lorem ipsum dolor sit amet consectetur adipisicing elit. "+
"Corrupti, saepe repudiandae."
```

สตริงว่าง (Empty String)



- สตริงว่าง คือ ข้อความที่ไม่มีตัวอักษรเลยแทนด้วย "" (double quotes ว่าง), ' ' (single quotes ว่าง), หรือ ` ` (backticks ว่าง)
- สตริงที่มีความยาวเป็น 0
- " " ไม่ใช่สตริงว่าง เพราะ มีช่องว่าง 1 ตัว

```
let empty1 = "";  
let empty2 = ' ';  
let empty3 = ` `;  
  
console.log(empty1); // ""  
console.log(empty1.length); // 0
```

ค่าพิเศษ (special values)



1. null

- หมายถึง ไม่มีค่า (empty value) มักใช้เมื่อต้องการ “รีเซ็ต” ตัวแปร

2. undefined

- หมายถึง ยังไม่ได้กำหนดค่า (variable declared แต่ยังไม่ถูก assign)
- JavaScript จะให้ค่า undefined อัตโนมัติ ถ้าไม่กำหนดค่าให้ตัวแปร

3. NaN (Not a Number)

- หมายถึง ค่าที่ไม่ใช่ตัวเลข (แต่ type เป็น Number!)
- เกิดจากการคำนวณที่ไม่ถูกต้อง
- ตรวจสอบ NaN ใช้ Number.isNaN() แทนการใช้ ===

```
let data = null;
console.log(data);           // null
console.log(typeof data);    // "object"

let a;
console.log(a);              // undefined
console.log(typeof a);       // "undefined"

console.log("abc" * 2);       // NaN
console.log(0 / 0);           // NaN
console.log(typeof NaN);      // "number"

console.log(Number.isNaN(NaN)); // true
```

ข้อมูลชนิดบูลีน

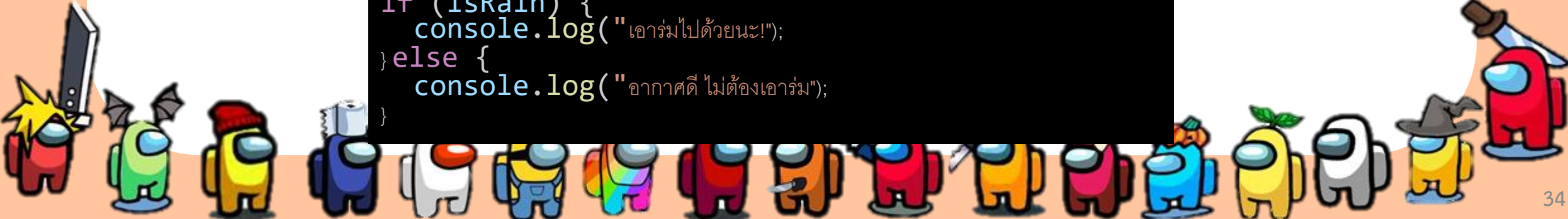


- Boolean เป็น ชนิดข้อมูลพื้นฐาน (Primitive type)
- มีค่าได้แค่ 2 ค่าเท่านั้น คือ
 - true (จริง)
 - false (เท็จ)
- นิยมใช้ในการเปรียบเทียบ (Comparison) หรือใช้ในเงื่อนไข (Conditional Statements)

```
console.log(5 > 3); // true
console.log(10 === 5); // false

let isRain = true;

if (isRain) {
  console.log("เอาร่มไปด้วยนะ!");
} else {
  console.log("อากาศดี ไม่ต้องเอาร่ม");
}
```



Truthy และ Falsy Values



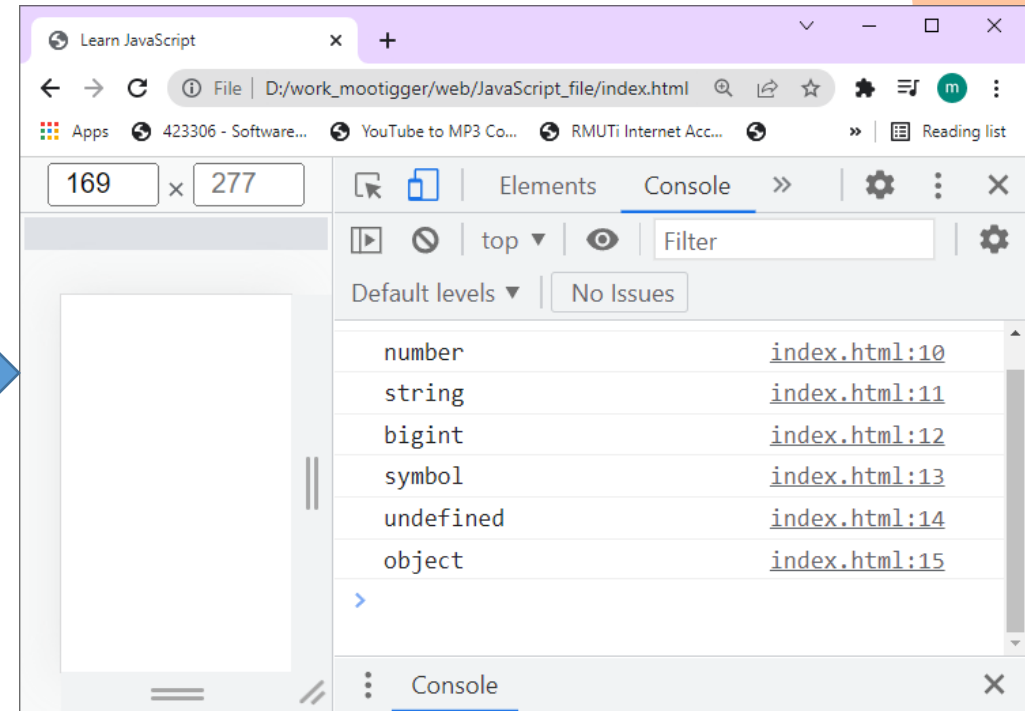
- JavaScript มีแนวคิดที่ว่า ค่าที่ไม่ใช่ boolean ก็สามารถถูกตีความเป็น boolean ได้ (truthy/falsy)
- Truthy value = ค่าที่ไม่ใช่ false โดยตรง แต่ JavaScript จะตีความเป็นจริง (true)
- Falsy value = ค่าที่ JavaScript ตีความเป็นเท็จ (false)
 - Falsy อ่านว่า ฟอล-ซี (ถูกมองเป็น false)
 - False
 - 0
 - "" (สตริงว่าง)
 - Null
 - Undefined
 - NaN
- Truthy อ่านว่า ทรู-ธี (ถูกมองเป็น true) ค่าทุกอย่างที่ไม่ใช่ Falsy เช่น "hello", 123, [], {}

การตรวจสอบประเภทข้อมูล



- ใช้ **typeof** เพื่อตรวจสอบประเภทข้อมูล โดยส่งค่าผลลัพธ์กลับเป็น String ที่เป็นชื่อคลาสของประเภทข้อมูลในตัวพิมพ์เล็ก

```
console.log(typeof 2);           // number
console.log(typeof 1.5);         // number
console.log(typeof "hello");     // string
console.log(typeof 155n);        // bigint
console.log(typeof Symbol());    // symbol
console.log(typeof undefined);   // undefined
console.log(typeof null);        // object
```



ตัวดำเนินการ ในภาษา JavaScript



- ตัวดำเนินการในการกำหนดค่า ใช้เครื่องหมาย =
 - หมายถึง “เอาค่าด้านขวาใส่ไปในตัวแปรด้านซ้าย”

```
let a = 10; // กำหนดค่า 10 ให้กับตัวแปร a
let b = "Hi"; // กำหนดค่า "Hi" ให้กับตัวแปร b
```

- สามารถกำหนดค่า (Assignment) แบบต่อเนื่อง (chained assignment)
 - คือการใช้เครื่องหมาย = หลายครั้งในบรรทัดเดียว เพื่อกำหนดค่าให้กับ ตัวแปรหลายตัวพร้อมกัน
 - การทำงานของจะเป็นการกำหนดค่าให้ตัวแปรจากขวาไปซ้าย (right-to-left)

```
let x, y, z;
x = y = z = 10; //การกำหนดค่าแบบต่อเนื่อง

console.log(x); // 10
console.log(y); // 10
console.log(z); // 10
```

ตัวดำเนินการ ในภาษา JavaScript



- ตัวดำเนินการทางคณิตศาสตร์ (Arithmetic operators) คือตัวดำเนินการที่ใช้ในการคำนวณทางคณิตศาสตร์

สัญลักษณ์	ตัวดำเนินการ	ตัวอย่าง
+	การบวก	$a + b$
-	การลบ	$a - b$
*	การคูณ	$a * b$
/	การหาร	a / b
%	การหารเอาเศษ	$a \% b$
**	การยกกำลัง	$a ** b$

```
let a = 2;  
let b = 5;  
console.log('a + b = ' + (a + b));  
console.log('a - b = ' + (a - b));
```



```
a + b = 7  
a - b = -3
```

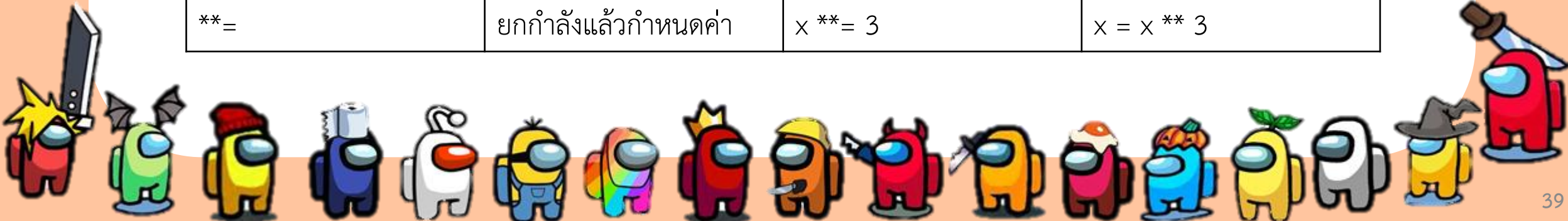


ตัวดำเนินการ ในภาษา JavaScript



- ตัวดำเนินการแบบย่อ (Compound Assignment Operators) คือตัวดำเนินการที่สามารถ “กำหนดค่า + ดำเนินการ” ได้พร้อมกัน เช่น

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	เทียบเท่ากับ
+=	บวกแล้วกำหนดค่า	<code>x += 2</code>	<code>x = x + 2</code>
-=	ลบแล้วกำหนดค่า	<code>x -= 3</code>	<code>x = x - 3</code>
*=	คูณแล้วกำหนดค่า	<code>x *= 4</code>	<code>x = x * 4</code>
/=	หารแล้วกำหนดค่า	<code>x /= 5</code>	<code>x = x / 5</code>
%=	หารเอาเศษแล้วกำหนดค่า	<code>x %= 2</code>	<code>x = x % 2</code>
**=	ยกกำลังแล้วกำหนดค่า	<code>x **= 3</code>	<code>x = x ** 3</code>



ตัวดำเนินการ ในภาษา JavaScript



- ตัวดำเนินการเพิ่มหรือลดค่า (Increment / Decrement)

- การเพิ่มค่า (Increment ++) ใช้ ++ เพื่อ บวกค่าเพิ่มทีละ 1

- $x++ \rightarrow$ Post-increment (บวกหลังจากใช้งานค่า)

- $++x \rightarrow$ Pre-increment (บวกก่อนแล้วค่อยใช้งานค่า)

```
let x = 5
let y = x++
console.log(y); // 5
console.log(x); // 6
```

- การลดค่า (Decrement --) ใช้ -- เพื่อ ลบค่าออกทีละ 1

- $y-- \rightarrow$ Post-decrement (ลบหลังจากใช้งานค่า)

- $--y \rightarrow$ Pre-decrement (ลบก่อนแล้วค่อยใช้งานค่า)



ตัวดำเนินการ ในภาษา JavaScript



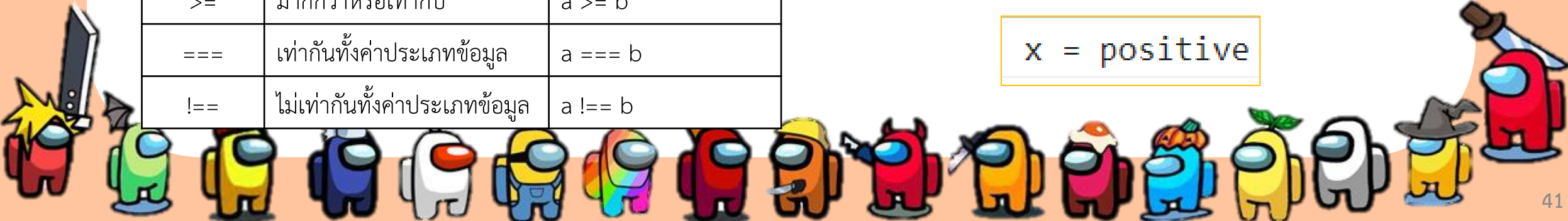
- ตัวดำเนินการเปรียบเทียบ (Comparison operators) คือตัวดำเนินการที่ใช้สำหรับเปรียบเทียบระหว่างค่าสองค่า ผลลัพธ์ของตัวดำเนินการเปรียบเทียบนั้นจะเป็น Boolean เพื่อบ่งบอกว่าผลการเปรียบเทียบเป็นจริงหรือไม่

สัญลักษณ์	ตัวดำเนินการ	ตัวอย่าง
==	เท่ากับ	a == b
!=	ไม่เท่ากับ	a != b
<	น้อยกว่า	a < b
<=	น้อยกว่าหรือเท่ากับ	a <= b
>	มากกว่า	a > b
>=	มากกว่าหรือเท่ากับ	a >= b
===	เท่ากันทั้งค่าประเภทข้อมูล	a === b
!==	ไม่เท่ากันทั้งค่าประเภทข้อมูล	a !== b

```
let x = 1;
if (x > 0) {
  console.log("x = positive");
} else if (x < 0) {
  console.log("x = negative");
} else {
  console.log("x = zero");
}
```



x = positive



- 

```
console.log("Invalid username or password.");
```

Invalid username or password.

สัญลักษณ์	ตัวดำเนินการ	ตัวอย่าง
&&	Logical AND	a && b
	Logical OR	a b
!	Logical NOT	!a

```
let user = "user";
let pass = "1234";

if (user == "user" && pass == "secret") {
    console.log("Logged in as Metin.");
} else {
    console.log("Invalid username or password.");
}
```

Invalid username or password.

ตัวดำเนินการ ในภาษา JavaScript



- ตัวดำเนินการแบบสามทาง (Ternary Operator) หมายถึง ตัวดำเนินการเงื่อนไข (conditional operator) ที่ใช้ ? :
- รูปแบบทั่วไป
 - เงื่อนไข ? ค่าถ้าเป็นจริง : ค่าถ้าเป็นเท็จ
 - ถ้า เงื่อนไข (condition) เป็นจริง → คืนค่าหลังเครื่องหมาย ?
 - ถ้า เงื่อนไขเป็นเท็จ → คืนค่าหลังเครื่องหมาย :

```
let age = 20;  
let status = (age >= 18) ? "ผู้ใหญ่" : "เด็ก";  
console.log(status); // "ผู้ใหญ่"
```



ตัวดำเนินการ ในภาษา JavaScript

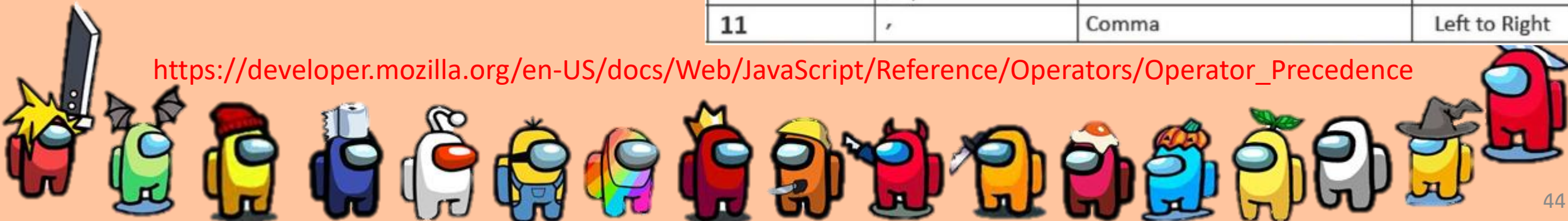
- ลำดับความสำคัญของตัวดำเนินการ (Operator precedence) คือการกำหนดความสำคัญการทำงานของตัวดำเนินการเมื่อโปรแกรมพบกับตัวดำเนินการมากกว่าหนึ่งตัวในหนึ่งคำสั่ง

```
console.log(1 + 2 * 3);
```

คำตอบคือ 7 นั่นเป็นเพราะว่าตัวดำเนินการคูณ * มีลำดับความสำคัญมากกว่าตัวดำเนินการบวก + ดังนั้นลำดับของการคำนวณจะเป็น $1 + (2 * 3)$

Precedence Order	Operator	Description	Associativity
1	()	Parentheses	Left to Right
2	++, --	Post-increment and Post-decrement operators	Not Applicable
3	typeof, ++, --, -, ~, !	Pre-increment and Pre-decrement operators, Logical NOT, Bitwise NOT, and Unary negation	Right to Left
4	*, /, -	Multiplication, Division, and Modulo	Left to Right
5	+, -	Addition and Subtraction	Left to Right
6	<, <=, >, >=	Less than, Less than or equal, Greater than, and Greater than or equal	Left to Right
7	==, ===, !=, !==	Equal to, Strict equal to, Not equal to, and Strict not equal to	Left to Right
8	&, , ^, &&,	Bitwise AND, Bitwise OR, Bitwise XOR, Logical AND, and Logical OR	Left to Right
9	?:	Conditional operator	Right to Left
10	=, +=, -=, *=, /=, %=	Assignment operators	Right to Left
11	,	Comma	Left to Right

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_Precedence



สตริงแม่แบบ (Template Literal)



- Template Literal → อ่านว่า“เทมเพลต ลิเทอรัล”
- เป็นรูปแบบการเขียน String ใช้เครื่องหมาย backtick (`) แทน ' ' หรือ " "
- ทำให้เขียนข้อความได้สะดวกขึ้น โดยเฉพาะเวลาต้อง
 - แทรกตัวแปร/การคำนวณด้วย \${...} เพื่อแทรกค่าตัวแปรหรือ expression ลงใน string ได้เลย
 - เขียน string หลายบรรทัด

```
let name = "Maprang";  
let message = `Hello, ${name}!`;  
console.log(message); // Hello, Maprang!  
  
let a = 5;  
let b = 10;  
  
console.log(`ผลบวกของ ${a} และ ${b} คือ ${a + b}`);  
// ผลบวกของ 5 และ 10 คือ 15
```

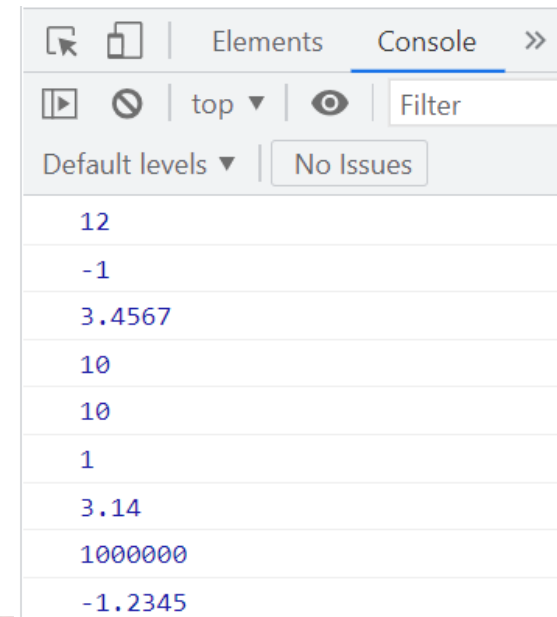


วิธีการแปลง type ของ JavaScript ใช้ฟังก์ชันของ JavaScript



- เพื่อแปลง String หรือค่าใดๆ ในภาษา JavaScript ให้เป็นตัวเลข สามารถทำได้โดยใช้สามฟังก์ชันดังต่อไปนี้
 - **Number()**: แปลงออบเจ็กต์ใด ๆ ให้เป็นตัวเลข
 - **toString()**: แปลง ตัวเลข ให้เป็น String
 - **parseInt()**: แปลง String ให้เป็นตัวเลขจำนวนเต็ม
 - **parseFloat()**: แปลง String ให้เป็นตัวเลขจำนวนเต็มทศนิยม

```
console.log(Number("12"));  
console.log(Number("-1"));  
console.log(Number("3.4567"));  
  
console.log(parseInt("10"));  
console.log(parseInt("10.5"));  
console.log(parseInt("1.2345"));  
  
console.log(parseFloat("3.14"));  
console.log(parseFloat("10E5"));  
console.log(parseFloat("-1.2345"))
```



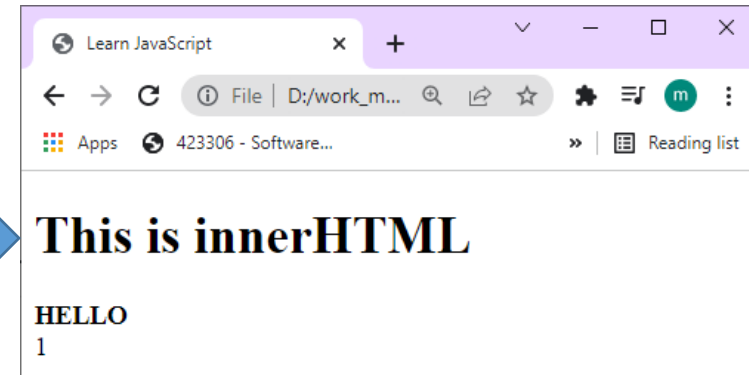
การใช้ document.write()



- document.write() ใช้เพื่อทำการเขียนข้อความและสามารถใส่โค้ด HTML ลงไปได้

```
<body>
  <h1>This is innerHTML</h1>
  <b id="demo"></b>

  <script>
    var i =1;
    document.write("<b>HELLO</b>");
    document.write("<br>"+i);
  </script>
</body>
```



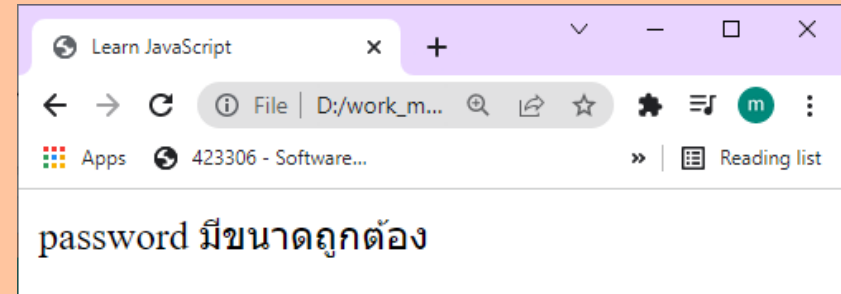
คำสั่งที่ใช้กำหนดเงื่อนไข



- 1. เงื่อนไขที่ใช้ **if** เป็นการเปรียบเทียบว่าตรงกับเงื่อนไขที่กำหนดหรือไม่ หากตรงจะกระทำตามคำสั่งภายในปีกกา หากไม่ตรงจะข้ามไปยังคำสั่งถัดไป

```
if (เงื่อนไข){  
    // คำสั่ง  
}
```

```
let password = 'abcdefgh';  
if (password.length >= 8){ //length ใช้ความยาว text  
    document.write("password มีขนาดถูกต้อง");  
}
```



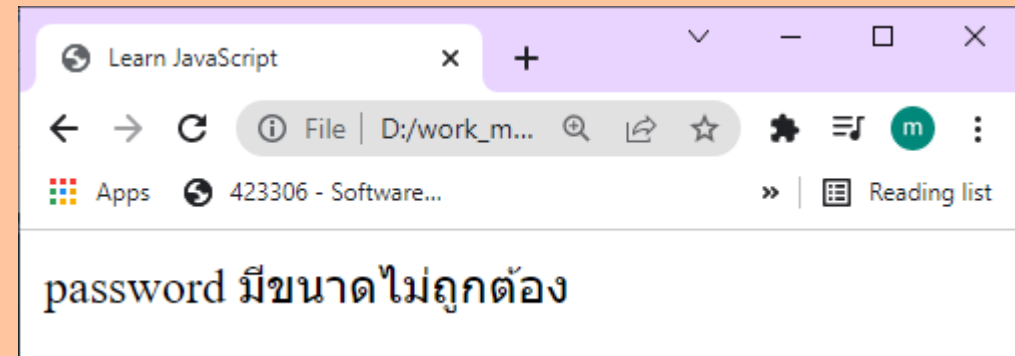
คำสั่งที่ใช้กำหนดเงื่อนไข



- 2.เงื่อนไขที่ใช้ **if...else** เป็นการกำหนดทางเลือกอื่นๆ ในการทำงานนอกเหนือจากที่กำหนดใน if โดยที่หากเงื่อนไขไม่ตรงกับที่ระบุไว้ใน if โปรแกรมจะทำตามคำสั่งที่ระบุไว้ใน else มีรูปแบบดังนี้

```
if (เงื่อนไข){  
    // คำสั่งกรณีตรงกับเงื่อนไข  
}  
else{  
    // คำสั่งกรณีที่ไม่ตรงกับเงื่อนไข  
}
```

```
let password = 'abcdefghijklmnop';  
//length ให้ความยาว text  
if (password.length >= 8 && password.length <= 12){  
    document.write("password มีขนาดถูกต้อง");  
}else{  
    document.write("password มีขนาดไม่ถูกต้อง");  
}
```



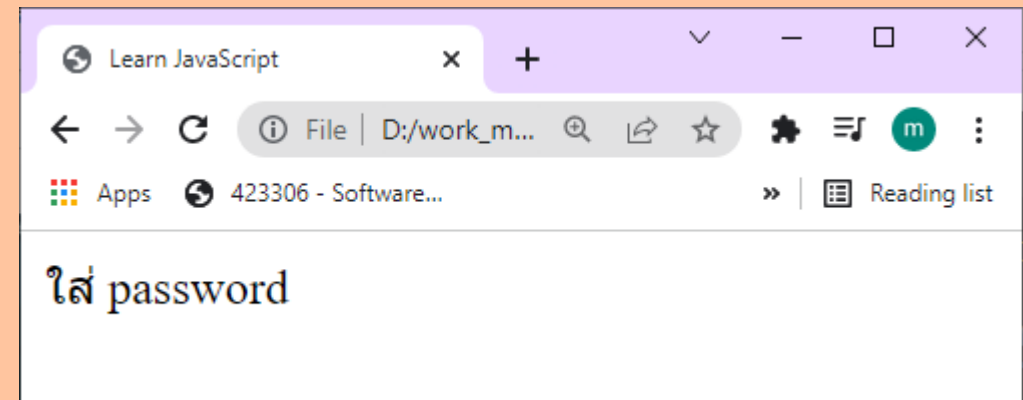
คำสั่งที่ใช้กำหนดเงื่อนไข



- 3. เงื่อนไขที่ใช้ **if...else if...else** เป็นการกำหนดทางเลือกอื่น ๆ ที่มีเงื่อนไขก็ได้ โดยหลักการทำงานคือ ทำการตรวจสอบเงื่อนไข ภายใน if ถ้าเงื่อนไขเป็นจริง ก็จะทำคำสั่งที่อยู่ภายในบล็อก ถ้าหากไม่จริง ก็จะไปเช็คเงื่อนไขถัดไป ถ้าพบว่าจริงก็จะทำคำสั่ง ถ้าเท็จก็เช็คเงื่อนไขต่อไปเรื่อย ๆ มีรูปแบบดังนี้

```
if (เงื่อนไข1){  
    // คำสั่งกรณีตรงกับเงื่อนไข1  
}  
else if (เงื่อนไข2){  
    // คำสั่งกรณีตรงกับเงื่อนไข2  
}  
else{  
    // หากไม่ตรงกับเงื่อนไขที่ 1 และ 2 ก็จะใช้คำสั่งนี้แทน  
}
```

```
let password = '';  
if(password == ''){  
    document.write("ใส่ password");  
}  
else if (password.length >= 8 && password.length<=12){  
    document.write("password มีขนาดถูกต้อง");  
}else{  
    document.write("password มีขนาดไม่ถูกต้อง");  
}
```



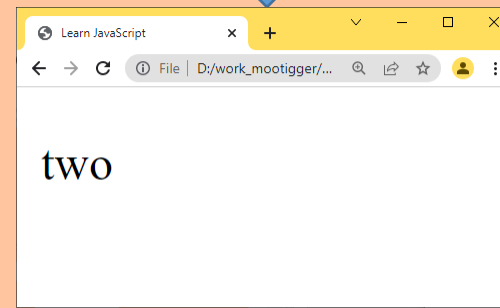
คำสั่งที่ใช้กำหนดเงื่อนไข



- เงื่อนไขแบบ **Switch** จะใช้การตรวจสอบเพียงครั้งเดียวแล้วไปดูว่าเข้ากับ case ไหน ก็จะทำตามคำสั่งใน case นั้นไป แต่ถ้าไม่ตรงกับ case ใด ๆ เลย ก็จะทำตามคำสั่งใน default

```
switch(สิ่งที่ต้องการตรวจสอบ) {  
  case 1:  
    คำสั่งที่ 1  
    break;  
  case 2:  
    คำสั่งที่ 2  
    break;  
  default:  
    กรณีที่ไม่ตรงกับคำสั่งใดๆเลย  
}
```

```
<body>  
  <p id="demo"></p>  
  <script>  
    var i=2  
    switch (i) {  
      case 1:var x = "one"; break;  
      case 2:var x = "two"; break;  
      case 3:var x = "three"; break;  
      default:x="ไม่ตรงกับค่าไหนเลย";  
    }  
    document.getElementById("demo").innerHTML = x;  
  </script>  
</body>
```



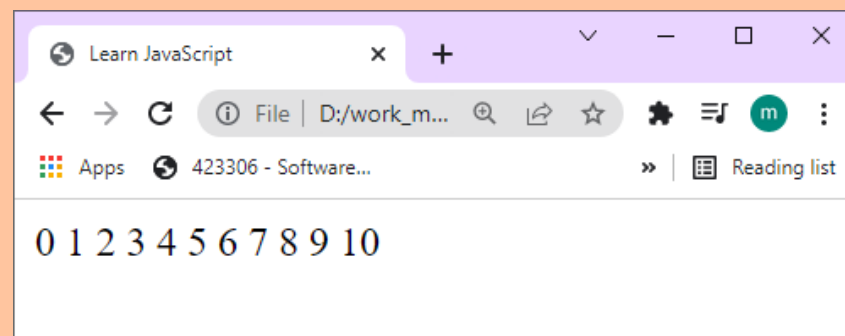
การวนลูปแบบ for



- การวนลูปแบบ **for** เป็นการทำงานแบบซ้ำ ๆ ตามเงื่อนไขที่กำหนด โดยลูปแบบ for จะต้องมิตัวแปรที่เป็นตัวนับสำหรับกำหนดค่าเริ่มต้น และค่าสุดท้ายของการทำซ้ำ และมีการกำหนดการเพิ่มหรือลดค่าของตัวแปรที่เป็นตัวนับ มีรูปแบบคำสั่งดังนี้

```
for (ตัวแปร=ค่าเริ่มต้น; เงื่อนไข; การเปลี่ยนค่าตัวแปร){  
    // เมื่อเงื่อนไขเป็นจริงจะทำตามคำสั่งนี้  
}
```

```
for (let i=0;i<=10;i++){  
    document.write( i + " " );  
}
```



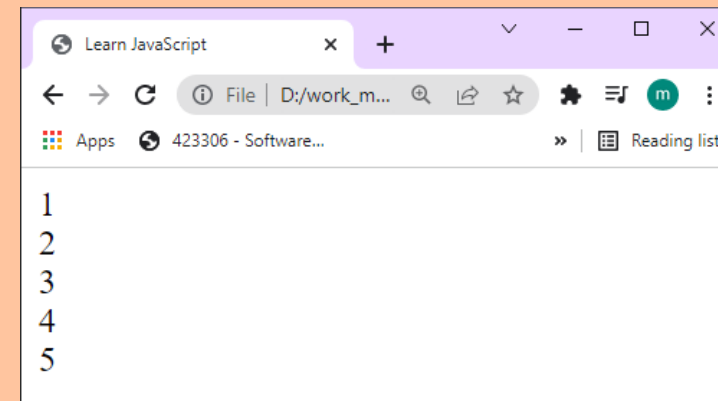
การวนลูปแบบ while



- การวนลูปแบบ **while** เป็นการทำงานแบบซ้ำ ๆ ตามเงื่อนไขที่กำหนด การทำงานของ Loop จะทำการตรวจสอบเงื่อนไขก่อน หากผลการตรวจสอบแล้วเป็นตามที่เงื่อนไขระบุ ก็จะทำตามคำสั่ง แต่หากเป็นเท็จก็จะกระโดด ออกจาก Loop

```
while (เงื่อนไข) {  
    // คำสั่ง  
}
```

```
var x=1;  
while (x <= 5){  
    document.write(x+ "<br>");  
    x++;  
}
```



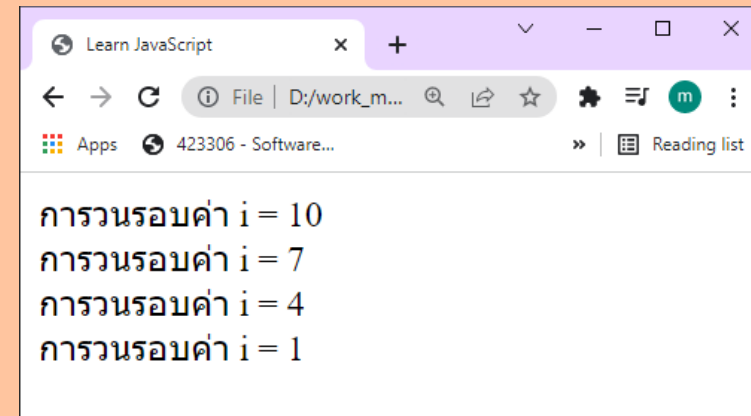
การวนรูปแบบ do...while



- **do...while** เป็นการทำงานแบบซ้ำ ๆ ตามเงื่อนไขที่กำหนด แต่จะทำตามคำสั่งใน { } ก่อนถึงจะทำการตรวจสอบเงื่อนไขที่ท้ายลูป

```
do{  
    // คำสั่ง  
}while (เงื่อนไข);
```

```
var i=10;  
do {  
    document.write("การวนรอบค่า i = "+i);  
    document.write ("<br>");  
    i=i-3;  
} while (i>=0);
```

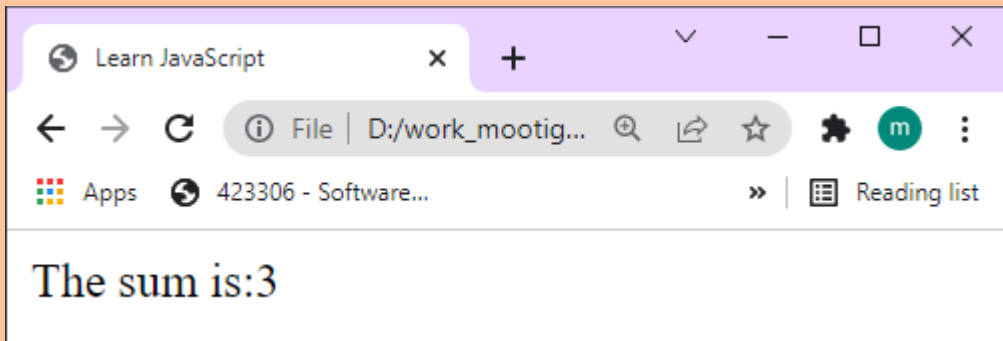


Break และ Continue

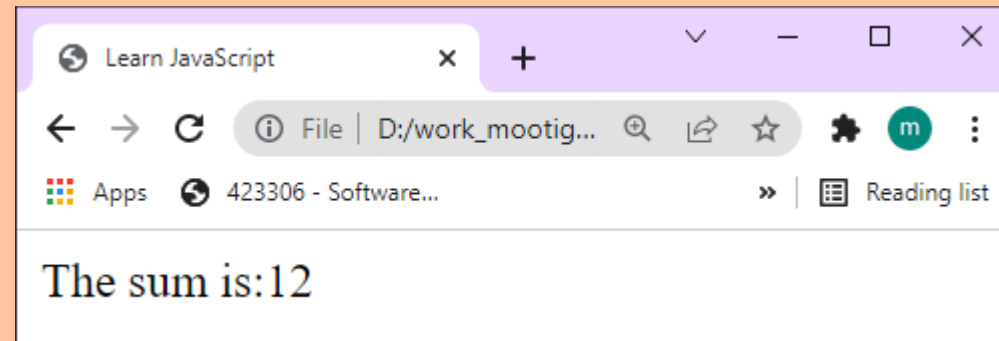


- **break** เป็นการให้ "ออกจากลูป" โดยหยุดการทำงานทั้งหมดของลูปนั้น ๆ
- **continue** เป็นการสั่งให้ "วนลูปถัดไป" ออกจากลูปปัจจุบันและไปเริ่มต้นใหม่ ต่างกันกับ break ตรงที่จะไม่หยุดการทำงานในลูปทั้งหมด

```
var sum = 0;
for (var i=1;i<=5;i++) {
    if (i==3){
        break;
    }
    sum+= i ;
}
document.write("The sum is:"+sum);
```



```
var sum = 0;
for (var i=1;i<=5;i++) {
    if (i==3){
        continue;
    }
    sum+= i ;
}
document.write("The sum is:"+sum);
```



ข้อควรระวังของ let กับ var เมื่อใช้กับ loop



- ขอบเขตการมองเห็นตัวแปร
 - var → ตัวแปรยังอยู่ นอกloop
 - let → ตัวแปรจะ จำกัดในloop เท่านั้น (อยู่แค่ใน { } ที่ครอบมัน)

```
for (var i = 0; i < 3; i++) {  
  console.log("ในloop var:", i);  
}  
console.log("นอกloop var:", i); // ใช้ได้ (i = 3)
```

```
for (let j = 0; j < 3; j++) {  
  console.log("ในloop let:", j);  
}  
console.log("นอกloop let:", j); // Error (j ไม่ถูกประกาศนอกloop)
```



ฟังก์ชัน (Function)



- ฟังก์ชัน เป็นชุดคำสั่งที่เขียนเก็บแยกเอาไว้ แล้วสามารถเรียกใช้งานได้เมื่อต้องการใช้งาน
- ใช้เพื่อลดการเขียนโค้ดซ้ำ ทำให้โค้ดอ่านง่าย แบ่งการทำงานของโค้ดเป็นส่วน ๆ
- การประกาศฟังก์ชัน (Function Declaration)

```
function nameOfFunction() {  
    .....  
}  
  
nameOfFunction(); // เรียกใช้งานฟังก์ชัน nameOfFunction()
```

- เช่น

```
function sayHello() {  
    console.log("สวัสดี");  
}  
  
sayHello(); // เรียกใช้งาน → สวัสดี
```



ฟังก์ชัน (Function)



- ฟังก์ชันที่มีพารามิเตอร์ (Parameters)

```
function greet(name) {  
    console.log("สวัสดีคุณ " + name);  
}  
  
greet("Maprang"); // สวัสดีคุณ Maprang  
greet("Somchai"); // สวัสดีคุณ Somchai
```

- ฟังก์ชันที่ส่งค่ากลับ (Return)

```
function add(a, b) {  
    return a + b;  
}  
  
let sum = add(5, 3);  
console.log(sum); // 8
```



ฟังก์ชัน (Function)



- ฟังก์ชันแบบ Expression (Function Expression) สามารถเก็บฟังก์ชันไว้ในตัวแปรได้
 - นิยมใช้ const กับฟังก์ชันป้องกันการถูกเปลี่ยนค่าฟังก์ชันโดยไม่ได้ตั้งใจ

```
const multiply = function(x, y) {  
  return x * y;  
};  
  
console.log(multiply(4, 5)); // 20  
multiply = 123; //Error (Assignment to constant variable)
```



ฟังก์ชันแบบ Arrow



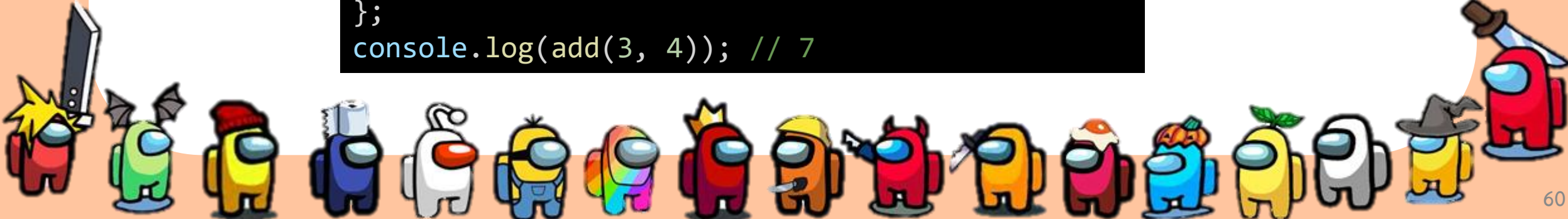
- ฟังก์ชันแบบ Arrow เป็น รูปแบบย่อของ Function Expression ที่เพิ่มเข้ามาใน ES6 (ECMAScript 2015)

- รูปแบบพื้นฐาน

```
const ชื่อตัวแปร = (พารามิเตอร์) => {  
  // โค้ดในฟังก์ชัน  
};
```

- ตัวอย่างรูปแบบปกติ

```
const add = (a, b) => {  
  return a + b;  
};  
console.log(add(3, 4)); // 7
```



ฟังก์ชันแบบ Arrow



- ตัวอย่างรูปแบบปกติ

```
const add = (a, b) => {  
  return a + b;  
};  
console.log(add(3, 4)); // 7
```

- ย่อได้ถ้ามีคำสั่งเดียว (implicit return)

```
const add = (a, b) => a + b;  
console.log(add(5, 10)); // 15
```

- ถ้ามีพารามิเตอร์เดียว ไม่ต้องใส่วงเล็บ

```
const square = n => n * n;  
console.log(square(6)); // 36
```

- ถ้าไม่มีพารามิเตอร์ ต้องใส่วงเล็บว่าง

```
const hello = () => console.log("Hello World!");  
hello(); // Hello World!
```



การแจ้งเตือนด้วย alert



- alert() ใช้เพื่อ แสดงกล่องข้อความ (dialog box) ที่มีปุ่ม OK เพียงปุ่มเดียว
- จะ หยุดการทำงานของสคริปต์ชั่วคราว จนกว่าผู้ใช้จะกด OK
- ถ้าใช้เยอะเกินไป จะทำให้ผู้ใช้รำคาญ
- ใช้สำหรับ debug แบบง่าย ๆ หรือ แจ้งเตือนเล็กน้อย เท่านั้น

```
<script>  
  alert("สวัสดีค่ะ! ยินดีต้อนรับเข้าสู่เว็บไซต์");  
  let name = "Maprang";  
  alert("Hello, " + name + "!");  
</script>
```

This page says

สวัสดีค่ะ! ยินดีต้อนรับเข้าสู่เว็บไซต์

OK

This page says

Hello, Maprang!

OK

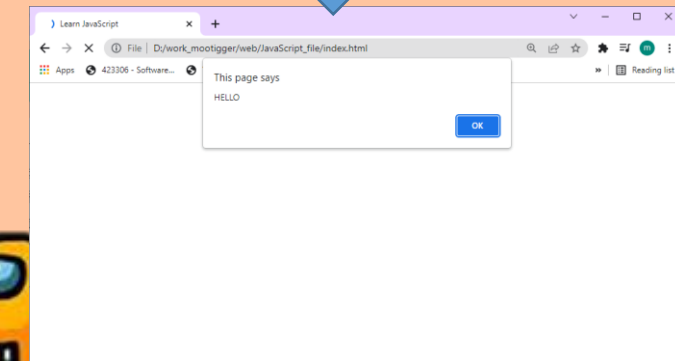
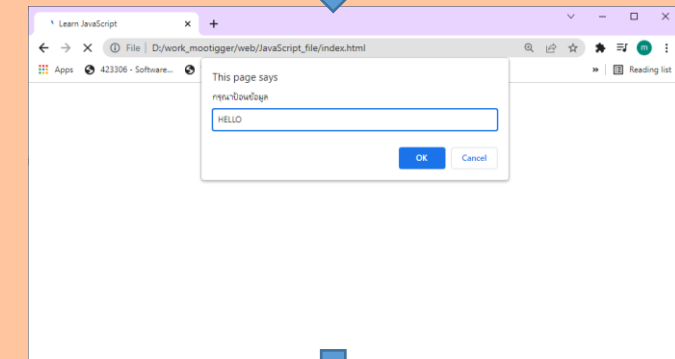
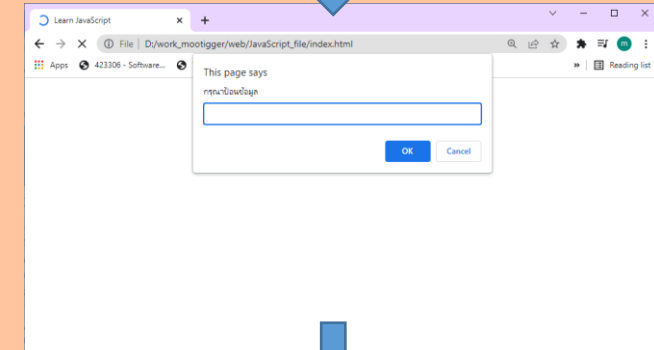


รับข้อมูลจากผู้ใช้โดยใช้ prompt



- คำสั่ง prompt ใช้สำหรับการแสดง popup เพื่อให้ผู้ใช้งานกรอกข้อมูลเข้ามา ซึ่งเมื่อ popup ชนิดนี้แสดงขึ้นมาจะมีช่องให้กรอกข้อมูลเข้ามา และมีปุ่ม ตกลง กับยกเลิก เมื่อกด ตกลง จะ return ค่าที่ได้กรอกเข้าไป แต่ถ้ากดยกเลิกจะ return ค่า null
- รูปแบบการใช้ `ตัวแปร = prompt("ข้อความ","ค่าเริ่มต้น");`
 - ตัวแปร คือ ตัวแปรที่ใช้เก็บค่าที่รับจากผู้ใช้
 - ข้อความ คือ ข้อความที่บอกผู้ใช้งานว่าต้องการให้ทำอะไร
 - ค่าเริ่มต้น คือ ค่า หรือข้อความใด ๆ ที่ต้องการให้ปรากฏในช่องกรอกข้อมูลจะกำหนดหรือไม่ก็ได้ถ้าไม่กำหนดจะมีข้อความในช่องกรอกข้อมูลออกมา

```
let a;  
a = prompt("กรุณาป้อนข้อมูล");  
alert(a);
```



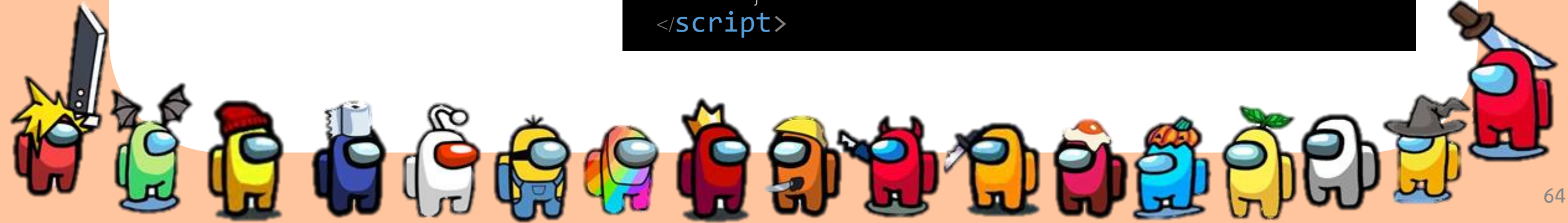
การยืนยันด้วย confirm()



- confirm() ใช้แสดง กล่องโต้ตอบ (dialog box)
- มีปุ่ม OK และ Cancel จะ
- คืนค่า (return) เป็น Boolean:
 - กด OK → คืนค่า true
 - กด Cancel → คืนค่า false

```
<script>
  let result = confirm("คุณต้องการลบข้อมูลหรือไม่?");
  console.log(result);
</script>
```

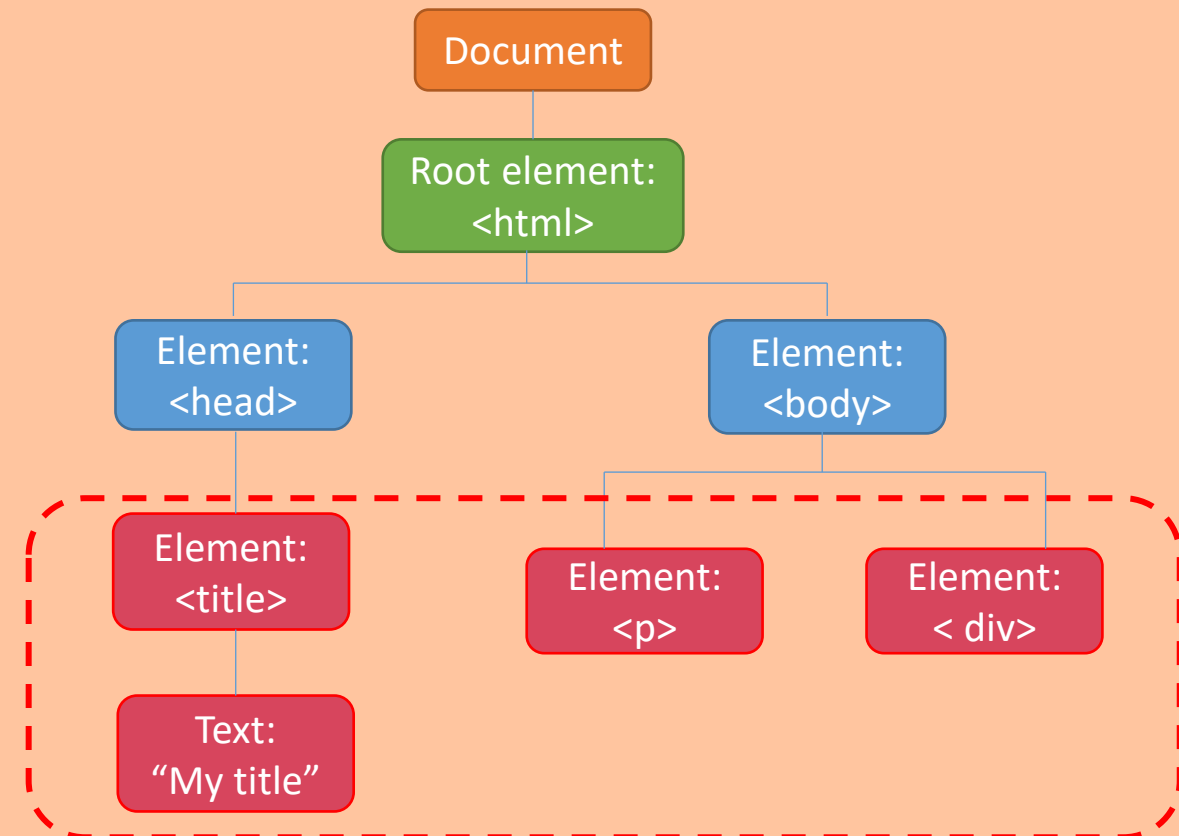
```
<script>
  if (confirm("คุณแน่ใจหรือไม่ที่จะออกจากระบบ?")) {
    alert("ออกจากระบบเรียบร้อยแล้ว");
  } else {
    alert("ยกเลิกการออกจากระบบ");
  }
</script>
```



Document Object Model (DOM)



- DOM หรือ Document Object Model คือโครงสร้างของไฟล์ที่ทาง W3C (World Wide Web Consortium หรือองค์กรระหว่างประเทศที่ทำหน้าที่จัดระบบมาตรฐานที่ใช้กันบนเวปไซด์ไวด์เว็บ) กำหนดขึ้นเป็นมาตรฐาน ซึ่งจะช่วยให้นักพัฒนาสามารถเขียนโค้ดหรือสคริปต์เพื่อเข้าถึง, เพิ่มเติม, เปลี่ยนแปลง รวมไปถึงลบเนื้อหาหรือโครงสร้างของไฟล์เอกสารประเภท HTML หรือ XML ได้
- ถูกออกแบบเพื่อจัดเก็บโครงสร้างเอกสาร HTML ซึ่งใน DOM จะมอง element ของ HTML ในรูปของ Object โดยที่ JavaScript สามารถจัดการกับเอกสาร HTML ผ่านโครงสร้างของ DOM ซึ่งเปรียบเสมือนเป็นพิมพ์เขียวของเอกสาร HTML
- DOM เป็นส่วนที่มองโครงสร้าง HTML เป็นแผนภาพต้นไม้ เมื่อหน้าเว็บโหลดเสร็จเรียบร้อยแล้ว Browser จะสร้าง DOM ของหน้านั้นขึ้นมา หมายถึง Browser จะมอง HTML เป็นต้นไม้ ดังนั้นการเขียนโค้ดเพื่อดึง element จาก DOM จึงต้องทำหลังสุด



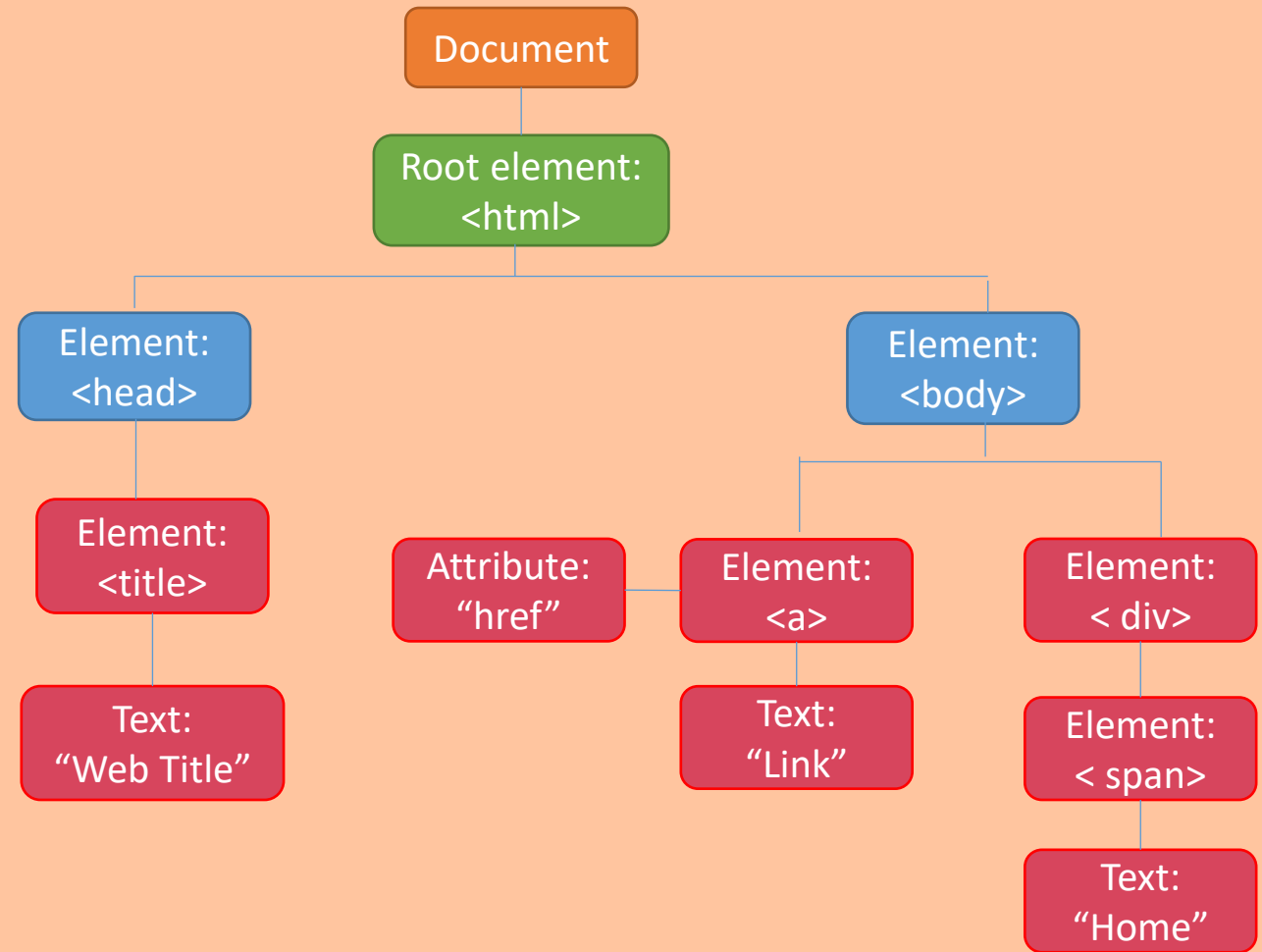
- - - ส่วนนี้จะมีการเปลี่ยนแปลงขึ้นอยู่กับไฟล์ HTML

Document Object Model (DOM)



```
<html>
  <head>
    <title>Web Title</title>
  </head>

  <body>
    <a href="test.html">Link</a>
    <div>
      <span>Home</span>
    </div>
  </body>
</html>
```



- **Element** หมายถึง Tag ของ HTML เช่น <html>, <head>, <title>, <body>, <a>
- **Text** หมายถึง ข้อความที่จัดเก็บอยู่ภายใน Element (Tag) นั้น เช่น <title>Web Title</title> : Text ของ Element (Tag) <title> ก็คือ “Web Title”
- **Attribute** หมายถึง คุณสมบัติของ Element (Tag) นั้น เช่น Link : Attribute ของ Element (Tag) <a> คือ href ซึ่งจัดเก็บ “test.html”

วิธีการเรียกใช้ DOM



- ในโค้ด JavaScript สามารถใช้ DOM จากตัวแปรชื่อ "document" ซึ่งเก็บ Element Object ทุกตัวในหน้าเว็บ
- Document จะมี method ที่ใช้ดึง element ของ HTML เพื่อนำมาแก้ไขค่า หรือทำให้หน้าเว็บมีการตอบโต้กับผู้ใช้
- เมื่อต้องการเรียกใช้ DOM ซึ่งเป็นวัตถุ (Object) ที่บรรจุในเอกสาร (document) ที่สร้างขึ้น โดยมีวิธีการค้นหาวัตถุใน DOM มี Method สำหรับดึง Element ที่ต้องการเพื่อแก้ไข มีทั้งหมด 5 วิธี
 - การเข้าถึง HTML Elements ด้วยการใช้อีดี `id`
`getElementById()`
 - การเข้าถึง HTML Elements ด้วยการใช้อีแท็ก `Tag`
`getElementsByTagName()`
 - การเข้าถึง HTML Elements ด้วยการใช้อีคลาส `Class`
`getElementsByClassName()`
 - การเข้าถึง HTML Elements ด้วยการใช้อี CSS selectors
`querySelector()`
 - การเข้าถึง HTML Elements ด้วยการใช้อี HTML Object collections
`document.forms[ชื่อforms]`

การเรียกใช้งาน DOM



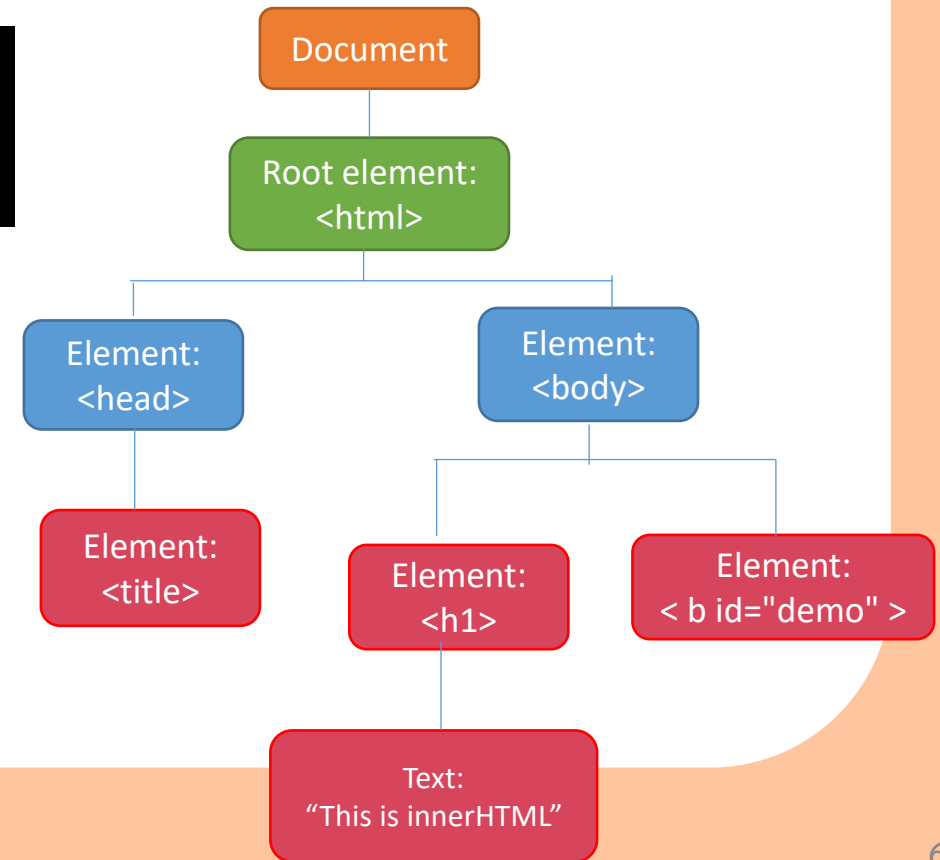
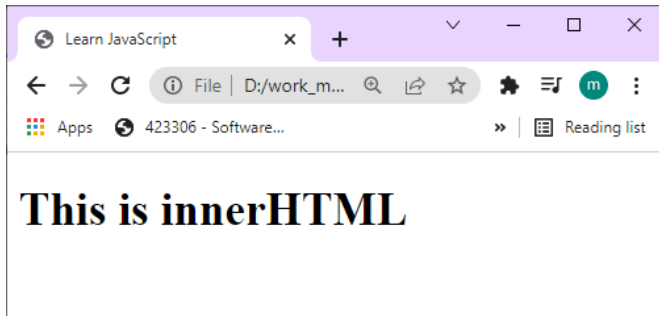
- Property ต่างๆ ของ DOM ที่สามารถอ่านหรือกำหนดได้ด้วย Javascript
 - innerHTML อ่านหรือกำหนดเนื้อหาภายใน element แบบ HTML
 - innerText อ่านหรือกำหนดเนื้อหาภายใน element แบบ Text
 - value อ่านข้อมูลที่อยู่ใน element ของฟอร์ม (form)
- setAttribute() กำหนดค่า attribute ของ element
- getAttribute() อ่านค่า attribute ของ element
- removeAttribute() ลบ attribute ของ element
- firstChild คืนค่า element แรกที่พบภายใน element ที่กำหนด
- className อ่านหรือกำหนดค่า CSS class
- id อ่านหรือกำหนด ID ของแต่ละ element เพื่อใช้อ้างอิงโดย Javascript และ CSS ค่านี้จะต้องไม่ซ้ำกันในแต่ละหน้า
- style อ่านหรือกำหนดค่า การแสดงผลของแต่ละ element
- createElement('tagName') สร้าง element โดยระบุชื่อแท็กของ element นั้น
- appendChild() ใช้สำหรับแทรก element ใดๆ ลงใน element ที่กำหนด โดย element ที่แทรกจะเป็น โหนดลูก (child node) ของ element ที่แทรก
- insertBefore() สำหรับการแทรก Elements ด้านหน้า Elements ที่ต้องการ

การใช้ getElementById เพื่อใช้แสดงผล (1)



- `document.getElementById(id)` คือ คำสั่งสำหรับการเข้าถึง Element ที่ต้องการผ่าน Attribute id ใน HTML
 - ถ้าใช้ร่วมกับ `innerText` คือเข้าไปเปลี่ยนแปลงข้อความที่อยู่ระหว่างแท็กเปิดและแท็กปิดใน HTML element

```
<body>
  <h1>This is innerHTML</h1>
  <b id="demo"></b>
</body>
```



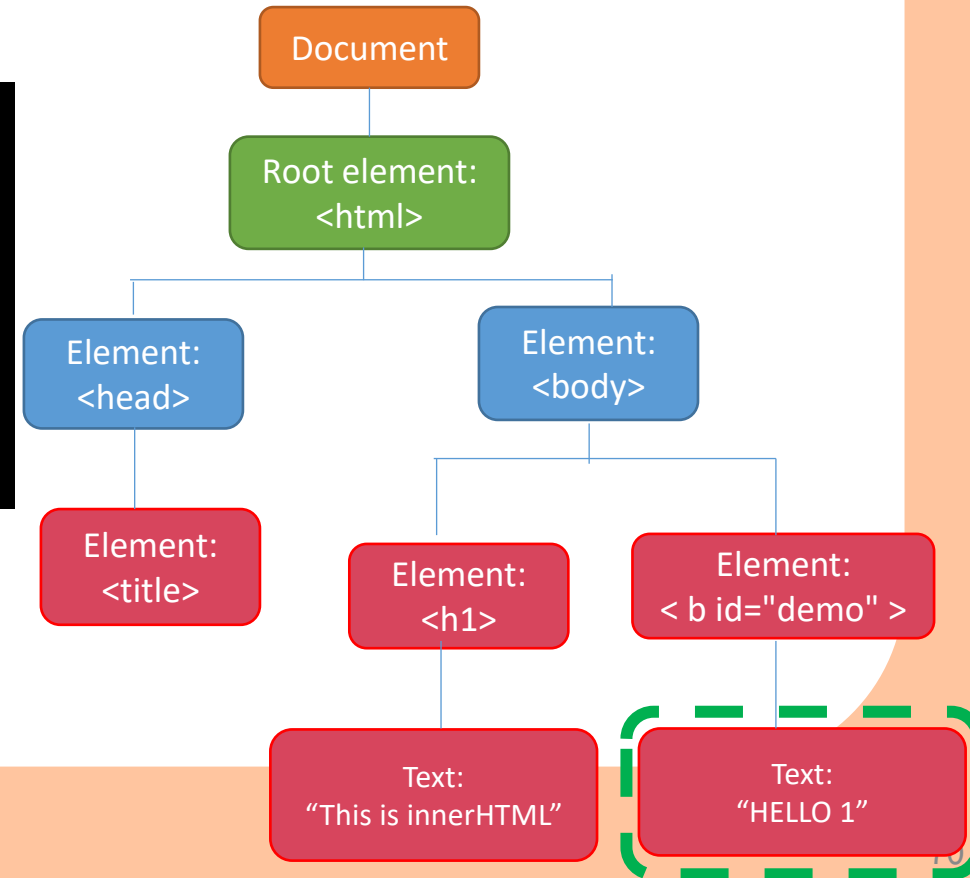
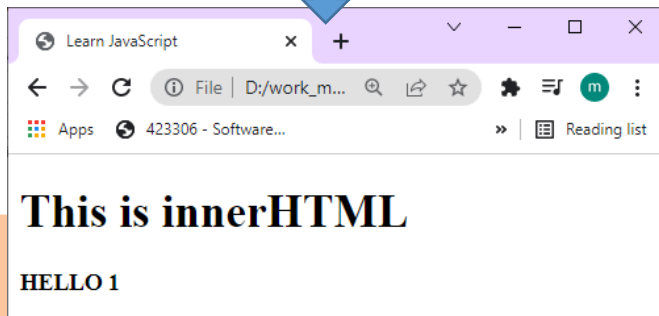
การใช้ getElementById เพื่อใช้แสดงผล (2)



- **document.getElementById(id)** คือ คำสั่งสำหรับการเข้าถึง Element ที่ต้องการผ่าน Attribute id ใน HTML
 - โดย **id** คือ ค่า id ของ tag ที่ต้องการเข้าถึง
- ถ้าใช้ร่วมกับ **innerHTML** คือเข้าไปเปลี่ยนแปลงข้อความที่อยู่ระหว่างแท็กเปิดและแท็กปิดใน HTML element และสามารถใส่โค้ด HTML ลงไปได้

```
<body>
  <h1>This is innerHTML</h1>
  <b id="demo"></b>

  <script>
    var x = 1;
    document.getElementById("demo").innerHTML = "HELLO " + x;
  </script>
</body>
```

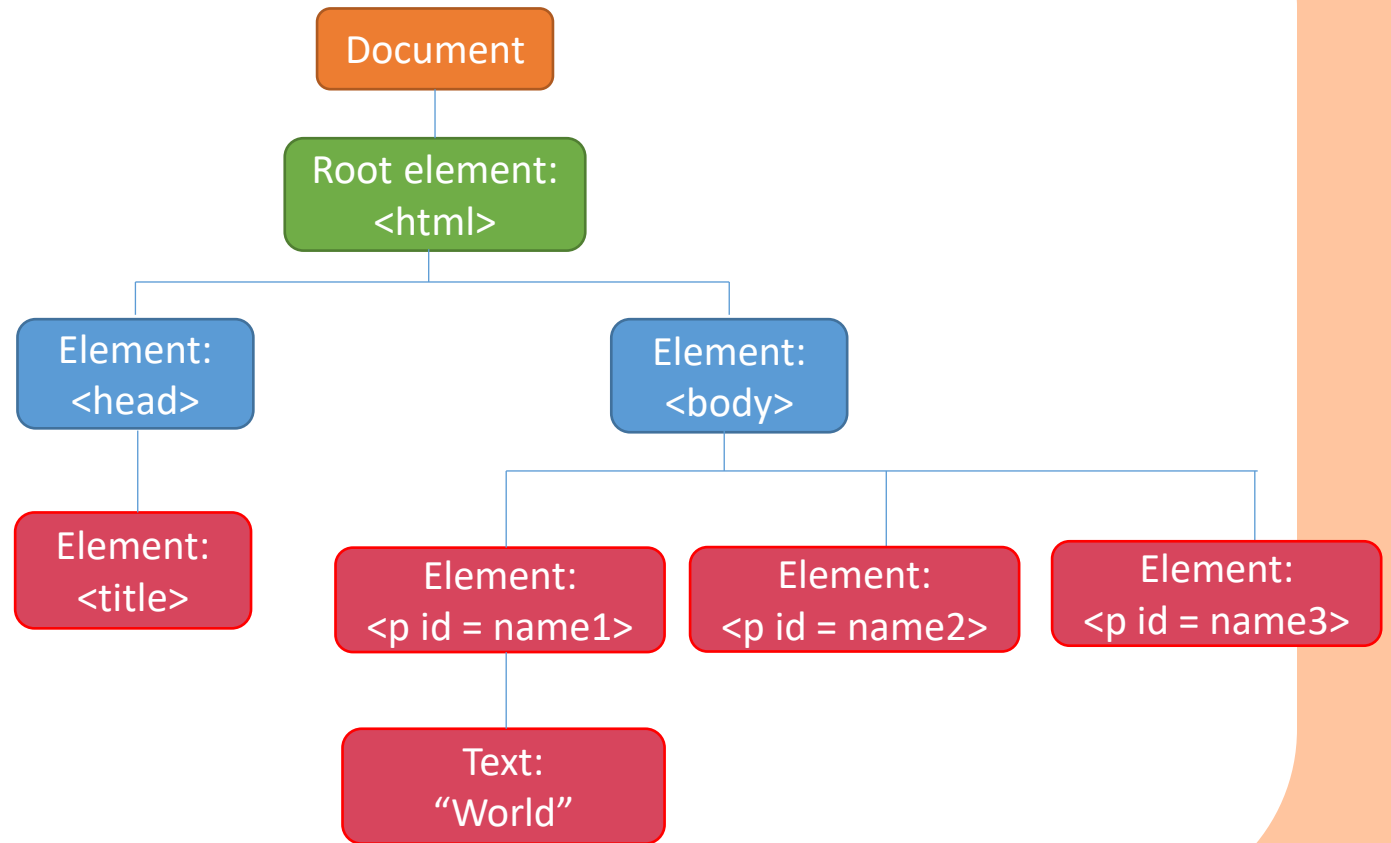
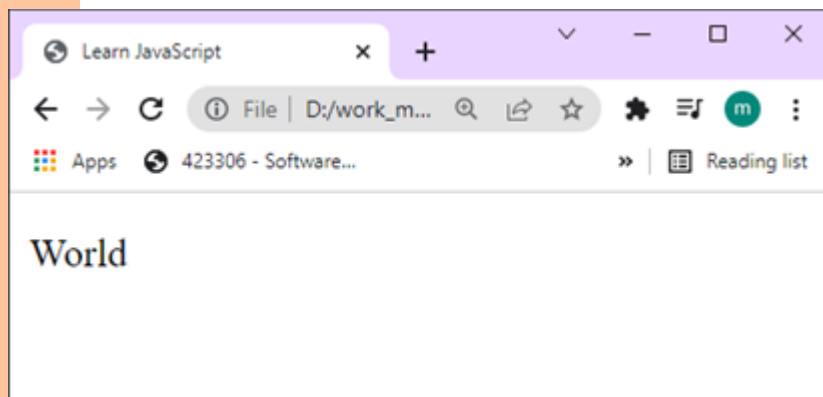


การเข้าถึง HTML Elements ด้วยการใช้ id



- ใช้ `.getElementById(id)` เพื่อเข้าถึง Elements ที่ด้วยชื่อ id

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Document</title>
</head>
<body>
  <p id="name1">World</p>
  <p id="name2"></p>
  <p id="name3"></p>
</body>
</html>
```

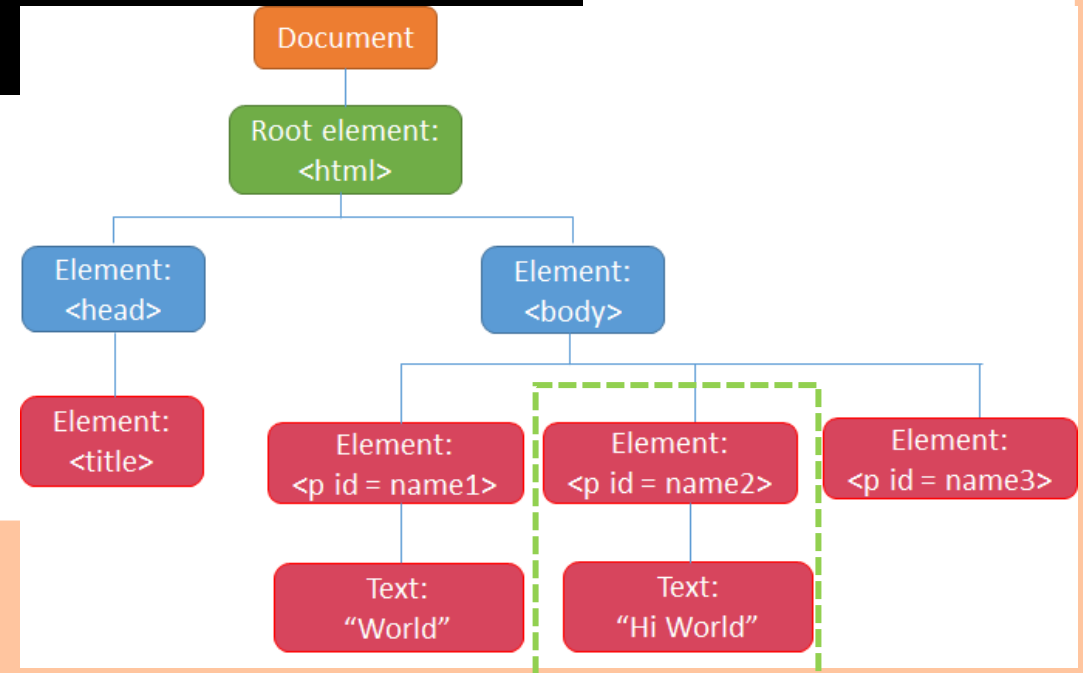
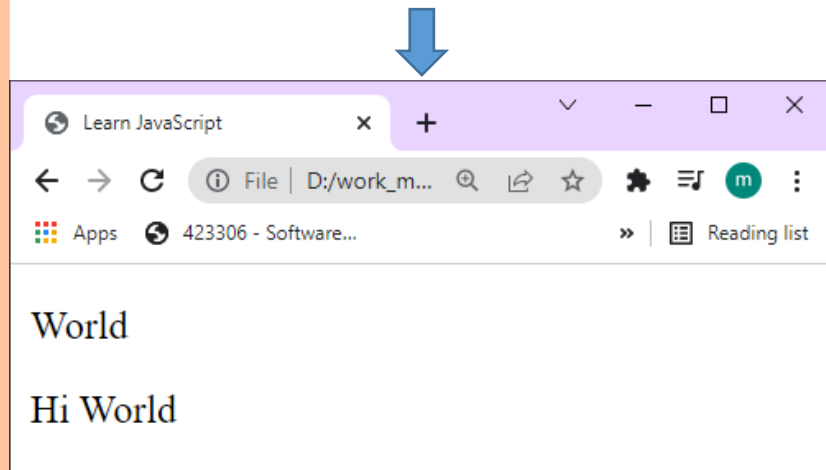


การเข้าถึง HTML Elements ด้วยการใช้ id



- ใช้ `.getElementById(id)` เพื่อเข้าถึง Elements ที่ด้วยชื่อ id

```
<body>
  <p id="name1">World</p>
  <p id="name2"></p>
  <p id="name3"></p>
  <script>
    //การอ้างอิง Element ที่มี attribute id เท่ากับ "name1" ในไฟล์แล้วนำมาเก็บอยู่ในตัวแปรที่ชื่อว่า myObject
    let myObject = document.getElementById('name1');
    //การเปลี่ยนแปลงเนื้อหาหรือส่วน text ของ Element ที่มี attribute id เท่ากับ "name2" ให้มีค่า Hi World
    document.getElementById('name2').innerHTML = 'Hi ' + myObject.innerHTML;
  </script>
</body>
```



การเข้าถึง HTML Elements ด้วยการใช้อ้างอิง Tag



- ใช้ `.getElementsByTagName(tag name)` เพื่อเข้าถึง Elements ที่ด้วยชื่อ tag

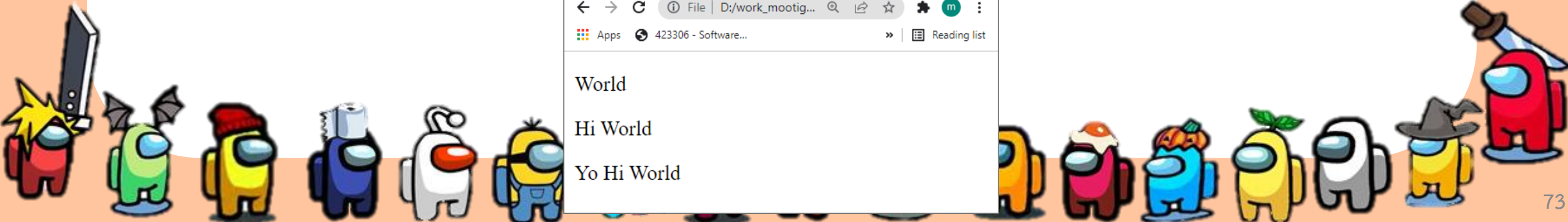
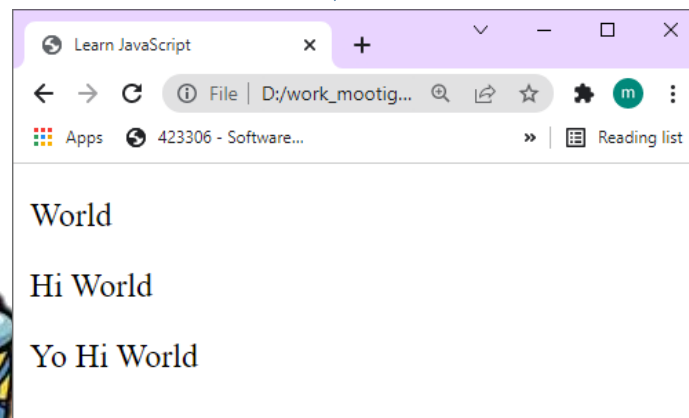
```
<body>
  <p id="name1">World</p>
  <p id="name2">Hello</p>
  <p id="name3"></p>
  <script>
    let myObject = document.getElementsByTagName('p');
    document.getElementById('name2').innerHTML = 'Hi ' + myObject[0].innerHTML;
    document.getElementById('name3').innerHTML = 'Yo ' + myObject[1].innerHTML;
  </script>
</body>
```

```
var myObject = document.getElementsByTagName('p');
```

`myObject [0] =` `<p id="name1">World</p>`

`myObject [1] =` `<p id="name2">Hello</p>`

`myObject [2] =` `<p id="name3"></p>`



การเข้าถึง HTML Elements ด้วยการใช้อ้างอิง Tag

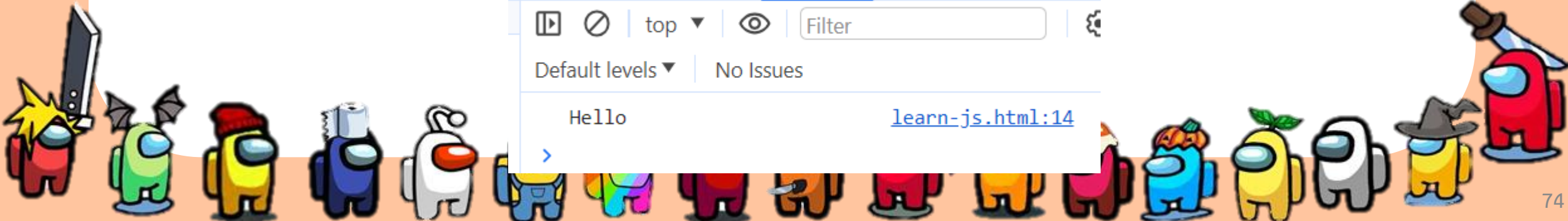
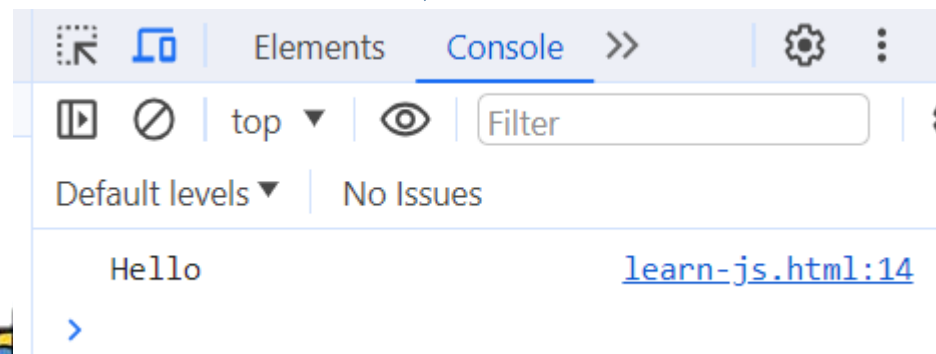


- ใช้ `.getElementsByName(tag name)` เพื่อเข้าถึง Elements ที่ด้วยชื่อ tag

```
<body>
  <p id="name1">World</p>
  <p id="name2">Hello</p>
  <p id="name3"></p>
  <script>
    let myObject = document.getElementsByTagName('p')[1].innerText;
    console.log(myObject);
  </script>
</body>
```

```
var myObject = document.getElementsByTagName('p');
```

myObject [0] =	<p id="name1">World</p>
myObject [1] =	<p id="name2">Hello</p>
myObject [2] =	<p id="name3"></p>



การเข้าถึง HTML Elements ด้วยการใช้อีชื่อ Class



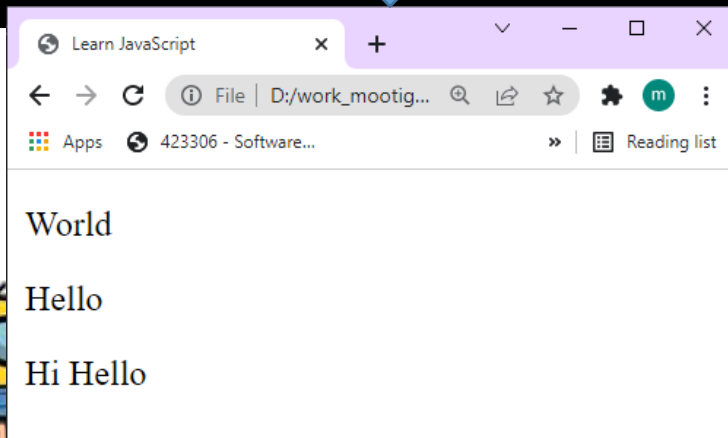
- ใช้ `.getElementsByClassName(Class name)` เพื่อเข้าถึง Elements ที่ด้วยชื่อ class
- ปัญหาคือ `getElementsByClassName` ไม่ส่งคืนองค์ประกอบ แต่จะส่งคืน `NodeList` อาจจะต้องระบุอ้างอิงว่าใช้ตัวไหน เพราะ class สามารถมีชื่อซ้ำได้

```
<body>
  <p class="name1">World</p>
  <p class="name1">Hello</p>
  <p id="name2"></p>
  <script>
    let myObject = document.getElementsByClassName('name1');
    document.getElementById('name2').innerHTML = 'Hi ' + myObject[1].innerHTML;
  </script>
</body>
```

```
var myObject = document.getElementsByClassName('name1');
```

```
myObject [0] = <p class="name1">World</p>
```

```
myObject [1] = <p class="name1">Hello</p>
```

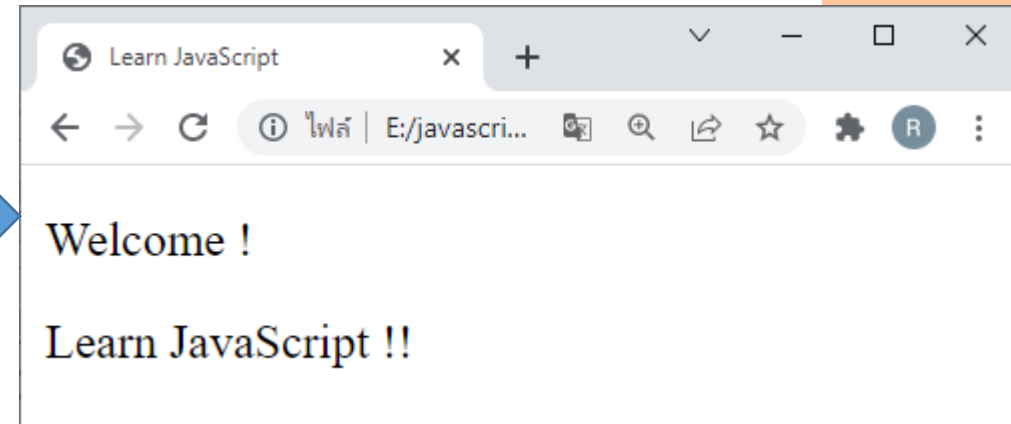


การเข้าถึง HTML Elements ด้วยการใช้ CSS selectors

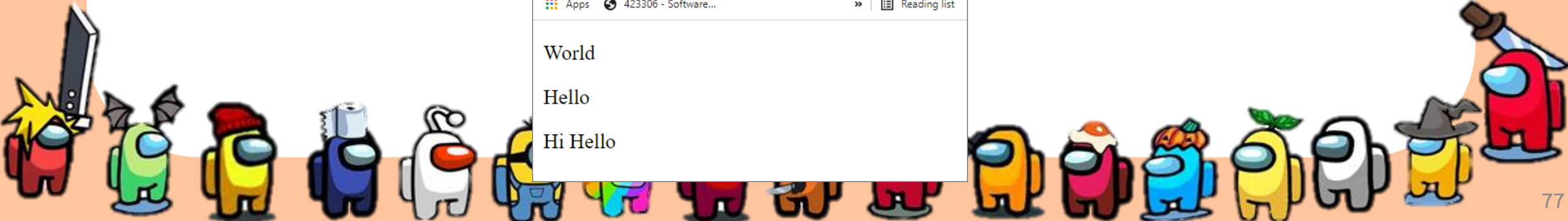
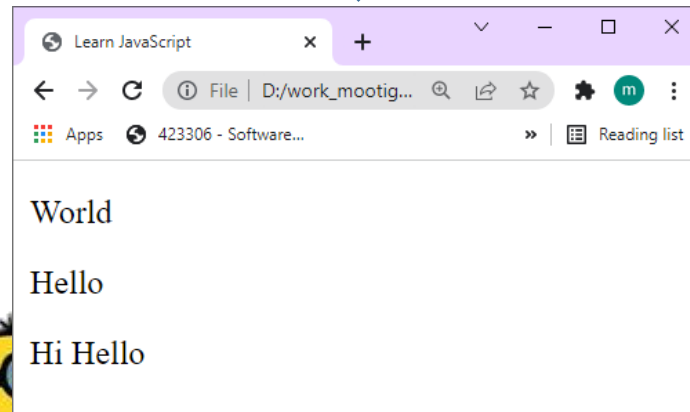


- `.querySelector(CSS selectors);` เพื่อนำ CSS selector มาใช้ในการค้นหา element

```
<body>
  <p id="welcome">
    Welcome !
  </p>
  <p class="message">
    Learn JavaScript !!
  </p>
  <script>
    var p1 = document.querySelector('#welcome');
    var p1 = document.querySelector('.message');
  </script>
</body>
```



- ```
<body>
 <p class="name1">World</p>
 <p class="name1">Hello</p>
 <p id="name2"></p>
 <script>
 var myObject = document.querySelectorAll('p.name1');
 document.getElementById('name2').innerHTML = 'Hi ' + myObject[1].innerHTML;
 </script>
</body>
```

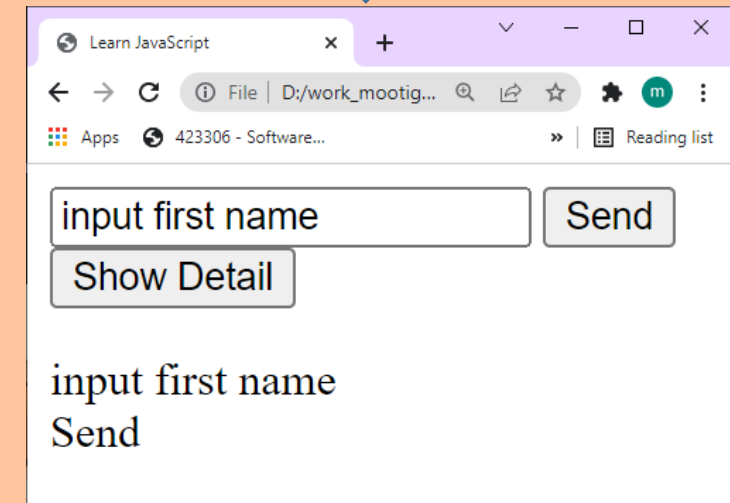
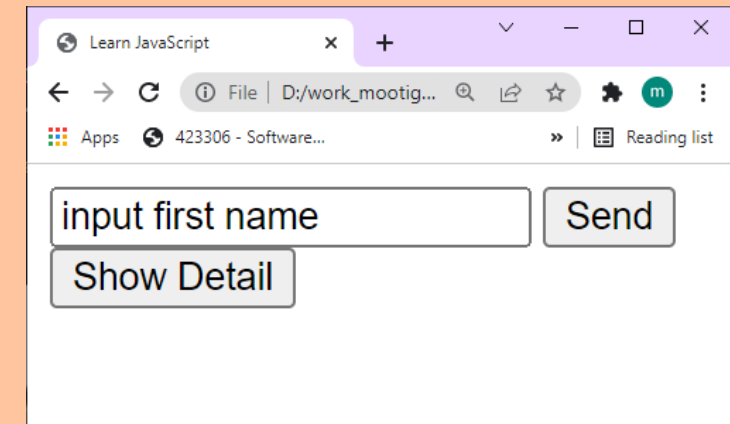


# การเข้าถึง HTML Elements ด้วยการใช HTML Object collections



- ใช้ `.document.forms[ชื่อforms]` หรือ `.document.forms.ชื่อforms` จะสามารถถึง Elements ทั้งหมดใน forms

```
<body>
 <form id="myForm">
 <input type="text" name="fname" value="input first name">
 <input type="submit" value="Send">
 </form>
 <button onclick="showDetail()">Show Detail</button>
 <p id="name1"></p>
 <script>
 function showDetail(){
 var i;
 var result = '';
 var data = document.forms['myForm'];
 for (i = 0; i < data.length; i++){
 result += data[i].value + '
';
 }
 document.getElementById('name1').innerHTML = result;
 }
 </script>
</body>
```



# Event ใน JavaScript



- **Event** คือ เหตุการณ์ที่เกิดขึ้นขณะที่ผู้ใช้งานกำลังดูหน้าเว็บนั้น ๆ อยู่
- การเพิ่ม attribute ที่ขึ้นต้นด้วย on.... ภายในแท็ก HTML และระบุฟังก์ชัน JavaScript จะทำให้เกิดการเรียกใช้ฟังก์ชันที่กำหนดอัตโนมัติ ตามเหตุการณ์ที่กำหนดไว้
  - ประเภทของ event เช่น
    - **Windows Event** เหตุการณ์ที่เกิดกับหน้าต่างเว็บเบราว์เซอร์
    - **Mouse Event** เหตุการณ์ที่ใช้เมาส์กับส่วนต่าง ๆ ในหน้าเว็บ
    - **Keyboard Event** เหตุการณ์ที่ใช้คีย์บอร์ดกับส่วนต่าง ๆ ในหน้าเว็บ
    - **Form Event** ที่เกิดกับส่วนต่าง ๆ ของฟอร์มใน HTML



# ประเภทของ event



- Event คือ "เหตุการณ์" ที่เกิดขึ้นกับ อิลิเมนต์ (Element) เช่น การคลิก การเลื่อนเมาส์ การกดแป้นพิมพ์ หรือการโหลดหน้าเว็บเมื่อ
- event เกิดขึ้น → สามารถเขียนโค้ดให้ทำงานตอบสนอง (event handler)
- ประเภทของ event เช่น
  - Windows Event เหตุการณ์ที่เกิดกับหน้าต่างเว็บเบราว์เซอร์
  - Mouse Event เหตุการณ์ที่ใช้เมาส์กับส่วนต่าง ๆ ในหน้าเว็บ
  - Keyboard Event เหตุการณ์ที่ใช้คีย์บอร์ดกับส่วนต่าง ๆ ในหน้าเว็บ
  - Form Event ที่เกิดกับส่วนต่าง ๆ ของฟอร์มใน HTML



# Windows Event



- Windows Event เป็น event ที่เกิดขึ้นกับ เอกสาร (document) หรือ หน้าต่าง (window) ทั้งหมด ไม่ใช่เฉพาะ element เดียว ๆ
  - onload → เมื่อหน้าเว็บโหลดเสร็จ ใช้กับ window, <body>
  - onresize → เมื่อย่อ/ขยายหน้าต่าง ใช้กับ window
  - onscroll → เมื่อเลื่อนหน้าจอ ใช้กับ window, document
  - onunload → เมื่อออกจากหน้าเว็บ ใช้กับ window, <body>
  - onfocus → เมื่อหน้าต่างได้รับโฟกัส ใช้กับ window



# Mouse Event



- Mouse Event คือเหตุการณ์ (event) ที่เกิดจากการกระทำของผู้ใช้ด้วยเมาส์ เช่น การคลิก การเลื่อน การชี้ ฯลฯ
- ใช้กับแทบทุก element (เช่น <button>, <div>, <img>)
  - onclick → เมื่อคลิก
  - ondblclick → ดับเบิลคลิก
  - onmouseover → เอาเมาส์ไปชี้
  - onmouseout → เอาเมาส์ออก
  - onmousedown → กดปุ่มเมาส์ลง
  - onmouseup → ปล่อยปุ่มเมาส์



# Keyboard Event



- **Keyboard Events** คือเหตุการณ์ (event) ที่เกิดขึ้นเมื่อผู้ใช้กดหรือปล่อยปุ่มบนแป้นพิมพ์ เหมาะสำหรับการตรวจจับ input, ทำข้อตัด, หรือควบคุมเกม
- onkeydown → กดปุ่มลง (ไม่ต้องรอปล่อย)
- onkeyup → ปล่อยปุ่มที่กดอยู่



# Form Event



- **Form Events (อีเวนต์ของฟอร์ม)** เกิดขึ้นเมื่อผู้ใช้มีปฏิสัมพันธ์กับ ฟอร์ม หรือ องค์ประกอบของฟอร์ม (เช่น `<input>`, `<select>`, `<textarea>`, `<form>`)
  - `onsubmit` → เมื่อส่งฟอร์ม
  - `onchange` → เมื่อค่ามีการเปลี่ยน
  - `onfocus` → เมื่อ element ได้ focus (เช่น เคอร์เซอร์มาที่ input)
  - `onblur` → เมื่อ element เสีย focus



# วิธีการผูก Event



- การผูก Event (Event Binding) คือการบอก browser ว่า “เมื่อเกิด event ที่ element นี้ → ให้ทำสิ่งนี้”

1. Inline Event Handler (เขียนใน HTML โดยตรง)
2. ใช้ DOM Property
3. ใช้ `addEventListener()` (วิธีที่นิยมที่สุด)

**\*\* Handler = ฟังก์ชันที่ถูกกำหนดขึ้นมาเพื่อ “จัดการ” เหตุการณ์ (Event) หรือการทำงานบางอย่าง**



# 1. Inline Event Handler (เขียนใน HTML โดยตรง)



- Inline Event Handler คือ การเขียนโค้ด JavaScript ไว้ตรง ๆ บน attribute ของ HTML element
- ข้อดี เข้าใจง่ายเหมาะกับตัวอย่างเล็ก ๆ หรือโค้ดสั้น ๆ จะเห็นทันทีว่า element นั้นมี event อะไร
- ข้อควรระวัง ไม่เหมาะกับโปรเจกต์ใหญ่ เพราะโค้ด JavaScript ปนอยู่ใน HTML ทำให้ยุ่งยากต่อการแก้ไข
- ใช้รูปแบบ: `<element onEvent="JavaScript code">`
- เช่น Alert เมื่อคลิก `<button onclick="alert('สวัสดี!')">คลิกฉัน</button>`
  - เมื่อกด → ดังกล่องแจ้งเตือน
- เช่น เรียกฟังก์ชัน

ผู้ handler เข้ากับ event

```
<button onclick="sayHello()">คลิกฉัน</button>
```

```
<script>
 function sayHello() {
 alert("สวัสดีครับ!");
 }
</script>
```

สร้าง handler

# 1. Inline Event Handler (เขียนใน HTML โดยตรง)



- เช่น ส่งค่าเข้าไป

```
<button onclick="greet('สมชาย')">สวัสดี สมชาย</button>
<button onclick="greet('สมหญิง')">สวัสดี สมหญิง</button>

<script>
 function greet(name) {
 alert("สวัสดี " + name);
 }
</script>
```



# 1. Inline Event Handler (เขียนใน HTML โดยตรง)



- ใน 1 element → สามารถมี attribute event ได้หลายชนิด เช่น

```
<button onclick="alert('คลิก!)" onmouseover="alert('เอาเมาส์มาวาง!)">
ทดสอบ
</button>
```

- แต่กับ event ชนิดเดียวกัน (เช่น onclick) เขียน attribute ได้แค่ 1 ครั้งเท่านั้น
- ถ้าอยากรันหลายคำสั่งต้องเขียน หลายคำสั่งต่อกันในช่องเดียว

```
<button onclick="alert('สวัสดี!'); console.log('Hello Console');">
คลิกฉัน
</button>
```



## 2. ใช้ DOM Property



- ใช้ DOM Property เป็น การเข้าถึงและแก้ไขค่า property ของ element ผ่าน JavaScript โดยตรง
- ตัวอย่าง property เช่น: innerHTML, value, src, onclick, onchange (สำหรับ event handler)

```
<button id="btn">คลิก</button>

<script>
 const btn = document.getElementById("btn");
 btn.onclick = function() {
 alert("คลิกแล้ว!");
 };
</script>
```

- ข้อดี: เข้าใจง่าย
- ข้อเสีย: ผูกได้แค่งานเดียว ถ้ามีการเขียน btn.onclick = ... ใหม่ อันเก่าจะหาย



## 2. ใช้ DOM Property



- ข้อเสีย: ผูกได้แค่งานเดียว ถ้ามีการเขียน btn.onclick = ... ใหม่ อันเก่าจะหาย
- "ฟังก์ชันแรก" จะไม่ทำงานแล้ว เพราะถูกทับด้วยอันที่สอง

```
<button id="btn">คลิกฉัน</button>

<script>
 const btn = document.getElementById("btn");

 // ฟังก์ชันแรก
 btn.onclick = function() {
 alert("อันนี้ฟังก์ชันแรก");
 };

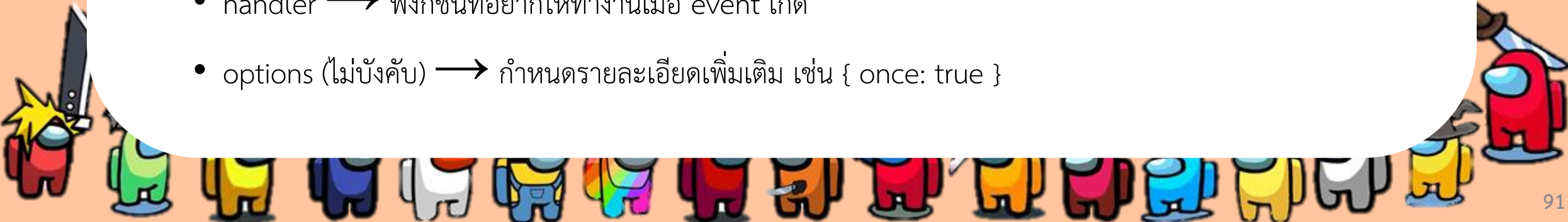
 // ฟังก์ชันที่สอง (เขียนทับอันแรก)
 btn.onclick = function() {
 alert("อันนี้ฟังก์ชันที่สอง");
 };
</script>
```



### 3. ใช้ addEventListener()



- addEventListener() เป็น เมธอดของ DOM element ที่ใช้สำหรับผูก event handler เข้ากับ element
- ทำให้ element หนึ่ง ๆ สามารถมี หลาย event handler ได้
- สามารถลบออกที่หลังได้ด้วย removeEventListener()
- รูปแบบการใช้งาน `element.addEventListener(event, handler, options);`
  - event → ชื่อเหตุการณ์ เช่น "click", "mouseover", "keydown" (ไม่มีคำว่า on)
  - handler → ฟังก์ชันที่อยากให้ทำงานเมื่อ event เกิด
  - options (ไม่บังคับ) → กำหนดรายละเอียดเพิ่มเติม เช่น { once: true }



### 3. ใช้ addEventListener()



- ตัวอย่างการใช้งาน
- 1) ปุ่มเดียว ผูกหลายฟังก์ชันได้

```
<button id="btn">คลิกฉัน</button>

<script>
 const btn = document.getElementById("btn");

 // ฟังก์ชันที่ 1
 btn.addEventListener("click", function() {
 alert("ฟังก์ชันแรกทำงาน");
 });

 // ฟังก์ชันที่ 2
 btn.addEventListener("click", function() {
 alert("ฟังก์ชันที่สองทำงาน");
 });
</script>
```

- เวลา คลิกปุ่ม → จะดึง "ฟังก์ชันแรกทำงาน" และ "ฟังก์ชันที่สองทำงาน" ต่อกัน



### 3. ใช้ addEventListener()



- ตัวอย่างการใช้งาน
- 2. ใช้ฟังก์ชันที่เขียนแยกไว้

```
<button id="btn">คลิกฉัน</button>

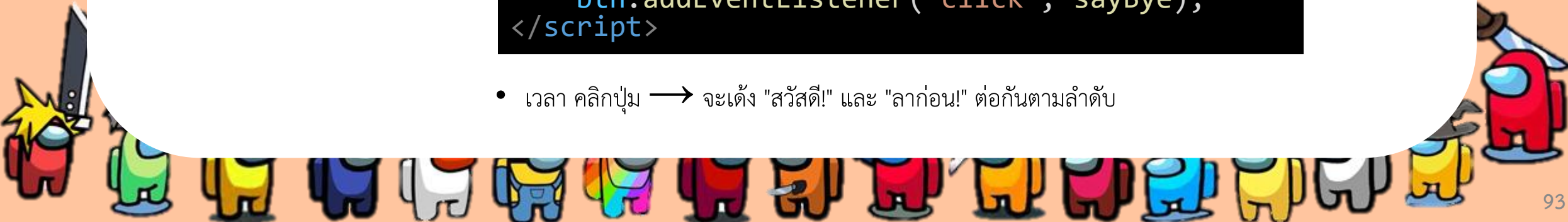
<script>
 const btn = document.getElementById("btn");

 function sayHello() {
 alert("สวัสดี!");
 }

 function sayBye() {
 alert("ลาก่อน!");
 }

 btn.addEventListener("click", sayHello);
 btn.addEventListener("click", sayBye);
</script>
```

- เวลา คลิกปุ่ม → จะแจ้ง "สวัสดี!" และ "ลาก่อน!" ต่อกันตามลำดับ



### 3. ใช้ addEventListener()



- ตัวอย่างการใช้งาน
- 3. ใช้ once (ทำงานแค่ครั้งเดียว)
  - ใช้ { once: true } เพื่อบอกให้ Browser ทราบว่า ให้รัน Event Handler แค่ครั้งเดียวเท่านั้น
  - หลังจากที Event นั้นเกิดขึ้นและ Handler ถูกเรียกใช้งาน → browser จะ ลบ handler ที่อัตโนมัติ

```
<button id="btn">คลิกฉัน</button>

<script>
 const btn = document.getElementById("btn");

 btn.addEventListener("click", () => {
 alert("คลิกครั้งเดียวเท่านั้น!");
 }, {once: true });
</script>
```



# ลบ Event ออก (removeEventListener)



- removeEventListener ใช้สำหรับ เอา event handler ออก จาก element ที่เคยถูกผูกไว้
  - ถ้าไม่ลบ → event handler จะอยู่ไปเรื่อย ๆ และถูกเรียกทุกครั้งเมื่อเกิด event
  - ถ้าลบ → event handler จะไม่ถูกเรียกอีก
- รูปแบบการใช้ `element.removeEventListener(event, handler, options);`
  - event → ชื่อ event เช่น "click", "keydown"
  - handler → ต้องเป็น ฟังก์ชันตัวเดียวกัน กับที่ใช้ตอน addEventListener
  - options → ค่าเสริม ไม่มีก็ได้



# ลบ Event ออก (removeEventListener)



```
<button id="btn">คลิกฉัน</button>

<script>
 const btn = document.getElementById("btn");

 // สร้าง handler ไว้เป็นตัวแปร
 function sayHello() {
 alert("สวัสดี!");
 }

 // ผูก event
 btn.addEventListener("click", sayHello);

 // ลบ event หลัง 5 วินาที
 setTimeout(() => {
 btn.removeEventListener("click", sayHello);
 alert("Event ถูกลบแล้ว!");
 }, 5000);
</script>
```

setTimeout() เป็น ฟังก์ชันของ JavaScript ที่ใช้สำหรับ หน่วงเวลา (delay) การทำงานของโค้ดตามเวลาที่กำหนด (หน่วยเป็น มิลลิวินาที)

รูปแบบ

```
setTimeout(function, delay, param1, param2, ...);
```

- function → ฟังก์ชันที่จะถูกเรียก
- delay → เวลาที่หน่วง (มิลลิวินาที: 1000ms = 1 วินาที)
- param1, param2,... → อาร์กิวเมนต์ที่จะส่งเข้าไปในฟังก์ชัน (optional)

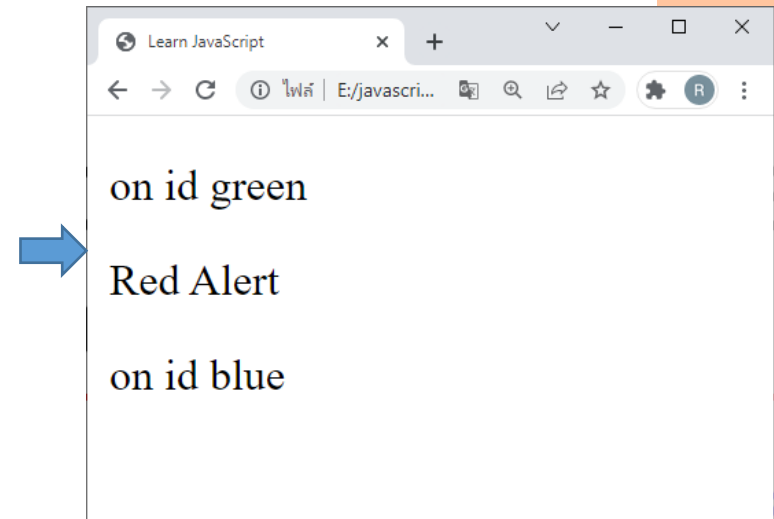
# การกำหนด event ใน JavaScript



1. ระบุ event ใน JavaScript
2. สร้างการทำงานเพื่อรองรับการเกิด event ใน JavaScript

ระบุ Event กำหนด function เพื่อรองรับ Event

```
<script>
 window.onload = function() {
 let planet = document.getElementById('red')
 planet.innerHTML = 'Red Alert'
 }
</script>
<p id="green">on id green</p>
<p id="red">on id red</p>
<p id="blue">on id blue</p>
```



# การกำหนด event ใน HTML



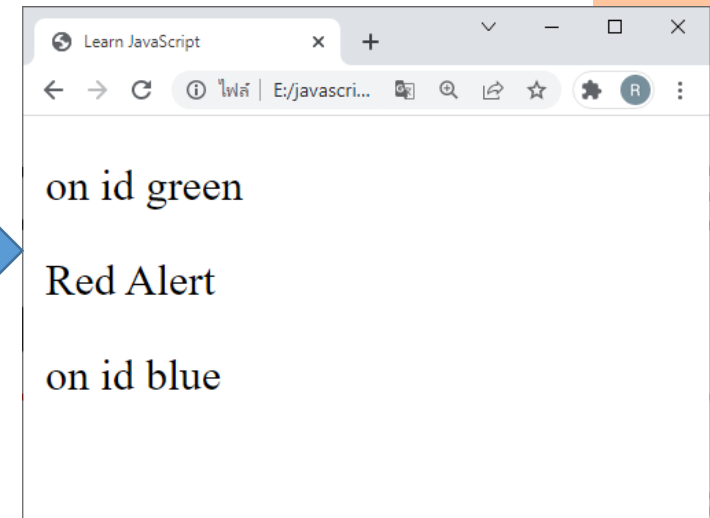
1. ระบุ event ใน HTML
2. สร้างการทำงานเพื่อรองรับการเกิด event ใน JavaScript

↓ ระบุ Event ใน HTML

↓ สร้าง function เพื่อรองรับ Event

```
<body onload="show()">
 <script>
 function show() {
 let planet = document.getElementById('red') //ประกาศตัวแปรสำหรับ
 เก็บ Element ที่ดึงออกมา โดยดึงเอา Element Object ที่มี id ชื่อว่า red ออกมา

 planet.innerHTML = 'Red Alert' //เปลี่ยนแปลงข้อความที่อยู่ระหว่างแท็กเปิดและ
 ปิดโดยกำหนดค่าใน innerHTML
 }
 </script>
 <p id="green">on id green</p>
 <p id="red">on id red</p>
 <p id="blue">on id blue</p>
</body>
```

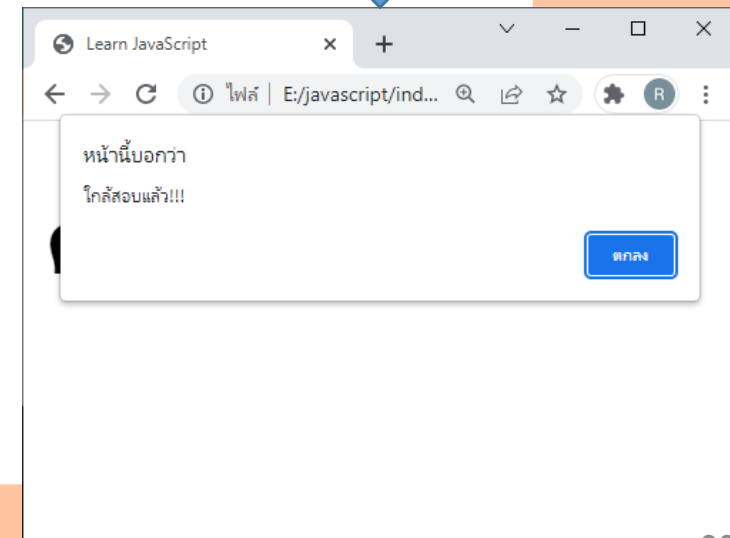
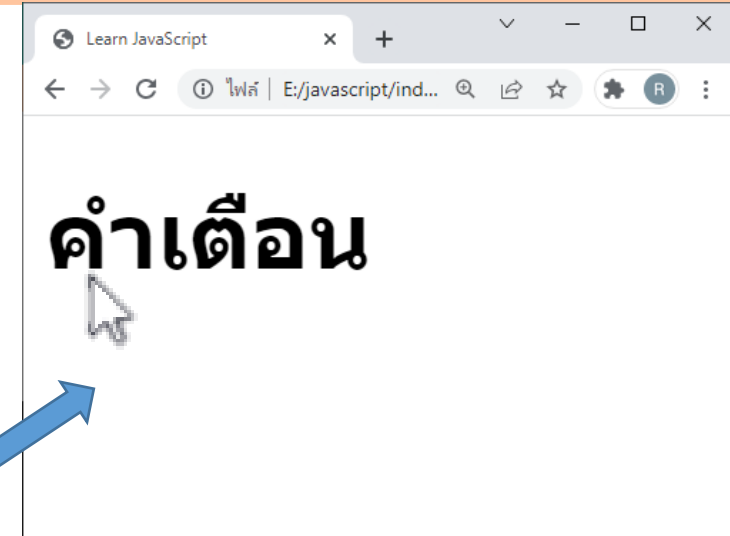


# การกำหนด event ใน JavaScript



1. ระบุ event ใน JavaScript
2. สร้างการทำงานเพื่อรองรับการเกิด event ใน JavaScript

```
<head>
 <meta charset="UTF-8">
 <title>Learn JavaScript</title>
 <script>
 function show() {
 alert("ใกล้สอบแล้ว!!!");
 }
 window.onload = function() { //การดึง Element ในส่วน<head> ฟังก์ชันจะทำงานหลังโหลดเว็บเสร็จแล้ว
 let h1 = document.getElementById('w1') // ดึงเอา Element ที่มีid=w1 ออกมาเพื่อใช้กำหนดเหตุการณ์
 h1.onmouseover = show //กำหนดให้เรียก function show เมื่อเกิดเหตุการณ์ onmouseover
 }
 </script>
</head>
<body>
 <h1 id="w1">คำเตือน</h1>
</body>
</html>
```

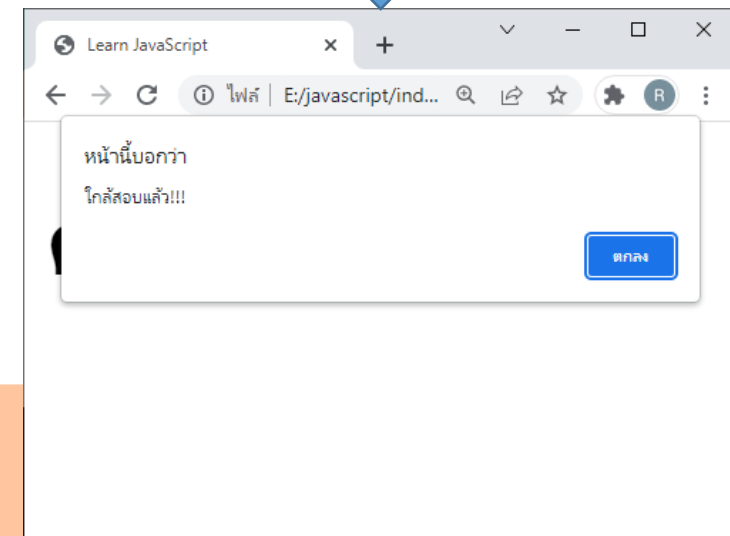
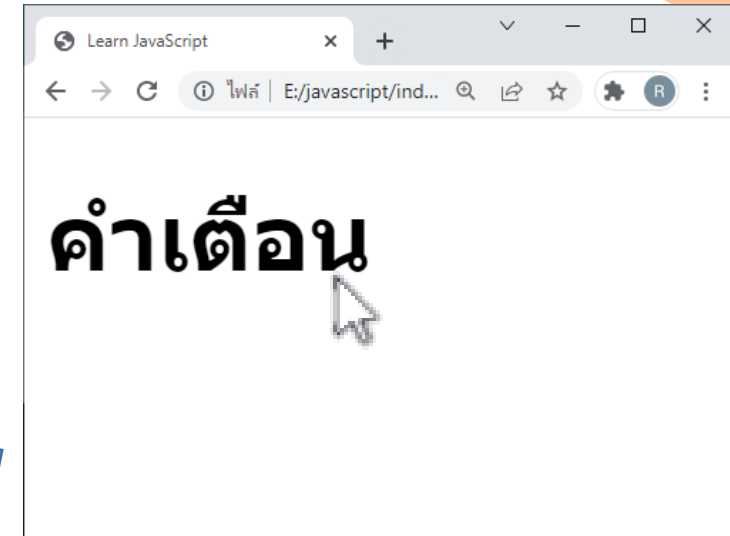
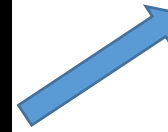


# การกำหนด event ใน HTML



1. ระบุ event ใน HTML
2. สร้างการทำงานเพื่อรองรับการเกิด event ใน JavaScript

```
<script>
 function show() {
 alert("โก๊สสอบแล้ว!!!");
 }
</script>
<h1 onmouseover = "show()">คำเตือน</h1>
```

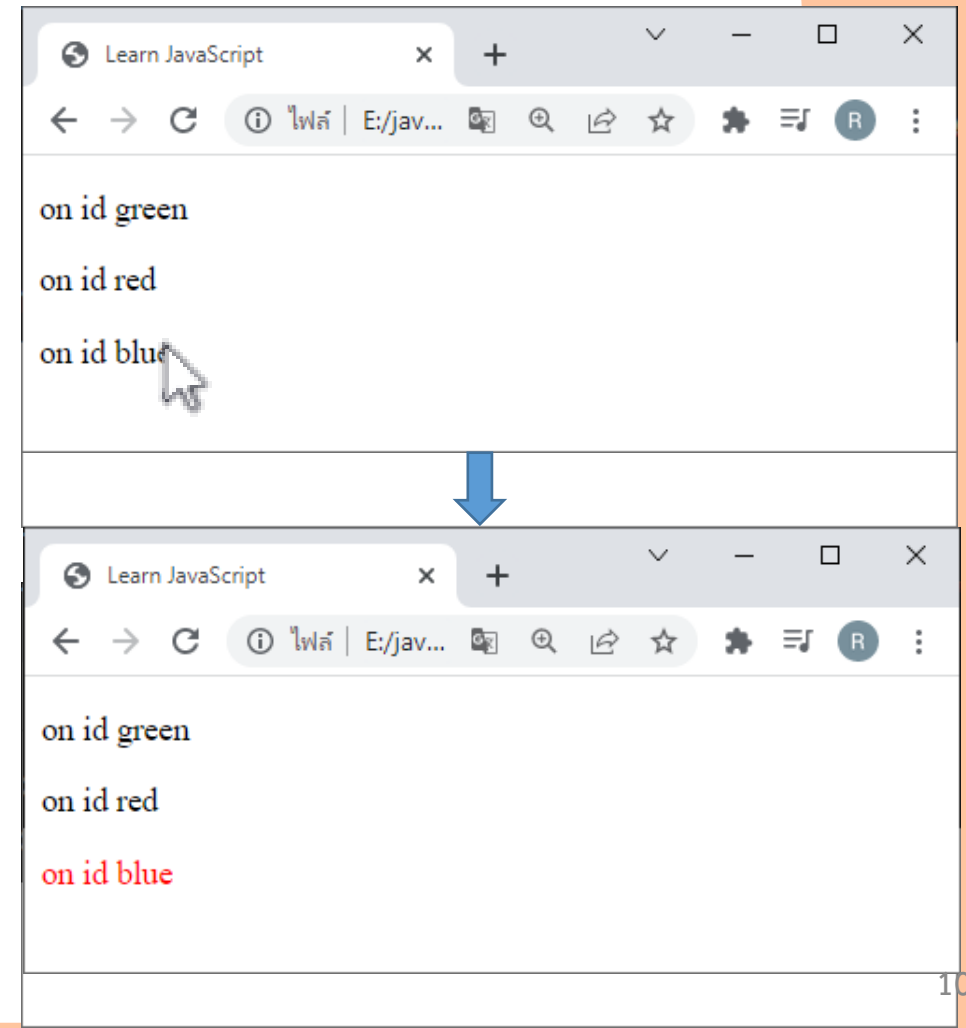


# การเปลี่ยนแปลงค่า CSS Property



- รูปแบบ: `document.getElementById(id).style.attribute = "ค่า css property"`

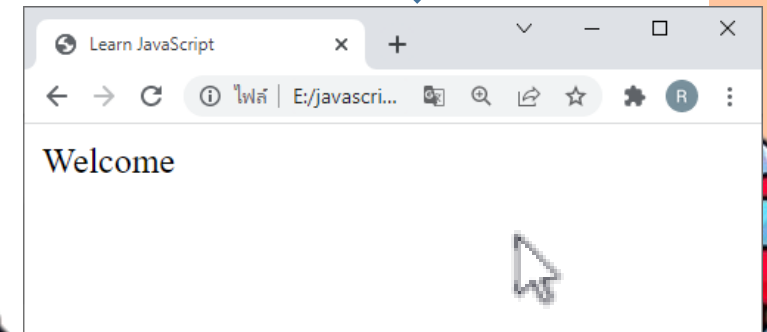
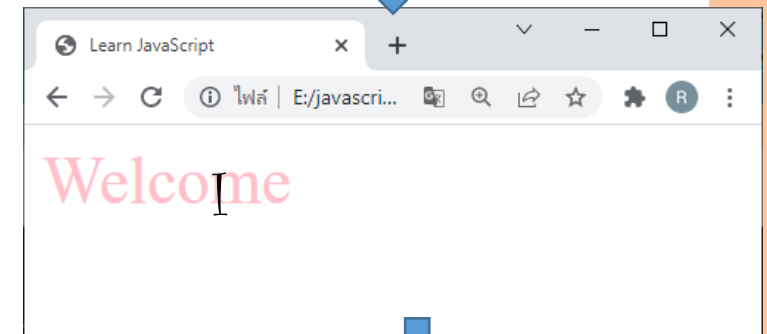
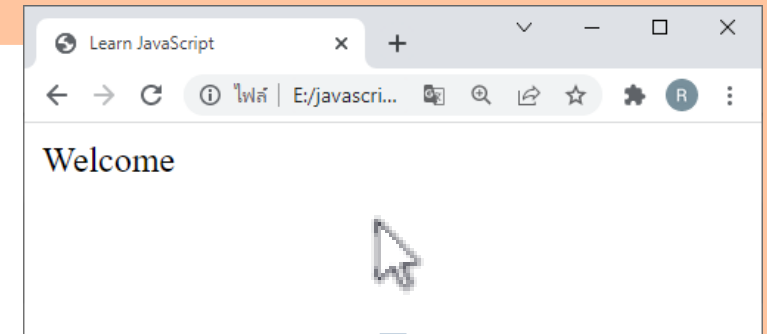
```
<p id="green">on id green</p>
<p id="red">on id red</p>
<p id="blue" onclick="changeColor()">on id blue</p>
<script>
 function changeColor() {
 let p = document.getElementById('blue')
 p.style.color = 'red'
 }
</script>
```



# การเปลี่ยนแปลงค่า CSS Property

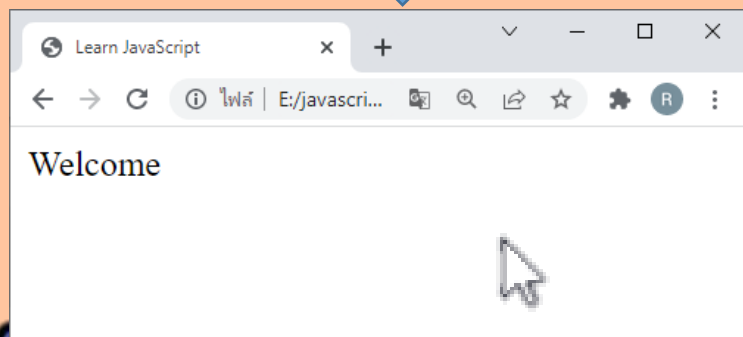
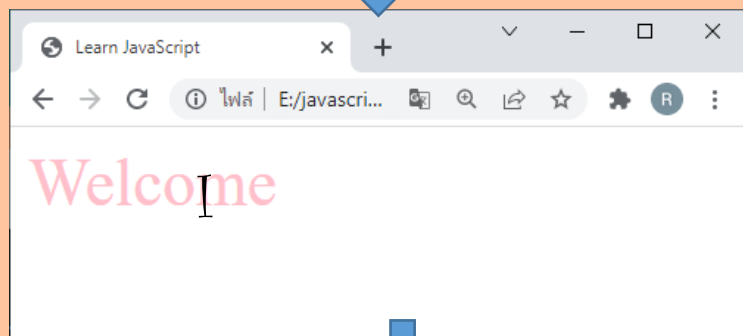
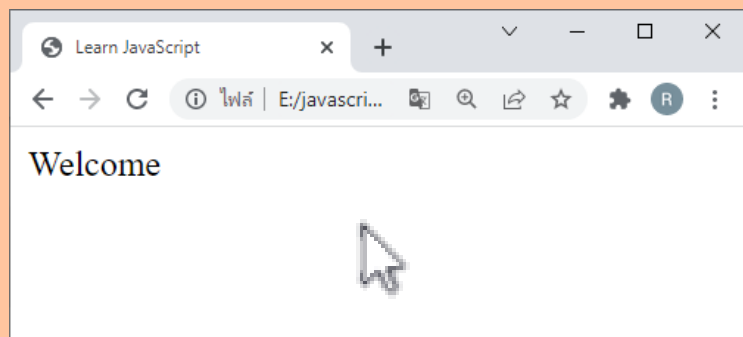


```
<body>
 <script>
 function over() {
 let titleObject = document.getElementById('pageTitle')
 titleObject.style.fontSize = '30px'
 titleObject.style.color = 'pink'
 }
 function out() {
 let titleObject = document.getElementById('pageTitle')
 titleObject.style.fontSize = '16px'
 titleObject.style.color = 'black'
 }
 </script>
 <div id="pageTitle" onmouseover="over()"
onmouseout="out()">Welcome</div>
</body>
```



# การกำหนดคลาส CSS ให้ Element

- ใช้ .classname เข้าถึงชื่อคลาสของ Element



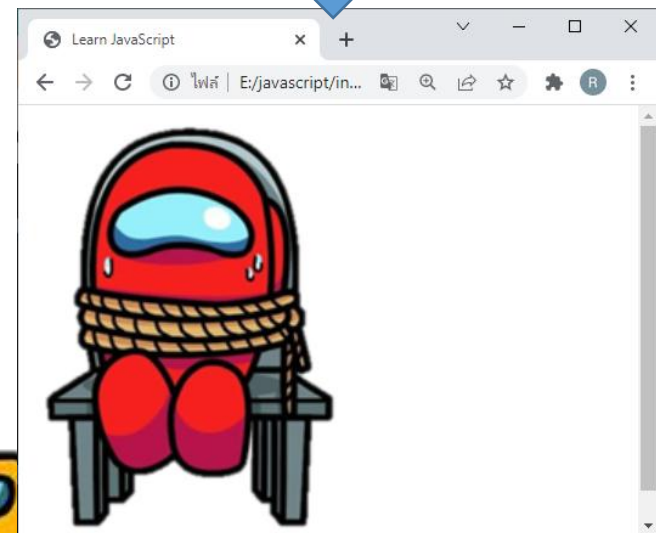
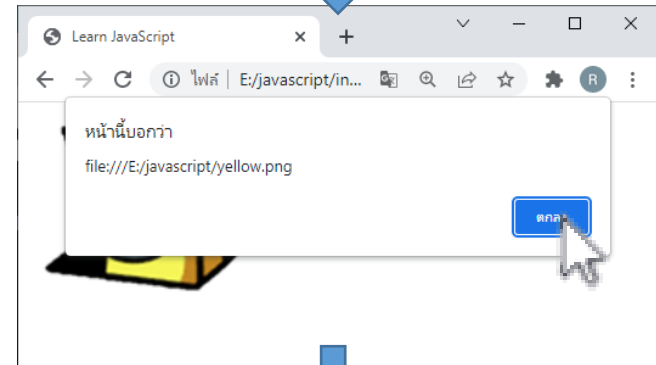
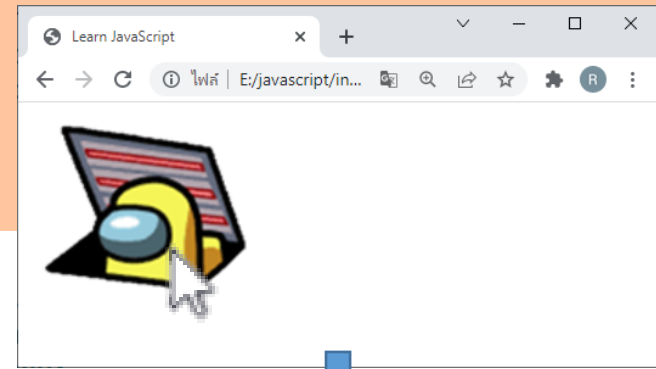
```
<head>
 <meta charset="UTF-8">
 <title>Learn JavaScript</title>
 <style>
 .style1 {
 font-size: 30px;
 color: pink;
 }
 .style2 {
 font-size: 16px;
 color: black;
 }
 </style>
</head>
<body>
 <script>
 function over() {
 let titleObject =
document.getElementById('pageTitle')
 titleObject.className = 'style1'
 }
 function out() {
 let titleObject =
document.getElementById('pageTitle')
 titleObject.className = 'style2'
 }
 </script>
 <div id="pageTitle" onmouseover="over()"
onmouseout="out()">Welcome</div>
</body>
```



# ตัวอย่างการเข้าถึง src ของ<img>

```
<body>
 <script>
 function go() {
 let imgObject = document.getElementById('Image')
 alert(imgObject.src)
 imgObject.src = 'red.png'
 }
 </script>

</body>
```



# ตัวอย่างการดึงค่าจาก form



```
<body>
 <form>
 <input type="text" id="Input1" >+
 <input type="text" id="Input2" >=
 <input type="text" id="Input3" >
 <input type="button" id="sumButton" value="sum"
onclick="sumNum()">
 </form>
 <script>
 function sumNum() {

 let num1 = document.getElementById('Input1').value;
 let num2 = document.getElementById('Input2').value;
 let Ans = document.getElementById('Input3');
 Ans.value = Number(num1)+Number(num2) ;

 }
 </script>
</body>
```

Learn JavaScript

ไฟล์ | E:/javascript/index.html

+  =



Learn JavaScript

ไฟล์ | E:/javascript/index.html

5 + 24 =



Learn JavaScript

ไฟล์ | E:/javascript/index.html

5 + 24 = 29



# การจัดการ Element บน DOM



- หน้าเว็บที่แสดงผลแล้วสามารถสร้าง element ใหม่เพิ่มเติม โดยต่อท้ายกับ element เก่าได้ด้วยคำสั่งใน JavaScript
  - คำสั่งที่ใช้สร้าง element ใหม่ คือ `document.createElement('ชื่อแท็ก');`
  - คำสั่งที่ใช้ในการกำหนดให้ไปต่อท้าย element เก่า คือ `appendChild('ชื่อแท็ก')`



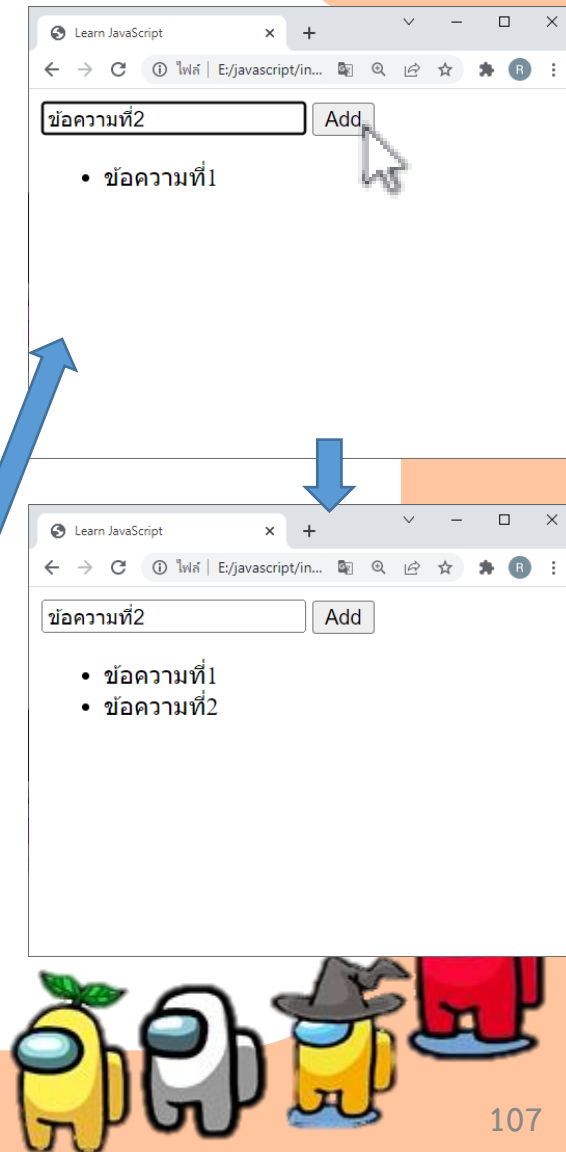
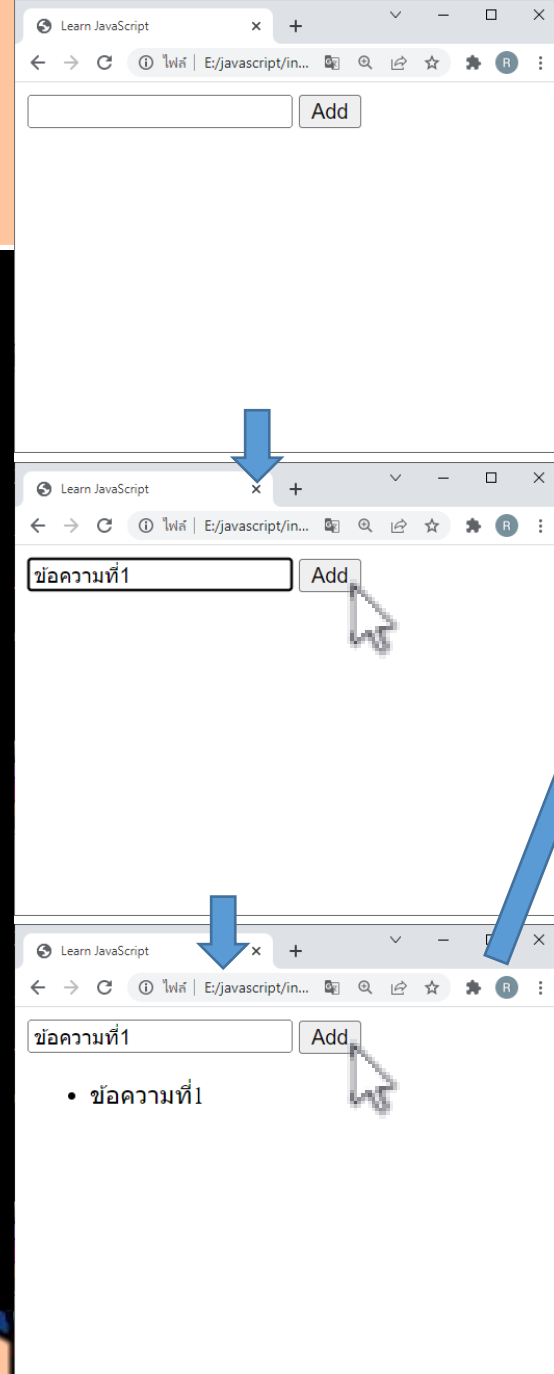
# ตัวอย่างการเพิ่มแท็ก li โดยใช้ JavaScript

```
<body>
 <form>
 <input type="text" id="TextInput" >
 <input type="button" id="addButton" value="Add"
onclick= "addli()">
 </form>
 <ul id="list">

 <script>
 function addli() {
 let textInput = document.getElementById('TextInput')
 let Name = textInput.value

 let li = document.createElement('li') //1. สร้าง
 Element
 li.innerHTML = Name //2. นำค่าไปเก็บไว้ใน Element

 let ul = document.getElementById('list')//3. ดึง
 Element ที่หน้าเว็บ
 ul.appendChild(li)//4. นำ Element ที่สร้างขึ้นไปเป็น Element ลูก
 }
 </script>
</body>
```



# ตัวอย่างการ เพิ่มและลบ แท็ก li โดยใช้ JavaScript

addEventListener() คือรอให้เกิด  
เหตุการณ์ที่กำหนดแล้วทำงานใน  
ฟังก์ชัน

เช่น  
element .addEventListener(  
"click", function(){  
alert("Hello World!"); });

\*ไม่ได้ใช้ "on" คำนำหน้าสำหรับ  
เหตุการณ์;  
ใช้ "click" แทน "onclick"



```
<body>
 <div class="myclass">
 <h1>TODO App</h1>
 <!-- user input -->
 <input class="user_input" type="text" name="mytext" placeholder="Enter text">
 <button class="mybutton" onclick="creation()">Add</button> <!--เมื่อกดปุ่ม ฟังก์ชัน creation() ทำงาน -->
 </div>
 <div>
 <ul class="ul-container">
 </div>

 <script type="text/javascript">

 function creation() {
 const userInput = document.querySelector(".user_input").value; //ดึงค่าจากช่องกรอกข้อมูล
 const ulContainer = document.querySelector(".ul-container"); //เลือกแท็ก ul

 console.log(userInput);
 if (userInput !== "") { //ค่าที่ดึงค่าจากช่องกรอกข้อมูลต้องไม่ใช่ค่าว่าง
 let li = document.createElement("li"); //สร้าง li
 let test = document.createTextNode(userInput); //สร้างโหนดข้อความ
 li.appendChild(test); //เพิ่มโหนดข้อความให้ li
 li.addEventListener('click', function (e) { //รอให้มีการคลิกที่ li
 li.parentElement.removeChild(li); //เลือก parentElement ของ li แล้วลบโหนด li
 })
 ulContainer.appendChild(li); //เพิ่ม li ใน ul
 document.querySelector(".user_input").value = ""; //ลบข้อความในช่องกรอกข้อมูล
 }
 }
 </script>
</body>
```

# ตัวอย่างการ เพิ่มและลบ เท็ก li โดยใช้ JavaScript

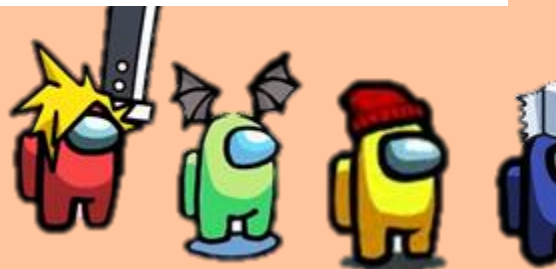
addEventListener() คือรอให้เกิด  
เหตุการณ์ที่กำหนดแล้วทำงานใน  
ฟังก์ชัน

เช่น

```
element .addEventListener(
"click", function(){
alert("Hello World!"); });
```

\*\*ไม่ได้ใช้ "on" คำแนะนำสำหรับ  
เหตุการณ์;

ใช้ "click" แทน "onclick"



The sequence of screenshots shows the following steps:

- The initial state of the TODO App with an empty input field and an 'Add' button.
- The input field is populated with the Thai text 'ทดสอบ1' (Todsob1).
- The 'Add' button is clicked, as indicated by a mouse cursor.
- The final state shows the text 'ทดสอบ1' added to a list below the input field.

# การดึงข้อมูลจาก radiobox

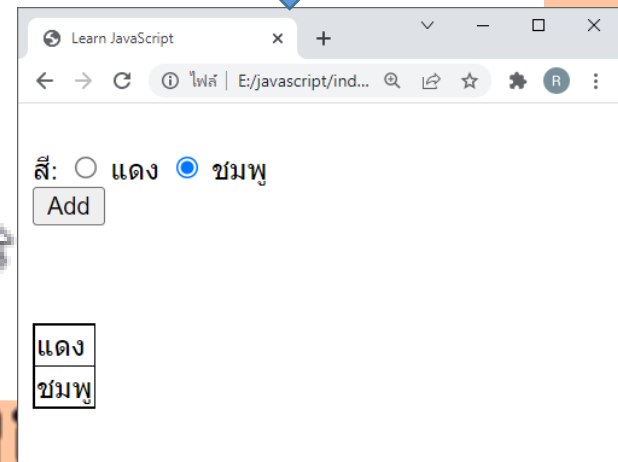
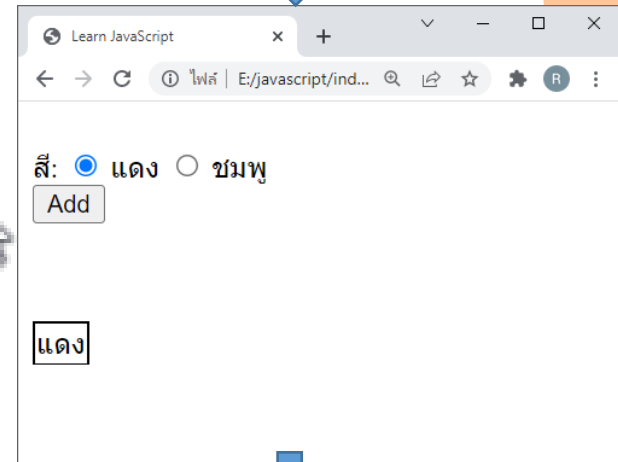
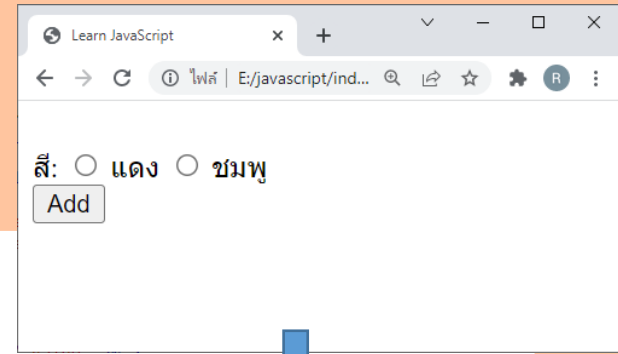
```
<style>
table,td,tr,th{
 border: 1px black solid;
 border-collapse: collapse;
}
```

```
<form>

สี:
<input type="radio" id="red" name="color" value="แดง">
<label for="red">แดง</label>
 <input type="radio" id="pink" name="color" value="ชมพู">
<label for="pink">ชมพู</label>

 <input type="button" id="addButton" value="Add" onclick= "addData()">
</form>

<table id="table1">
</table>
<script>
function addData() {
 let color = document.getElementsByName('color');
 var sizes = color.length;
 for(i = 0; i < sizes; i++) {
 if(color[i].checked==true){
 color =color [i].value;
 }
 }
 let tr = document.createElement('tr');
 let td = document.createElement('td');
 td.innerHTML=color;
 let t = document.getElementById('table1')
 t.appendChild(tr)
 t.appendChild(td)
}
</script>
```



# การดึงข้อมูลจาก radiobox แสดงแบบมีหัวตาราง



- เลือกสีและ กด Add ครั้งแรก

- เลือกสีและ กด Add ครั้งที่ 2

No.3

File | D:/web-1-...

สี: ☐ แดง ☐ ขมพู

Add

No.3

File | D:/web-1-...

สี: ☒ แดง ☐ ขมพู

Add

สี
แดง

No.3

File | D:/web-1-...

สี: ☐ แดง ☒ ขมพู

Add

สี
แดง
ขมพู



## การดึงข้อมูลจาก radiobox แสดงแบบมีหัวตาราง

```
<!DOCTYPE html>
<html lang="th">
<head>
 <meta charset="UTF-8">
 <title>No.3</title>
</head>
<style>
 table,td,tr,th{
 border: 1px black solid;
 border-collapse: collapse;
 }
</style>
<body >
 <form>

สี:
 <input type="radio" id="red" name="color" value="แดง">
<label for="red">แดง</label>
 <input type="radio" id="pink" name="color" value="ชมพู">
<label for="pink">ชมพู</label>

 <input type="button" id="addButton" value="Add"
onclick= "addData()">
 </form>

 <table id="table1">
</table>
```

```
<script>
 let n=0;
 function addData() {
 let color = document.getElementsByName('color');
 var sizes = color.length;
 for(i = 0; i < sizes; i++) {
 if(color[i].checked==true){
 color =color[i].value;
 }
 }
 let t = document.getElementById('table1');
 if(n==0){
 let th = document.createElement('th');
 th.innerHTML='สี';
 let tr = document.createElement('tr');
 let td = document.createElement('td');
 td.innerHTML=color;
 t.appendChild(tr);
 t.appendChild(th);
 t.appendChild(tr);
 t.appendChild(td);

 n++;
 }else{
 let tr = document.createElement('tr');
 let td = document.createElement('td');
 td.innerHTML=color;
 t.appendChild(tr);
 t.appendChild(td);

 n++;
 }
 }
</script>
```

```
</body>
</html>
```



# Workshop

- ใบงานที่ 6
- JavaScript เบื้องต้น



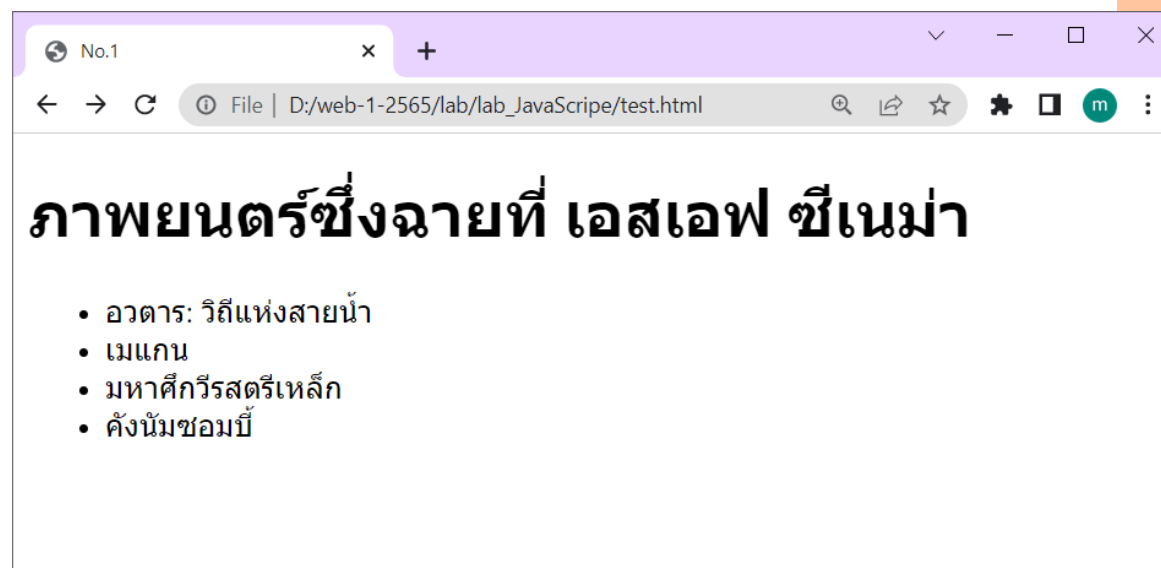


- 1. จากโค้ดด้านล่างมีรายการแบบ unordered list 4 รายการ จงเพิ่มคำสั่งที่ระหว่าง `<script>` เพื่อดึงโหนด DOM ออกมากำหนดค่าให้ `<li>` ให้แสดงข้อมูลตามหน้าเว็บเพจดังรูป

```
<!DOCTYPE html>
<html lang="th">
<head>
 <meta charset="UTF-8">
 <title>No.1</title>
</head>
<body>
 <h1>ภาพยนตร์ซึ่งฉายที่ เอสเอฟ ซีเนม่า</h1>
 <ul id="movie">
 <li id="movie1">

 <li id="movie3">

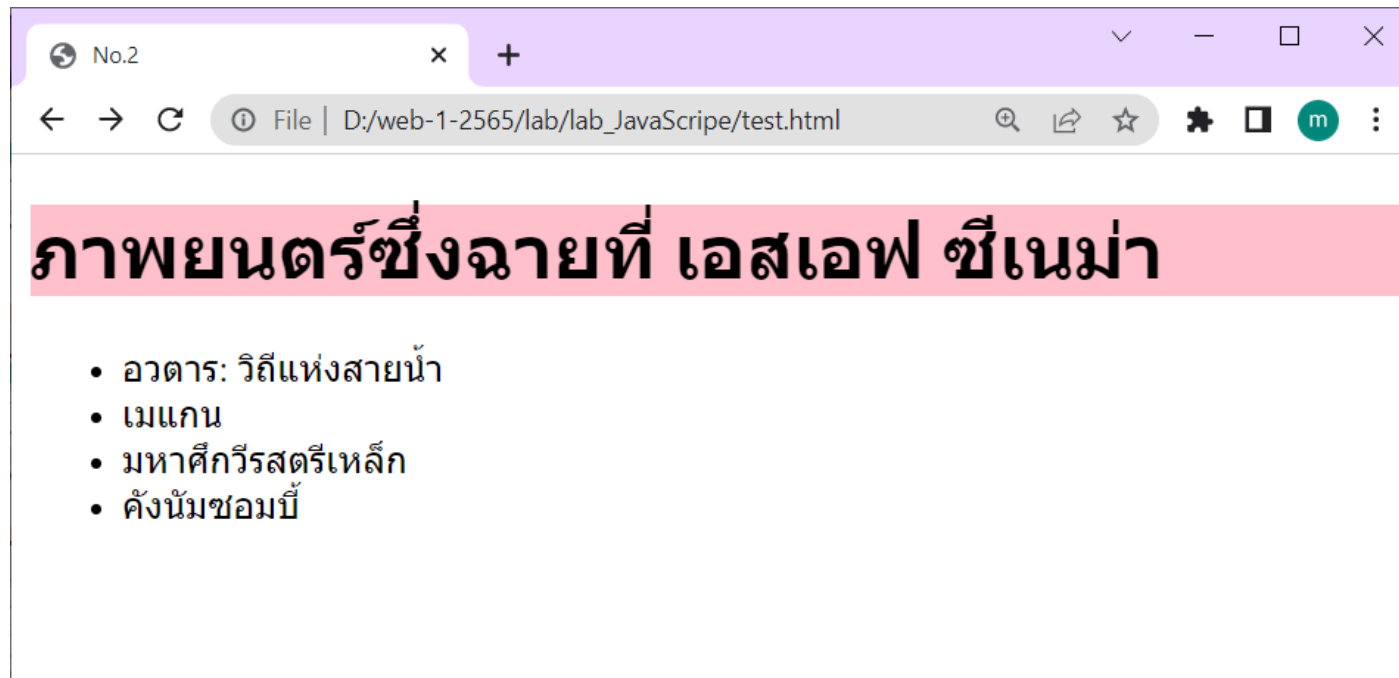
 <script>
 ...
 </script>
</body>
</html>
```



# Workshop



- 2. จากข้อ 1 แก้ไขโค้ด (สร้างไฟล์ใหม่นำโค้ดข้อ 1 มาแก้ไข) ให้มีการกำหนดเหตุการณ์ (Event) และสร้างการทำงานเพื่อรองรับการเกิด event
  - กำหนด Event ในแท็กของโค้ด HTML โดยเมื่อโหลดหน้าเพจให้แสดงหน้าเว็บเพจได้ดังรูป
  - สร้างการทำงานเพื่อรองรับการเกิด event ใน JavaScript เพื่อดึงโหนด DOM ออกมากำหนดค่าให้ `<li>` และใส่สีพื้นหลัง `<h1>`



# Workshop



- 3. สร้างปุ่มใน HTML:

```
<button id="btn">กดฉัน</button>
<p id="count">จำนวนครั้งที่ดับเบิลคลิก: 0</p>
```

- เงื่อนไขทุกครั้งที่มีการ double click (dblclick) ที่ปุ่ม ให้ตัวเลขใน <p> เพิ่มขึ้น 1 แต่ให้กดได้แค่ 5 ครั้ง หลังจากนั้นให้ปุ่ม disabled (กดไม่ได้)

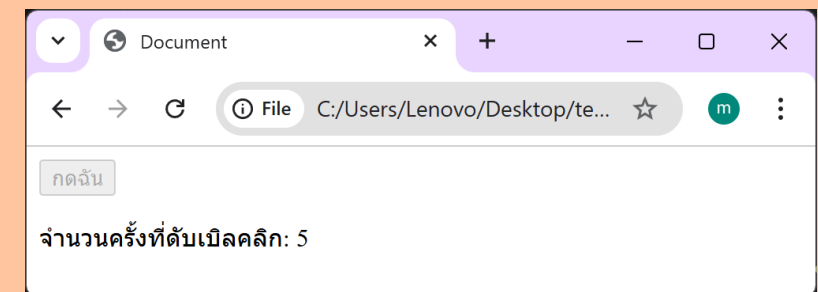
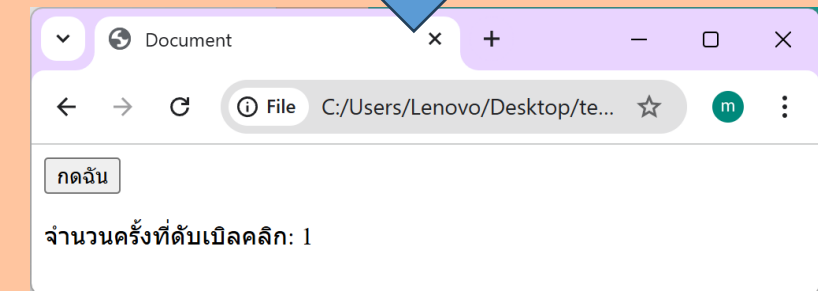
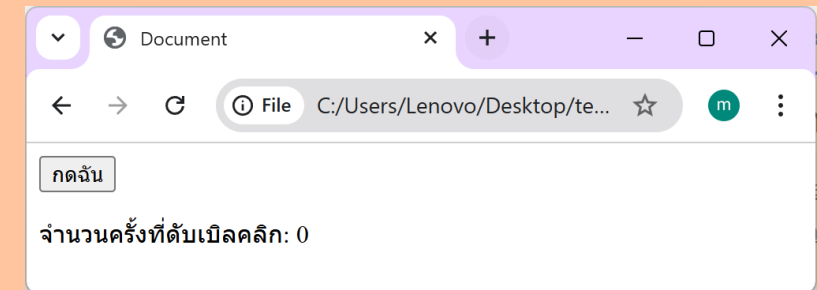
การปิดใช้งานปุ่มสามารถทำได้ 2 วิธี

- 1. ใส่ disabled ตรง ๆ ใน HTML

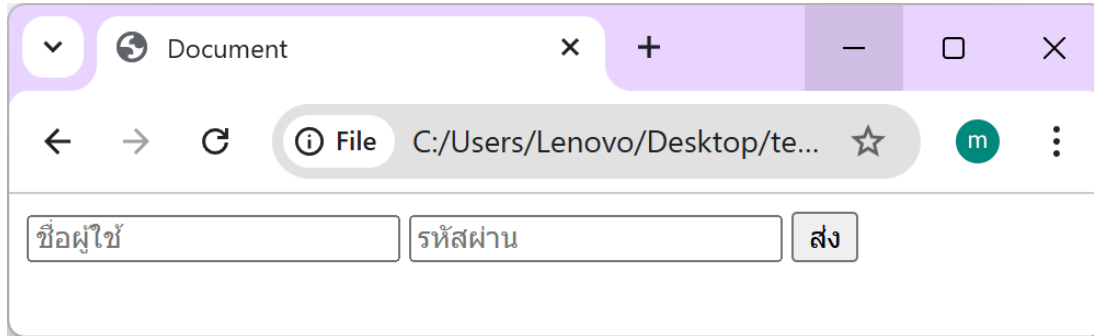
```
<button id="btn" disabled>กดฉัน</button>
```

- 2. ใช้ JavaScript เพิ่ม/เอาออก disabled

```
const btn = document.getElementById("btn");
// ปิดการใช้งาน
btn.disabled = true;
// เปิดใช้งานอีกครั้ง
btn.disabled = false;
```



- 4. สร้างฟอร์มลงทะเบียน



Document

File C:/Users/Lenovo/Desktop/te... ☆ m

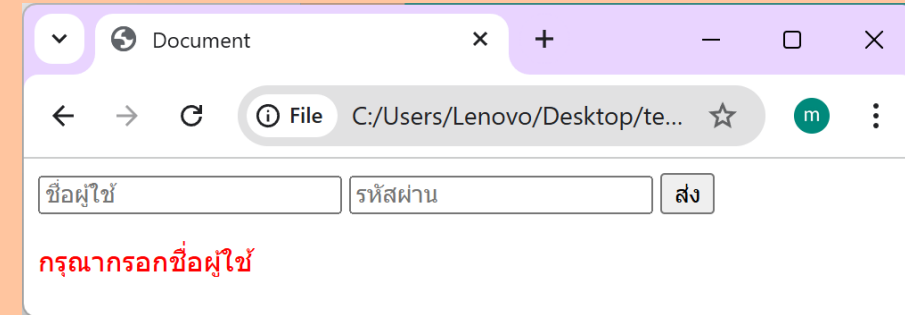
ชื่อผู้ใช้ รหัสผ่าน ส่ง

- เมื่อกด submit:

- ถ้า username ว่าง → แสดง "กรุณากรอกชื่อผู้ใช้"
- ถ้า password น้อยกว่า 6 ตัว → แสดง "รหัสผ่านต้องมียาวอย่างน้อย 6 ตัว"  
สามารถใช้ `value.length` ในการหาขนาดจำนวนตัวอักษรที่กรอกใน input  
เช่น `document.getElementById("password").value.length`
- ถ้า ผ่าน → แสดง "ลงทะเบียนสำเร็จ!"
- ใช้ `event.preventDefault()` ป้องกันไม่ให้ฟอร์มรีเฟรชหน้าเว็บเมื่อกด

```
form.addEventListener("submit", function(event) {
 event.preventDefault(); // ป้องกันการรีเฟรช
});
```

# Workshop

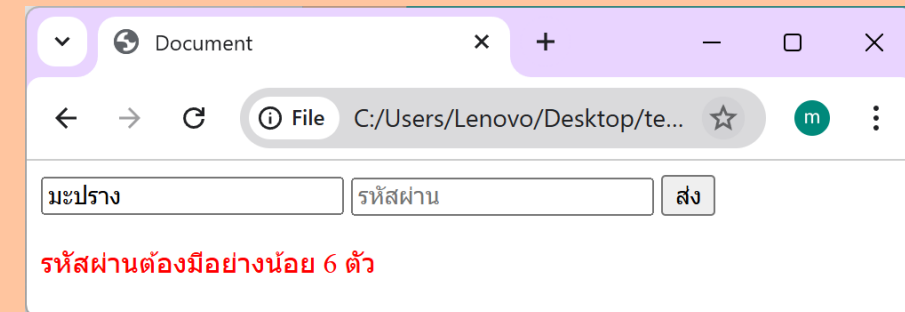


Document

File C:/Users/Lenovo/Desktop/te... ☆ m

ชื่อผู้ใช้ รหัสผ่าน ส่ง

กรุณากรอกชื่อผู้ใช้

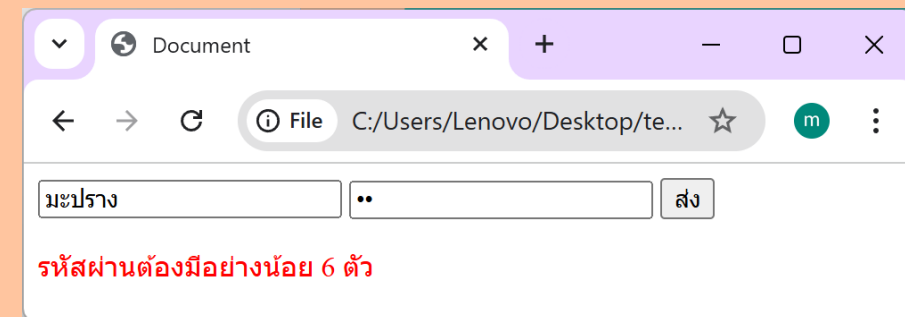


Document

File C:/Users/Lenovo/Desktop/te... ☆ m

มะปราง รหัสผ่าน ส่ง

รหัสผ่านต้องมียาวอย่างน้อย 6 ตัว



Document

File C:/Users/Lenovo/Desktop/te... ☆ m

มะปราง .. ส่ง

รหัสผ่านต้องมียาวอย่างน้อย 6 ตัว

# Workshop

- 5. จงสร้างแบบฟอร์มสำหรับกรอกข้อมูลลูกค้าเมื่อคลิกที่ปุ่ม เพิ่มแล้วให้นำข้อมูลทั้งหมดใส่ลงในตาราง

The sequence of screenshots illustrates the form submission process:

- Screenshot 1:** The form is empty. Fields: ชื่อ (Name), นามสกุล (Surname), เบอร์โทรศัพท์ (Phone Number). Button: Add.
- Screenshot 2:** The form is filled with: ชื่อ: สมรัก, นามสกุล: รักจัง, เบอร์โทรศัพท์: 0998877660. Button: Add. A mouse cursor is over the Add button.
- Screenshot 3:** The form is filled with the same data as Screenshot 2. A table appears below the form with the following data:

ชื่อ	นามสกุล	เบอร์โทร
สมรัก	รักจัง	0998877660
- Screenshot 4:** The form is filled with: ชื่อ: สมปอง, นามสกุล: หมดหวัง, เบอร์โทรศัพท์: 0678844223. Button: Add. A mouse cursor is over the Add button. The table from Screenshot 3 is still present, and a new row is added below it:

ชื่อ	นามสกุล	เบอร์โทร
สมรัก	รักจัง	0998877660
สมปอง	หมดหวัง	0678844223